The `posim` Command Language

Complete Grammar & Notebook Guide

physical_object_simulator

July 21, 2026

Contents

1 What is this document about? 3

2 Lexical structure (what the lexer sees) 3
- 2.1 Token kinds . . . 3
- 2.2 Keywords . . . 4
- 2.3 Magics . . . 4

3 The grammar (what the parser accepts) 5

4 The type system (what values exist) 7
- 4.1 Special functions . . . 8
- 4.2 Comparisons . . . 19
- 4.3 Complex numbers . . . 20

5 Command semantics 20
- 5.1 `NEW` — create an object . . . 20
- 5.2 `SET` / `GET` — write and read any field . . . 22
- 5.3 `STEP` and `RUN` — integrate (always via SUNDIALS) . . . 24
- 5.4 `METHOD` — choose the integrator . . . 24
- 5.5 Observables, bookkeeping, help . . . 25
- 5.6 `SCENE` — the graphical scene window . . . 25
- 5.7 `COLLIDE` and `CONTACTS` — rigid-body collisions . . . 27
- 5.8 `BOX` — the rigid bounding box . . . 27
- 5.9 User definitions: `DEF`, `LET`, `FUNCS`, `SHOW` . . . 28
- 5.10 `QM` — one-dimensional quantum mechanics . . . 30
- 5.11 `QM2` — two dimensions, by ADI . . . 33
- 5.12 `QM3` — three dimensions . . . 35
- 5.13 `DATASET`, `SQL`, `MOVIE` — SQLite recording, queries, and derived movies . . . 37
- 5.14 Coordinate transformations — the fourteen classical curvilinear systems . . . 40

6 The notebook (cells, editing, magics) 42
- 6.1 Cells . . . 42
- 6.2 State is cumulative . . . 42
- 6.3 Magics . . . 43

6.4 Scripts . . . 43

7 JupyterLab in one paragraph **43**

8 The stack machine (what runs your line) **43**

9 Eighteen worked examples **44**
9.1 Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass . . . 44
9.2 Example 2 — a thrown ball vs. the textbook formula . . . 44
9.3 Example 3 — cyclotron motion: expressions as arguments . . . 45
9.4 Example 4 — the tennis-racket theorem (Dzhanibekov effect) . . . 45
9.5 Example 5 — three bodies, then surgery with `DEL` . . . 46
9.6 Example 6 — magnetic torque (and why `ENERGY` grows) . . . 47
9.7 Example 7 — fixing a typo in an old cell with `%edit` . . . 47
9.8 Example 8 — checking the simulator against itself . . . 48
9.9 Example 9 — hand-built tensors and a quaternion orientation . . . 48
9.10 Example 10 — reproducibility: `%save`, `%reset`, `%load` . . . 48
9.11 Example 11 — watching a Kepler orbit live, then running it backward . . . 49
9.12 Example 12 — driving the camera from the notebook . . . 50
9.13 Example 13 — the box of shapes: every body type in a rigid, infinitely massive box 51
9.14 Example 14 — two dumbbells from a user-defined function . . . 53
9.15 Example 15 — a particle in a box, solved inside the language . . . 54
9.16 Example 16 — quantum mechanics: bound states, then tunnelling . . . 55
9.17 Example 17 — tunnelling, with the barrier as a user function . . . 57
9.18 Example 18 — the double slit, in two dimensions . . . 58

10 Error message tour **60**

11 The numerical engine underneath (SUNDIALS 7.8.0) **60**
11.1 Which solver runs which command . . . 61
11.2 What "rootfinding" buys you . . . 61
11.3 All six families, and what reaches each . . . 62

12 Constrained dynamics, equilibrium and sensitivity **62**
12.1 The four joints . . . 62
12.2 `CONSTRAIN` — a rod, not a spring . . . 63
12.3 `EQUILIBRIUM` — where does it come to rest? . . . 64
12.4 `SENSITIVITY` — how much does the answer depend on the input? . . . 64
12.5 The one rule that changed for anyone reading the Rust . . . 65

13 Browser videos: watching a run after it has finished **65**
13.1 How it works, and what it does not do . . . 65
13.2 The player . . . 66
13.3 The thirteen shipped recordings . . . 68
13.4 A trap worth knowing about . . . 69
13.5 What a BALL joint buys, measured exactly . . . 69
13.6 One number decides what a suspension does . . . 69
13.7 The right motion versus the right mechanism . . . 71
13.8 A closed form that is exact, and how to actually hit it . . . 72

13.9 A restart is not a continuation 72
13.10Count your rows before you brace a mechanism 73

14 SQLite datasets and movies: a complete worked session 74

1 What is this document about?

posim is the front end of the `physical_object_simulator`. Instead of writing Rust code to create objects and run physics, you type short *commands* — one per line — into a *notebook*. Each command line is:

1. chopped into *tokens* by a *lexer* (like the classic Unix tool `lex`/`flex`);
2. checked against a *grammar* and compiled into a small program by a *parser* (like `yacc`/`bison`);
3. executed by a *stack machine* — a tiny computer whose only memory is a stack of values — which reads and writes the simulator exclusively through its public get/set functions.

You never see steps 1–3; you type a line, press Enter (or shift-enter in JupyterLab), and see the result. This document specifies exactly what you may type.

There are three ways to talk to the same language:

mode	**start with**	**who it is for**
notebook REPL	`cargo run` (or `posim`)	humans at a terminal
script	`posim -script file`	batch runs, reproducibility
dynamic notebook	`posim -notebook file`	load a notebook, open its scene window, continue live
machine	`posim -machine`	programs, and the JupyterLab kernel

2 Lexical structure (what the lexer sees)

A command line is a sequence of *tokens* separated by optional whitespace. Everything from a `#` to the end of the line is a *comment* and is ignored.

2.1 Token kinds

token	**examples**	**rules**
keyword	`NEW`, `set`, `Run`	case-insensitive; list in §2.2
identifier	`obj0`, `position`	a letter or `_`, then letters, digits, `_`
string	`"ball"`, `"dumbell0"`	double-quoted, with the escapes `\"` `\\` `\n`; strings name objects (`NEW ... AS`, §5.1) and are passed to user functions (§5.9)
number	`2`, `0.5`, `.5`, `1e-3`	64-bit float; scientific notation allowed; a leading `-` is *not* part of the number — it is the negation operator
punctuation	`[ ] { } ( ) , . =`	brackets build vectors, braces enclose initializers, `.` builds dotted paths
operators	`+ - * /`	ordinary arithmetic

If the lexer meets a character it does not know (say `@`), it stops with an error naming the *column* (character position, starting at 1):

```
Err[1]: lexical error at column 10: unexpected character ‘@‘
```

2.2 Keywords

`NEW SET GET DEL DELETE LIST STEP RUN STEPS METHOD ADAMS BDF SPRK ENERGY COM MOMENTUM ANGMOM LAPLACE HELP POINT SPHERE CUBOID TORUS DISK CYLINDER BOX RESET`

and, for the graphical scene window (documented in `scene_info.pdf`; Examples 11–12 show live sessions):

`SCENE CREATE CLOSE DESTROY TRANSLATE ROTATE ZOOM IN OUT HIDE SHOW REFRESH REDRAW START STOP PAUSE REVERSE SET_TIME_STEP SETTIMESTEP STATUS EVENTS ALL`

and, for rigid-body collisions (§5.7):

`COLLIDE CONTACTS ON OFF`

and, for the SQLite dataset layer (§5.13):

`DATASET MOVIE SQL`

(`BALL`, `HINGE`, `UNIVERSAL`, `GEAR`, `RACK`, `PRISMATIC` — and likewise `DATASET`, `MOVIE` and `SQL` — are **contextual**: they are commands only at the start of a line. Anywhere else they are ordinary identifiers, so `new sphere as ball` and `get ball.mass` keep working — `ball` is exactly what a physics user calls a sphere, and reserving it outright would have broken existing notebooks. The same protection means `new sphere as movie` and `let sql = 3` are legal too.)

and, for the quantum families (§5.10, §5.11, §5.12):

`QM QM2 QM3`

and, for named objects, session variables and user functions (§5.1, §5.9):

`AS LET FUNCS DUMBBELL`

(`DESTROY` is an alias for `CLOSE`; `SETTIMESTEP` for `SET_TIME_STEP`; the spellings `CUBE` and `DISC` are lexer aliases for `CUBOID` and `DISK`; `FUNCTIONS` for `FUNCS`; and the single-b spelling `DUMBELL` for `DUMBBELL`. `collisions` is deliberately **not** a keyword so the field path `system.collisions` keeps its own spelling. `DEF` is not a keyword either: a line starting `DEF` is a **line form** recognized before the ordinary grammar — see §3 and §5.9.)

Keywords are reserved *at the start of paths*, but **field names may reuse keyword spellings** — `obj0.momentum` and `system.method` work even though `MOMENTUM` and `METHOD` are keywords, because after a dot (or inside `NEW { ... }`) the parser accepts either an identifier or a keyword as a field name. The same holds for the scene keywords: making `SHOW` or `IN` reserved does not stop you from ever using those spellings as field names after a `..`

2.3 Magics

A line whose first character is `%` is a *notebook magic* (§6.3) and is handled by the notebook itself; it never reaches the lexer.

3 The grammar (what the parser accepts)

The grammar in EBNF (Extended Backus–Naur Form: `[x]` means optional, `{x}` means zero-or-more repetitions, `|` separates alternatives):

```
line     := command | magic ;

command  := "NEW" shape [ "AS" (IDENT | STRING) ]
                        [ "{" init { "," init } "}" ]
                                          (* AS registers a user
                                             name for the object *)
          | "SET" path "=" expr
          | "GET" path
          | "DEL" NUMBER
          | "LIST"
          | "STEP" expr                   (* advance by dt      *)
          | "RUN" expr [ "STEPS" NUMBER ]     (* advance by t, n outs *)
          | "METHOD" ( "ADAMS" | "BDF" | "SPRK" IDENT [ NUMBER ] )
          | "ENERGY" | "COM" | "MOMENTUM" | "ANGMOM"
          | "LAPLACE" NUMBER
          | "RESET" | "HELP"
          | "SCENE" scenecmd              (* graphical scene     *)
          | "COLLIDE" [ "ON" | "OFF" ]        (* bare: report status *)
          | "CONTACTS"                    (* list last contacts  *)
          | "LET" IDENT "=" expr          (* session variable    *)
          | "FUNCS"                       (* list user functions *)
          | "SHOW" IDENT                  (* print a function    *)
          | "BOX" [ "OFF" | expr ]        (* rigid bounding box:
                                             expr = inner side
                                             length; bare = status;
                                             OFF removes it      *)
          | "DATASET" datasetcmd          (* SQLite recording    *)
          | "MOVIE" moviecmd              (* animations derived
                                             from datasets       *)
          | "SQL" STRING                  (* ANY SQLite statement,
                                             or several split on
                                             ';', run verbatim on
                                             the open dataset    *)
          | expr ;                        (* bare expression     *)
datasetcmd := "CREATE" name               (* fresh dataset at
                                             datasets/<name>.db
                                             (replaces an existing
                                             one), opened with
                                             recording armed     *)
          | "OPEN" name                   (* open an existing    *)
          | "CLOSE"                       (* checkpoint + close  *)
          | "RECORD" [ "ON" | "OFF" ]     (* auto-record RUN/STEP
                                             frames; bare=status *)
          | "SNAPSHOT"                    (* record one frame NOW *)
          | "LIST"                        (* names in datasets/  *)
          | "INFO" [ name ]               (* row counts          *)
          | "COPY" name name              (* src -> dst snapshot *)
          | "SAVEAS" name                 (* open dataset -> new *)
          | "DELETE" name
```

```
          | "PLAY" [ name ]                    (* plots in a browser  *)
          | "DIFF" name name "AS" name         (* out = a - b, frame-
                                                  aligned; alignment is
                                                  CHECKED              *)
          | "SUM" name name "AS" name          (* out = a + b          *)
          | "MAP" name "TO" mapsys
                 [ NUMBER [ NUMBER NUMBER ] ]
                 "AS" name ;                   (* curvilinear transform
                                                  of the positions     *)
moviecmd := "CREATE" name [ "AS" name ]        (* dataset -> movie
                                                  page movies/<n>.html *)
          | "DELETE" name | "LIST"
          | "PLAY" name ;                      (* open in a browser   *)
name     := IDENT | STRING ;                   (* quote a name that
                                                  collides with a
                                                  keyword: "box"       *)
mapsys   := "SPHERICAL" | "CYLINDRICAL" | "ELLIPSOIDAL" | "OBLATE"
          | "PROLATE" | "PARABOLOIDAL" | "ELLIPTIC_CYL"
          | "PARABOLIC_CYL" | "BIPOLAR_CYL" | "TOROIDAL"
          | "BISPHERICAL"                      (* three-dimensional   *)
          | "POLAR2" | "ELLIPTIC2" | "BIPOLAR2" ;
                                               (* two-dimensional, in
                                                  the plane z = 0      *)
scenecmd := "CREATE" [ NUMBER ]                (* open window [port]  *)
          | "CLOSE"                            (* aka DESTROY         *)
          | "TRANSLATE" term term [ term ]     (* camera dx dy [dz]   *)
          | "ROTATE" term term                 (* camera dyaw dpitch  *)
          | "ZOOM" ( "IN" | "OUT" | term )     (* factor > 1 zooms in *)
          | "HIDE" [ NUMBER | "ALL" ]          (* default: ALL        *)
          | "SHOW" [ NUMBER | "ALL" ]
          | "REFRESH"                          (* re-sync from state  *)
          | "REDRAW"                           (* re-send full scene  *)
          | "START" | "STOP" | "PAUSE" | "REVERSE"
          | "RESET"                            (* re-initialize: all
                                                  values and the time
                                                  return to initial;
                                                  START re-starts     *)
          | "SET_TIME_STEP" term               (* args are term-level:
                                                  -5 is negative five;
                                                  parenthesize sums   *)
          | "STATUS" | "EVENTS" ;
shape    := "POINT" | "SPHERE" | "CUBOID" | "TORUS" | "DISK" | "CYLINDER"
          | "DUMBBELL" ;                       (* two solid spheres +
                                                  a rigid rod, one
                                                  rigid body          *)

(* User-defined functions are a LINE FORM handled before this
   grammar: DEF name(param [= default], ...) { body } -- the body is
   newline/;-separated commands using the parameters as variables;
   each body line must itself satisfy this grammar. Invocation uses
   the ordinary call syntax name(arg, ...); trailing parameters
   take their defaults. *)
init     := IDENT "=" expr ;
path     := IDENT { "." IDENT } ;              (* objN.field[.x|y|z|w],
```

```
                                                system.field,
                                                contactK.field,
                                                name.field for
                                                AS-registered names *)
expr     := sum { ("<" | "<=" | ">" | ">=" | "==" | "!=") sum } ;
                                       (* comparisons yield 1 or 0, LOWEST
                                          precedence; 1/0 makes
                                          `(x > a) * (x < b)` an indicator
                                          function -- how a piecewise
                                          potential is written *)
sum      := term { ("+" | "-") term } ;
term     := unary { ("*" | "/") unary } ;
unary    := "-" unary | atom ;
atom     := NUMBER | STRING | "[" expr { "," expr } "]" | "(" expr ")"
          | IDENT "(" [ expr { "," expr } ] ")"   (* builtin or user
                                                     function call   *)
          | path
          | IDENT ;                              (* parameter / LET var *)
```

Operator precedence is the usual one: `*` and `/` bind tighter than `+` and `-`; unary minus binds tightest; parentheses override everything.

One subtlety worth learning early: `GET` takes *only a path*. To do arithmetic with a field, use a *bare expression*:

```
get obj0.position.x - 5        # ERROR -- GET wants just a path
obj0.position.x - 5            # OK -- bare expression
```

A second subtlety — `SCENE` arguments are *terms*, not full expressions. The scene commands take several numbers in a row with no commas between them. If those numbers were parsed as full expressions, `scene rotate 15 -5` would be read as *one* argument $15 - 5 = 10$ (whitespace never matters to the lexer) and the command would then be missing its second argument. So scene arguments stop one level lower in the grammar, at `term`: `-5` is negative five, `2*2` and `1/3` still work, but a sum or difference must be parenthesized:

```
scene rotate 15 -5             # yaw +15 deg, pitch -5 deg (two arguments)
scene zoom (1 + 0.5)           # a sum needs parentheses
scene translate 2*2 0 1/2      # * and / are fine unparenthesized
```

A third subtlety — bare identifiers compile and resolve at execution. An `IDENT` alone is an ordinary atom: the constants `pi` and `tau`, a function parameter, or a `LET` variable (§5.9). The name is looked up when the line *runs*, not when it parses, so a function body may use parameters freely; a name that is bound to nothing fails at execution with an error that lists the three ways to bind one (`LET`, a parameter, or a registered object's `name.field` path). A call `IDENT(...)` is a builtin (`dot`, `cross`, `norm`, `normalize`, `sqrt`, `abs`, `sin`, `cos`, `exp`, `log`) when the name matches one, and a user-defined function otherwise.

4 The type system (what values exist)

Every expression evaluates to one of these *values*:

type	written as	notes
number	`2`, `-0.5`, `1e-3`	64-bit float
vec3	`[1, 2, 3]`	exactly 3 numeric entries
quaternion	`[w, x, y, z]`	exactly 4 numeric entries, **w first**; assigned to `orientation` it is renormalized to unit length
mat3	`[[a,b,c],[d,e,f],[g,h,i]]`	3 rows of 3 numbers
string	`"ball"`	a double-quoted literal (§2.1) — names objects (`NEW ... AS`) and feeds user functions; also produced as results like `obj0` or run summaries

Bracket literals are *shape-directed*: 3 numbers make a vec3, 4 numbers make a quaternion, 3 vec3s make a mat3.

Arithmetic is *type-checked*:

operation	allowed
`+`, `-`	number±number, vec3±vec3, quat±quat
`*`	number·number, number·vec3, number·quat, number·mat3, mat3·vec3, mat3·mat3, quat·quat (Hamilton product)
`/`	number/number, vec3/number
vec3 `*` vec3	**forbidden** — the error tells you to use `dot()` or `cross()`

Builtins: `dot(a,b)`, `cross(a,b)`, `norm(v)` (length; also quaternions and numbers), `normalize(v)`, and scalar `sqrt abs sin cos exp log`. Constants: `pi`, `tau` ($= 2\pi$).

4.1 Special functions

The same call syntax reaches the `special_functions` library. Nothing about the grammar changes to accommodate them — the production `IDENT "(" [expr {"," expr}] ")"` already admits any builtin — so adding a function is a *registration*, not a grammar change.

Orders must be whole numbers. Where an argument is an integer order (the n, ℓ, m below), a fractional value is an error rather than being quietly truncated:

```
In[1]:= hermite_h(2.5, 1)
Err[1]: hermite_h(): argument 1 must be a whole number (an integer order), got 2.5
```

That refusal is deliberate. Truncating to `hermite_h(2, 1)` would return a confident, wrong number, and you would have no way to notice.

family	functions
spherical Bessel	`sph_j(n,x)`, `sph_y(n,x)`, `sph_j_prime(n,x)`, `sph_y_prime(n,x)`
Legendre	`legendre_p(n,x)`, `legendre_p_prime(n,x)`, `assoc_legendre_p(l,m,x)`, `norm_assoc_legendre_p(l,m,x)`
spherical harmonics	`sph_harm(l,m,theta,phi)` → `[re, im]`, `sph_harm_real(l,m,theta,phi)`
orthogonal polynomials	`hermite_h(n,x)`, `hermite_he(n,x)`, `laguerre_l(n,x)`, `laguerre_l_assoc(n,alpha,x)`, `chebyshev_t(n,x)`, `chebyshev_u(n,x)`, `gegenbauer_c(n,alpha,x)`, `jacobi_p(n,alpha,beta,x)`
cylindrical Bessel	`bessel_j(n,x)`, `bessel_j_array(n_max,x)` → list
cylindrical Bessel, **complex argument**	`bessel_j_z(n,z)`, `bessel_i_z(n,z)`, `bessel_y_z(n,z)`, `bessel_k_z(n,z)`
cylindrical Bessel, **any real order**	`bessel_j_nu(nu,z)`, `bessel_i_nu(nu,z)`, `bessel_y_nu(nu,z)`, `bessel_k_nu(nu,z)`
Hankel (travelling waves)	`hankel_h1_z(n,z)`, `hankel_h2_z(n,z)`, `hankel_h1_nu(nu,z)`, `hankel_h2_nu(nu,z)`
Hankel derivatives	`hankel_h1_prime_z(n,z)`, `hankel_h2_prime_z(n,z)`, `hankel_h1_prime_nu(nu,z)`, `hankel_h2_prime_nu(nu,z)`
spherical Hankel	`sph_hankel_h1(n,x)`, `sph_hankel_h2(n,x)`, `sph_hankel_h1_prime(n,x)`, `sph_hankel_h2_prime(n,x)`
scaled forms	`bessel_j_scaled(nu,z)`, `bessel_y_scaled(nu,z)`, `bessel_i_scaled(nu,z)`, `bessel_k_scaled(nu,z)`, `hankel_h1_scaled(nu,z)`, `hankel_h2_scaled(nu,z)`
gamma, complex argument	`gamma_z(z)`, `ln_gamma_z(z)`, `rgamma_z(z)`
Airy, complex argument	`airy_z(z)` → `[Ai, Ai', Bi, Bi']`
quadrature	`gauss_legendre(n)` → `[nodes, weights]`
eigenproblems	`eigenvalues(matrix)` → list, `jacobi_eigen(matrix)` → `[values, vectors]`
angular momentum	`wigner_3j(j1,j2,j3,m1,m2,m3)`, `wigner_6j(j1,j2,j3,j4,j5,j6)`, `wigner_9j(a,b,c,d,e,f,g,h,i)`, `clebsch_gordan(j1,m1,j2,m2,j3,m3)`
linear algebra	`solve_tridiag(sub, diag, sup, rhs)` → list, `solve_tridiag_c(...)` → list, `solve_cyclic_tridiag_c(sub, diag, sup, rhs, bl, tr)` → list
utility	`rel_err(a, b)`

`wigner_9j` takes its nine arguments **row by row** — it recouples four angular momenta, and is the overlap between coupling (1,2) and (3,4) first versus (1,3) and (2,4) first. It is evaluated as a single sum over 6-j symbols and vanishes when any of the six triads fails to close.

Angular momenta may be half-integers, so `wigner_3j`, `wigner_6j` and `clebsch_gordan` take

plain numbers rather than demanding whole ones — `clebsch_gordan(0.5, 0.5, 0.5, -0.5, 1, 0)` is exactly what you want to write for two spin-$\frac{1}{2}$ particles. A coupling that violates a selection rule returns **0**, which is the mathematically correct answer, not an error; a value that is not an angular momentum at all ($j = 0.3$, or a negative j) *is* an error.

Complex-argument Bessel

`bessel_j_z(n, z)` and `bessel_i_z(n, z)` take and return **complex** values — they exist because the language has `Value::Complex` (§4.3). Real arguments promote.

The algorithm is the same Miller downward recurrence as the real routine, deliberately: the three-term recurrence and the normalisation $J_0 + 2(J_2 + J_4 + \cdots) = 1$ are both identities in z, real or not, the second following from the generating function at $t = 1$.

Accuracy of J falls with the imaginary part, by cancellation rather than anything about the recurrence: the terms grow like $e^{|\mathrm{Im}\, z|}$ while their sum is exactly 1, so about $|\mathrm{Im}\, z|/\ln 10$ decimal digits are lost. Measured against the generating-function identity:

$\lvert\mathrm{Im}\, z\rvert$	0	5	8	12	18	25
relative error of J	10^{-16}	10^{-14}	10^{-13}	10^{-11}	10^{-9}	10^{-6}

That law governs J alone, and an earlier version of this manual let it stand for the whole family. It does not. I is J at right angles and obeys the mirror law, but Y comes from an ascending series and K is assembled from J and Y at imaginary argument. A series fails where a recurrence does not — on the **real** axis, which is precisely where J is at its best. There are four laws:

$$\text{relative error} \;\sim\; 10^{-16} e^{L}, \qquad L = \begin{cases} |\mathrm{Im}\, z| & \texttt{bessel_j_z} \\ |\mathrm{Re}\, z| & \texttt{bessel_i_z} \\ |z| - |\mathrm{Im}\, z| & \texttt{bessel_y_z} \\ \max(2|\mathrm{Re}\, z|,\ |z|) + \mathrm{Re}\, z & \texttt{bessel_k_z} \end{cases}$$

Measured against Cephes on the axis where each is at its worst:

x (real)	1	10	20	30	35
`bessel_j_z(0,x)`	1e-16	3e-16	5e-16	2e-15	7e-16
`bessel_j_z(0,ix)`	2e-16	1e-13	5e-9	5e-5	3e-3
`bessel_i_z(0,x)`	2e-16	1e-13	5e-9	5e-5	3e-3
`bessel_y_z(0,x)`	1e-15	3e-12	3e-8	2e-4	6e-2
`bessel_k_z(0,x)`	7e-16	3e-5	8e8	5e21	7e27

That table is now history rather than guidance. It describes what each *route* costs, and every failure in it has since been fixed by giving the function a second route:

- `bessel_j_z` near the imaginary axis uses $J_\nu(z) = i^\nu I_\nu(-iz)$, putting the argument back near the real axis where I has nothing to cancel;
- `bessel_y_z` near the imaginary axis uses $Y_n(z) = i^{n+1} I_n(w) - (2/\pi) i^{-n} K_n(w)$ with $w = -iz$, instead of the upward recurrence in n — the wrong direction there, because $Y_n(iy)$ is mostly $I_n(y)$ and I is the recessive solution of that recurrence;
- `bessel_y_z` along the real axis and `bessel_k_z` everywhere use their own $1/z$ expansions, single series with nothing to cancel;

- `bessel_j_z` and `bessel_y_z` in the wedge either side of the **negative real axis** continue the Hankel expansions from the positive one (DLMF 10.11.3, 10.11.4). Those expansions have sectors ending at $\arg z = \pi$, so the cut is the one direction neither reaches directly, and $w = -z$ puts it back where both are at their best. `bessel_k_z` needed only its sector widening: DLMF 10.40.2 is valid to $|\arg z| < 3\pi/2$, so the whole principal sheet was always interior to it.

The routes are chosen by comparing error estimates, so nothing changes in what you write. What changed is the answers:

	before	**after**
`bessel_y_z(0, 40)`	wrong in the first digit	1e-15
`bessel_k_z(0, 20)`	out by a factor of 8e8	2e-16
`bessel_y_z(2, 29.4e^1.6i)`	Wronskian 4.5e-6	below 1e-13

Measured across $|z|$ from 5 to 40 and $\arg z$ over the upper half plane, the J–Y Wronskian residual is now 10^{-10} **or better and mostly below** 10^{-15}.

On the cut itself that Wronskian is the wrong instrument. It is dominated there by the exponentially *recessive* Hankel member, so it measures the Stokes phenomenon rather than the answer. The right test is the pair of exact continuation identities

$$J_n(xe^{i\pi}) = (-1)^n J_n(x), \qquad Y_n(xe^{i\pi}) = (-1)^n\big[Y_n(x) + 2iJ_n(x)\big],$$

which relate a point on the cut to one on the positive real axis, where everything is at its best. Against those, `bessel_j_z` is **exact** and `bessel_y_z` is 10^{-14} out to $|z| = 300$; the wedge either side is 10^{-12} or better, where before it reached 1.0 at $|z| = 60$. The branch jump is still exactly $4i(-1)^n J_n$, and a test says so — widening coverage across a cut is only correct if the cut stays where it was. The J and I rows of the table agree digit for digit because they are the same measurement — $I_n(z)$ *is* $J_n(iz)$.

Two of those defects were found by accident and one by a test written for something else, and in each case the elementary Wronskian settled which side was wrong. The K one is worth stating plainly: its identity $K_n(z) = (\pi/2)i^{n+1}[J_n(iz) + iY_n(iz)]$ **cancels by construction** on the real axis — both terms carry $I_n(x)$, of size e^x, and what survives is e^{-x}. No ingredient was inaccurate. The identity was the wrong way to compute a recessive function.

Y_n and K_n: a different method, because they need one

Y_n has a **logarithmic branch point** at the origin, so no recurrence produces it from J_n alone. It comes from the ascending series (DLMF 10.8.1) for Y_0 and Y_1 — the one carrying the $(2/\pi)\ln(z/2)J_n(z)$ term and digamma coefficients — followed by **upward** recurrence in n.

That direction is the opposite of J's, deliberately: Y_n *grows* with order while J_n decays, so the stable sweep for one is the unstable sweep for the other.

K_n follows by identity, $K_n(z) = (\pi/2)i^{n+1}[J_n(iz) + iY_n(iz)]$ (DLMF 10.27.8). Both are verified by Wronskians, whose right-hand sides are elementary:

$$J_{n+1}Y_n - J_nY_{n+1} = \frac{2}{\pi z}, \qquad I_nK_{n+1} + I_{n+1}K_n = \frac{1}{z}.$$

Both inherit the branch cut of ln along the negative real axis, where they jump by $4iJ_n$ — crossing changes arg by 2π, and the $(2/\pi)\ln(z/2)J$ term turns that into $(2/\pi)(2\pi i)J$. Both are singular at $z = 0$ and report an error there.

Non-integer order

The `_z` family insists on a whole order. The `_nu` family does not: `bessel_j_nu(1.3, 2 + 1i)` is $J_{1.3}(2 + i)$, and half-integer orders — the ones spherical Bessel functions are made of — are the common case. Order stays **real**; complex *order* is not implemented.

Non-integer order needs no recurrence at all, because the two formulas that are hardest at integer order become the easy ones here:

$$Y_\nu(z) = \frac{J_\nu(z)\cos\nu\pi - J_{-\nu}(z)}{\sin\nu\pi}, \qquad K_\nu(z) = \frac{\pi}{2}\,\frac{I_{-\nu}(z) - I_\nu(z)}{\sin\nu\pi}.$$

Both have $\sin\nu\pi$ downstairs, which is exactly why they are useless at whole ν and exactly why Y_n needed its own logarithmic series. So the whole family reduces to **one ascending series** (DLMF 10.2.2 for J, 10.25.2 for I) evaluated at $\pm\nu$, and the reflections do the rest. Orders within 10^{-9} of a whole number are handed to the `_z` routines — not as a convenience but because by then the reflection has already lost nine digits to cancellation. `bessel_y_nu(2, z)` gives the right answer; it is `bessel_y_z(2, z)` underneath.

The series divides by $\Gamma(\nu + k + 1)$ using the **reciprocal** gamma, which is *zero* at the poles. That is what makes $J_{-n}(z) = (-1)^n J_n(z)$ come out for free at whole n, and it keeps very large orders in range where Γ itself would overflow past about 171.

Accuracy is governed by a different law from the `_z` family, and it is important not to carry the `_z` advice over. The largest term of the series is of size $e^{|z|}$, so what is lost is the ratio of that to the answer:

$$\text{relative error} \;\sim\; 10^{-16}e^{L}, \qquad L = \begin{cases} |z| - |\operatorname{Im} z| & \text{for } J_\nu,\ Y_\nu \\ |z| + \operatorname{Re} z & \text{for } I_\nu,\ K_\nu \end{cases}$$

L	0	10	20	30	> 35
relative error	1e-16	1e-12	1e-8	1e-4	nothing left

J and Y are therefore at their **worst on the real axis** and at machine precision straight up the imaginary one — good to $|z| = 70$ there. I and K are the mirror image: worst on the **positive** real axis, exact along the negative. On the real axis that means $|z|$ up to about 30 for J and Y but only 15 for I and K, since K is a *difference* of two Is whose leading parts cancel — that cancellation is what K *is*, not a defect of the method.

Large **order** costs nothing: at $\nu = 150$ the order recurrence still closes to 10^{-13}, because nothing cancels once ν exceeds $|z|$.

Measured, not asserted:

```
cargo run -p special_functions --release --example bessel_nu_accuracy
```

prints the full error surface against the closed forms $J_{1/2}(z) = \sqrt{2/\pi z}\,\sin z$ and $K_{1/2}(z) = \sqrt{\pi/2z}\,e^{-z}$, which hold for complex z and share no code with the series. The bound $10^{-14}e^{L}$

is pinned by the test suite at every point of that surface. Beyond it, use the integer-order routines where the order allows — they are Miller recurrence and reach much further along the real axis. Uniform asymptotics for large $|z|$ at non-integer order are **not** implemented.

```
In[1]:= bessel_j_nu(0.5, 2)
Out[1]= 0.5130161365618277 + 0i
In[2]:= sqrt(2/(pi*2))*sin(2)
Out[2]= 0.5130161365618278
In[3]:= bessel_j_nu(-0.5, 2)
Out[3]= -0.2347857104062484 + 0i
In[4]:= sqrt(2/(pi*2))*cos(2)
Out[4]= -0.2347857104062485
In[5]:= bessel_j_nu(1.3, 2 + 1i)
Out[5]= 0.7246607223671551 + 0.08999135538552433i
In[6]:= bessel_y_nu(2, 1.6 + 0.9i)
Out[6]= -0.6563546517741834 + 0.4094584881356723i
In[7]:= bessel_y_z(2, 1.6 + 0.9i)
Out[7]= -0.6563546517741834 + 0.4094584881356723i
In[8]:= bessel_j_z(1.5, 2)
Err[8]: bessel_j_z(): argument 1 must be a whole number (an integer order), got 1.5
```

`Out[1]`/`Out[2]` and `Out[3]`/`Out[4]` are the two half-integer closed forms agreeing to the last digit, computed two entirely different ways. `Out[6]` and `Out[7]` are byte-identical, which is the whole-order handover doing what it claims. `In[8]` is the contrast: the `_z` form still refuses a fractional order rather than truncating it.

Hankel functions — the travelling-wave pair

J and Y are the *standing*-wave basis of the Bessel equation. $H^{(1)} = J + iY$ and $H^{(2)} = J - iY$ are the *travelling*-wave basis of the same equation, and for wave problems they are the ones you want:

$$H_\nu^{(1)}(x) \sim \sqrt{\frac{2}{\pi x}}\, e^{i(x - \nu\pi/2 - \pi/4)},$$

a pure $e^{+ikr}/\sqrt{r}$ outgoing cylindrical wave under the $e^{-i\omega t}$ convention. A scattering boundary condition is stated in terms of $H^{(1)}$, not in terms of J and Y. The spherical pair `sph_hankel_h1(n,x)` $= j_n(x) + iy_n(x)$ does the same job in three dimensions; it takes a **real** argument and returns a complex value.

The `_z` forms take a whole order, the `_nu` forms any real order, and `_prime` gives the derivative, from $C_\nu'(z) = C_{\nu-1}(z) - (\nu/z)C_\nu(z)$ (DLMF 10.6.2), which holds for every cylinder function and so needs no separate algorithm.

Why these are entry points at all. Each is two calls to functions the language already had, and an earlier version of this manual said so and left it there. That was wrong, for the reason the accuracy note below makes concrete: the assembly $J + iY$ silently destroys most of its digits in half the plane, and a user writing it by hand has no way to know. A named function is where that knowledge lives.

```
In[1]:= hankel_h1_z(0, 3)
Out[1]= -0.26005195490193356 + 0.37685001001279045i
In[2]:= hankel_h1_prime_z(0, 3)
```

```
Out[2]= -0.3390589585259365 - 0.3246744247917999i
In[3]:= hankel_h1_z(1, 3)
Out[3]= 0.3390589585259365 + 0.3246744247917999i
In[4]:= abs(sph_hankel_h1(3, 800) * 800)
Out[4]= 1.0000046875439457
In[5]:= hankel_h1_z(0, 0 + 10i)
Out[5]= 0 - 0.00001131945100496523i
In[6]:= hankel_h2_z(0, 0 + 10i)
Out[6]= 5631.433256931902 + 0.00001131945100496523i
In[7]:= hankel_h1_nu(0.5, 2 - 0.5i)
Out[7]= 0.7802707009570485 + 0.4802074888838277i
In[8]:= sph_hankel_h1(0, 2.3)
Out[8]= 0.32421965746813924 + 0.2896852266434018i
In[9]:= sph_hankel_h2(0, 2.3)
Out[9]= 0.32421965746813924 - 0.2896852266434018i
```

`Out[2]` and `Out[3]` are exact negatives, which is $H_0^{(1)\prime} = -H_1^{(1)}$. `Out[4]` is the outgoing-wave property $|x\, h_n^{(1)}(x)| \to 1$, and the gap from 1 is 4.6875×10^{-6}, which is $n(n+1)/(4x^2)$ for $n = 3$, $x = 800$ to every printed digit. `Out[8]` and `Out[9]` are conjugates, as they must be for real x.

`Out[5]` and `Out[6]` are the accuracy warning in one pair of lines. Both come from the same $J_0(10i)$ and $Y_0(10i)$, each about 2800 in magnitude. $H^{(2)}$ comes out as 5631; $H^{(1)}$ comes out as 1.13×10^{-5}, which is what is left after those two large numbers cancel. Above the real axis $H^{(1)}$ is the small difference of large quantities and is only as accurate as that subtraction allows:

$$\text{rel. error of } H^{(1)} \sim 10^{-16} e^{3\,\mathrm{Im}\, z} \quad (\mathrm{Im}\, z > 0), \qquad H^{(2)} \text{ likewise for } \mathrm{Im}\, z < 0.$$

So $H^{(1)}$ is good to $\mathrm{Im}\, z \approx 8$, has three digits left at 10, and is gone by 12; below the axis it is exact, and $H^{(2)}$ mirrors all of it. **Use whichever of the two is on its good side.** Switching between the `_z` and `_nu` forms does not help — measured, they agree to within a factor of 1.5, because at whole order `_nu` hands Y to `_z` and the two share the dominant ingredient. Only a scaled formulation (returning $e^{-iz}H^{(1)}$ and letting the caller supply the exponential, as AMOS does) removes the problem, and that is **not** implemented.

That $e^{3\,\mathrm{Im}\, z}$ is the same law `bessel_k_z` obeys on the real axis, which is no coincidence: $K_\nu(y) = (\pi/2) i^{\nu+1} H_\nu^{(1)}(iy)$ (DLMF 10.27.8), so they are literally the same computation.

The spherical pair has none of this trouble — j_n and y_n are recurrences on the real line, and `sph_hankel_h1` is accurate wherever they are.

Measured, not asserted:

```
cargo run -p special_functions --release --example hankel_accuracy
```

Scaled forms — the exponential factored out

The three sections above each end in the same place: a small quantity computed from large ones, losing digits it cannot get back. $Y_0(40)$ is wrong in its first digit, $K_0(15)$ is worthless, $H^{(1)}$ above the real axis is gone by $\mathrm{Im}\, z = 12$. Each said the remedy was "a scaled formulation, which is not implemented". It is now.

function	returns
`bessel_j_scaled(nu,z)`	$e^{-\lvert\mathrm{Im}\, z\rvert} J_\nu(z)$
`bessel_y_scaled(nu,z)`	$e^{-\lvert\mathrm{Im}\, z\rvert} Y_\nu(z)$
`bessel_i_scaled(nu,z)`	$e^{-\lvert\mathrm{Re}\, z\rvert} I_\nu(z)$
`bessel_k_scaled(nu,z)`	$e^{z} K_\nu(z)$
`hankel_h1_scaled(nu,z)`	$e^{-iz} H^{(1)}_\nu(z)$
`hankel_h2_scaled(nu,z)`	$e^{iz} H^{(2)}_\nu(z)$

Multiplying by the exponential afterwards would fix nothing — the digits are gone before the multiplication. What makes these work is a different algorithm: the asymptotic expansions of DLMF 10.17 and 10.40, plain series in $1/z$ with leading term 1 and **no cancellation anywhere in them**:

$$e^{-iz} H^{(1)}_\nu(z) \sim \sqrt{\tfrac{2}{\pi z}}\, e^{-i(\nu\pi/2+\pi/4)}\, S(i), \qquad e^{z} K_\nu(z) \sim \sqrt{\tfrac{\pi}{2z}}\, S(1),$$

$$S(c) = \sum_k \frac{c^k a_k(\nu)}{z^k}, \qquad a_0 = 1, \quad a_k = a_{k-1} \frac{4\nu^2 - (2k-1)^2}{8k}.$$

J and Y then come from the Hankel pair without the growing exponential ever being formed, because $e^{\pm iz - \lvert\mathrm{Im}\, z\rvert}$ has modulus at most 1 by construction.

```
In[1]:= bessel_y_scaled(0, 40)
Out[1]= 0.12593641705826092 + 0i
In[2]:= bessel_y_z(0, 40)
Out[2]= 0.09242859229058376 + 0i
In[3]:= bessel_k_scaled(0, 2000)
Out[3]= 0.02802320501460432 + 0i
In[4]:= sqrt(pi/4000)
Out[4]= 0.028024956081989644
In[5]:= bessel_i_scaled(0, 1000)
Out[5]= 0.012617240455891252 + 0i
In[6]:= hankel_h1_scaled(0.5, 3 + 700i)
Out[6]= -0.02127852045069675 - 0.021369909523857591i
In[7]:= bessel_k_scaled(40, 25)
Out[7]= 213915362812.9533 + 0i
In[8]:= bessel_y_scaled(0, 5000)
Out[8]= -0.009116740769643965 + -0i
In[9]:= bessel_k_scaled(0.5, 1e6)
Out[9]= 0.0012533141373154998 + 0i
In[10]:= sqrt(pi/2e6)
Out[10]= 0.0012533141373155003
```

On the real axis the J/Y scaling is $e^0 = 1$, so `Out[1]` *is* $Y_0(40)$ and `Out[2]` is the same number computed the old way. The true value is 0.12593641705826097: the scaled form has fifteen correct digits, the unscaled one none.

`Out[3]` and `Out[4]` are $e^x K_0(2000)$ and its leading asymptotic $\sqrt{\pi/2x}$, agreeing to four digits with the gap equal to $-1/(8x)$ to the digit. **Unscaled, that value does not exist**: $K_0(2000)$ is about 10^{-870}, below the smallest `f64`. `Out[5]` is the same story upward, $I_0(1000) \approx e^{1000}$ being above the largest. `Out[6]` is $H^{(1)}$ seven hundred nepers above the real axis, where J and Y are each about e^{700}. `Out[9]`/`Out[10]` are the exact half-integer form $e^z K_{1/2}(z) = \sqrt{\pi/2z}$ at $z = 10^6$.

Order is handled by recurrence. K, Y, $H^{(1)}$ and $H^{(2)}$ all grow with order, so upward recurrence in ν is stable; the expansion is used at a base order below 1 and stepped up. `Out[7]` is order 40, where the vendored Cephes `kn` overflows outright.

Large order is a separate expansion. A series in $1/z$ at fixed order cannot reach z below ν, where J is exponentially small and was being built as the difference of two exponentially large Hankel values — at $\nu = 400.5$, $z = 240$ that gave an answer wrong by a factor of 5×10^{89}. The remedy is a series in $1/\nu$: the **Debye polynomials** and the uniform expansions of DLMF 10.19 and 10.41,

$$U_0(p) = 1, \qquad U_{k+1}(p) = \tfrac{1}{2}p^2(1-p^2)U_k'(p) + \tfrac{1}{8}\int_0^p (1-5t^2)U_k(t)\,dt,$$

which produce the small number directly. They are chosen automatically by comparing error estimates, so nothing in the language changes; what changes is that the answers are now right. Measured against Cephes over orders 10 to 1000 and z/ν from 0.1 to 2, every value is within 10^{-9} and most within 10^{-14} — and at $\nu = 400.5$ it is *Cephes* that is looser, by 1.4×10^{-9}, which the elementary J–Y Wronskian adjudicates.

When no method reaches a point these return an error naming both estimates. Since the large-order expansions arrived, most such points instead fail for a different and better-stated reason: *the value is outside* `f64`. For z well below ν, J is smaller than the smallest double and Y larger than the largest, and the error says so and quotes the logarithm.

The turning point has its own expansion. Olver's uniform Airy-type expansion (DLMF 10.20) replaces the elementary prefactor with an Airy function, the exact local model of where the equation stops oscillating:

$$J_\nu(\nu x) \sim \left(\tfrac{4\zeta}{1-x^2}\right)^{1/4}\left[\tfrac{\mathrm{Ai}(\nu^{2/3}\zeta)}{\nu^{1/3}}\sum_k \tfrac{A_k(\zeta)}{\nu^{2k}} + \tfrac{\mathrm{Ai}'(\nu^{2/3}\zeta)}{\nu^{5/3}}\sum_k \tfrac{B_k(\zeta)}{\nu^{2k}}\right].$$

Its coefficients are built from the same Debye polynomials, and every term in them is singular at $\zeta = 0$ with the singularities cancelling exactly — which is why near the turning point they come from Taylor series in $1-x$ generated at 70 digits instead. Two of the resulting constants are known independently and match: $A_1(0) = -1/225$ and $\zeta'(1) = -2^{1/3}$.

An earlier version of this manual argued that 10.20 was unnecessary, because across z/ν from 0.95 to 1.1 the other routes already gave 10^{-14}. **That measurement was too coarse and the conclusion was wrong.** At $z/\nu = 0.85$ and 0.90 — not sampled — the error reached 1.4×10^{-9}. With 10.20 in the mix the whole band is 10^{-14}. Complex *order* remains absent.

Measured, not asserted:

```
cargo run -p special_functions --release --example bessel_scaled_accuracy
```

Complex order

Every `_nu` form above takes a **complex** order as well as a real one. `bessel_j_nu(1 + 2i, 3)` is $J_{1+2i}(3)$; a real order reaches exactly the routine it always did, so nothing that worked before changes.

This was the last entry on the list of things chapter 10 did not cover, and the one no expansion could have closed. The obstacle was not an algorithm — the ascending series is the same — it

was that $1/\Gamma(\nu+k+1)$ had no meaning here for complex ν. So the stage is really a **complex gamma**, and the Bessel functions follow.

`gamma_z(z)`	Γ at complex argument
`ln_gamma_z(z)`	still defined where Γ has left `f64`
`rgamma_z(z)`	$1/\Gamma$, **entire** — exactly zero at the poles

`gamma_z` is Stirling with argument shifting, not Lanczos. Lanczos is a little faster and is rejected for the reason that has shaped this crate: its coefficients are a *table*, and the tables in circulation are most often reproduced from *Numerical Recipes*, whose licence this project will not inherit. Stirling needs no table — only the Bernoulli numbers, defined by a recurrence the crate states and a test re-derives.

```
In[1]:= gamma_z(0.5)
Out[1]= 1.7724538509055292 + 0i
In[2]:= sqrt(pi)
Out[2]= 1.7724538509055159
In[3]:= gamma_z(1 + 1i)
Out[3]= 0.4980156681183574 - 0.15494982830181092i
In[4]:= rgamma_z(-3)
Out[4]= 0 + 0i
In[5]:= ln_gamma_z(200)
Out[5]= 857.9336698258575 + 0i
In[6]:= bessel_j_nu(1 + 2i, 3)
Out[6]= 2.616967138257939 + 0.5245621513264704i
In[7]:= bessel_j_nu(0.5, 2)
Out[7]= 0.5130161365618277 + 0i
In[8]:= bessel_k_nu(1i, 2)
Out[8]= 0.09238545989039124 + 0i
In[9]:= bessel_k_nu(-1i, 2)
Out[9]= 0.09238545989039124 + 0i
```

`Out[1]` and `Out[2]` differ in the fourteenth digit, which is the method's honest accuracy: reaching the Stirling regime from $z = 0.5$ takes fourteen shifts and each rounds. `Out[4]` is the reciprocal being *entire*. `Out[5]` is a value `gamma_z(200)` refuses, $\Gamma(200)$ being about 10^{372}.

`Out[8]` and `Out[9]` are the result worth looking at. $K_{iy}(x)$ is real for real y and real positive x — the Macdonald function of imaginary order, which is why it appears as an eigenfunction on the half line. Nothing in the implementation arranges that: K is built from two Is at $\pm\nu$ divided by $\sin\nu\pi$, all three thoroughly complex. It comes out real, and even in the order, because the mathematics says so.

Two things change once the order is complex, both because it now sits in an exponent. $(z/2)^\nu = e^{\nu\ln(z/2)}$, so its *modulus* depends on $\arg z$ as well as $|z|$ — a branch choice is no longer a phase convention. And $\sin\nu\pi$ in the reflections grows like $e^{\pi|\operatorname{Im}\nu|}$, so they get **better** conditioned off the real axis: a whole-numbered real part is not a special case unless the imaginary part is small too. The accuracy law gains one term for the same reason:

$$\text{relative error} \sim 10^{-16}\exp\bigl(|z| - |\operatorname{Im} z| + \operatorname{Im}\nu\cdot\arg z\bigr),$$

so complex order is free on the positive real axis and costs $\operatorname{Im}\nu\cdot\arg z$ elsewhere.

Beyond the series. The $1/z$ asymptotics (DLMF 10.17, 10.40) are expansions at *fixed* order

in which the order enters only as $\mu = 4\nu^2$ — polynomially — so nothing in them assumes a real order and they extend unchanged. The uniform expansions of DLMF 10.41 for I and K extend in the sector $|\arg\nu| < \pi/2$ (DLMF 10.41.5). Both are offered alongside the series and chosen by comparing error estimates, so the reach runs from $|z| \approx 25$ out past $|z| = 600$.

Two things the estimates could not see had to be measured. The $1/z$ expansion needs $|4\nu^2|$ small compared with $|z|$, not merely $|z|$ large — at $\nu = 0.5 + 5i$ and $|z| = 22$ the actual error was **2163 times** the truncation estimate — so that is checked as a validity condition rather than folded into a tolerance. And the term DLMF 10.40.1 drops from I is not of relative size $e^{-2\mathrm{Re}z}$ but $e^{\pi|\mathrm{Im}\nu|-2\mathrm{Re}z}$: **complex order makes it bigger**, by a factor this crate had no reason to carry until now.

The turning point is covered too. Olver's Airy-type expansion (DLMF 10.20) now runs at complex order, using the complex Airy functions of `airy_z`. It needed no new mathematics: near $x = 1$ every ingredient — ζ, the prefactor, and the coefficients A_k, B_k — is already a Taylor series in $w = 1 - x$, generated at 70 digits, and a Taylor series does not care whether its variable is real. What did *not* continue to complex x are the closed forms outside that neighbourhood, where 10.20 is the wrong tool anyway.

The Debye region is covered too, on both sides of the turning point and at complex order as well as real. With $t = \sqrt{1 - x^2}$, $\alpha = \ln((1 + t)/x)$, $q = 1/t$ and $x = z/\nu$,

$$F_\pm(\nu, x) = \frac{e^{\pm\nu(t-\alpha)}}{\sqrt{2\pi\nu t}} \sum_k \frac{(\pm 1)^k U_k(q)}{\nu^k}$$

reads two ways. For $|x| < 1$ these are DLMF 10.19.3/4 directly, $J = F_+$ and $Y = -2F_-$. For $|x| > 1$ — the **oscillatory** region, where t turns imaginary — they continue into the Hankel functions instead, $H^{(1)} = 2F_+$ and $H^{(2)} = 2iF_-$, so $J = F_+ + iF_-$ and $Y = -iF_+ - F_-$. That flip is the Stokes phenomenon, and **the constants were identified by experiment rather than transcribed**: continuing F_+ past $x = 1$ and dividing by each of J, Y, $H^{(1)}$, $H^{(2)}$ in turn showed $F_+/H^{(1)} = \frac{1}{2}$.

That band had no method at all, at real order as well as complex. `bessel_y_z(20, 60)` used to return 10^8; it is now correct to 2×10^{-14}, and across orders 8 to 150 and z/ν from 2 to 30 the whole band is 10^{-14} or better.

What remains is a sliver: $|\nu|$ between about 4 and 8 with $|z|$ a few times larger, where the $1/z$ route is refused for $|4\nu^2|$ and the Debye one for being a $1/\nu$ series at too small an order. Both refusals are deliberate and measured. There is also a narrow ridge near $z/\nu \approx 1.3$ where Y reaches only about 10^{-7} at moderate order.

Airy functions, complex argument

`airy_z(z)` returns all four at once — `[Ai, Ai', Bi, Bi']` — because the routine computes them together and a caller who wants a Wronskian or a turning-point boundary condition wants all four.

```
In[1]:= airy_z(0)
Out[1]= [0.3550280538878172 + 0i, -0.2588194037928068 + 0i,
         0.6149266274460007 + 0i, 0.4482883573538264 + 0i]
In[2]:= airy_z(2 - 3i)
Out[2]= [0.008104457809530868 - 0.131178382604566i,
         0.09665817903311252 + 0.23198718538548577i,
         -0.3963682550403918 + 0.5697309129559497i,
```

```
        0.34945767192946653 + 1.105328588933856i]
```

`Out[1]` is the closed form at the origin: $\mathrm{Ai}(0) = 3^{-2/3}/\Gamma(2/3)$, $\mathrm{Ai}'(0) = -3^{-1/3}/\Gamma(1/3)$ and $\mathrm{Bi}(0) = \sqrt{3}\,\mathrm{Ai}(0)$, the last of which you can read straight off the numbers.

Three regimes, one connection formula. The ascending series (DLMF 9.4) up to $|z| \approx 6$; the asymptotic expansions (DLMF 9.7.5, 9.7.6) in $\zeta = \frac{2}{3}z^{3/2}$ for $|\arg z| \le 2\pi/3$; and nearer the negative real axis the connection $\mathrm{Ai}(z) + \omega\mathrm{Ai}(\omega z) + \omega^2\mathrm{Ai}(\omega^2 z) = 0$ with $\omega = e^{2\pi i/3}$, which rotates $\arg z \sim \pi$ to $\arg \sim \pm\pi/3$ where the expansion is at its best — the same trick the Bessel functions use for their own cut, and it works for the same reason.

Bi needs no expansion of its own past the series: its asymptotic is valid only for $|\arg z| < \pi/3$, and $\mathrm{Bi}(z) = e^{i\pi/6}\mathrm{Ai}(ze^{2\pi i/3}) + e^{-i\pi/6}\mathrm{Ai}(ze^{-2\pi i/3})$ gives it everywhere.

How it is checked. The Wronskian $\mathrm{Ai}\,\mathrm{Bi}' - \mathrm{Ai}'\,\mathrm{Bi} = 1/\pi$ is exact, elementary, and free of both the argument and any transcendental function on the right. Across $|z|$ from 0.1 to 200 and the full range of $\arg z$ the residual is 10^{-11} **or better**, except in one band: between $|z| = 3$ and 10 it is 10^{-9} — the crossover, where Ai is exponentially recessive. The test states that band separately rather than loosening the whole bound to accommodate it. Also checked: the connection formula away from where it is used, conjugation symmetry, the defining equation $\mathrm{Ai}'' = z\,\mathrm{Ai}$ by finite difference, and Cephes on the real axis.

Past $|z| \approx 90$ off the real axis the *dominant* solution leaves `f64` — it grows like $e^{2|z|^{3/2}/3}$ — and `airy_z` says so rather than returning an infinity.

A wrinkle worth knowing about lists. The bracket literal is overloaded: `[a,b,c]` is a *vector*, `[a,b,c,d]` is a *quaternion*, and any other length is a list. That is convenient for physics and would be a nuisance here, so all three shapes are accepted anywhere a numeric list is expected — a 4×4 matrix whose rows are 4-element brackets works exactly as you would expect, and `norm_assoc_legendre_p` never lectures you about quaternions. A 3×3 matrix value is likewise accepted directly wherever a matrix argument is wanted.

4.2 Comparisons

`<`, `<=`, `>`, `>=`, `==`, `!=` compare two numbers and yield **1 for true, 0 for false**. There is no boolean type, and that is the point:

```
In[1]:= 3 < 5
Out[1]= 1
In[2]:= (0.5 > 0) * (0.5 < 1)
Out[2]= 1
In[3]:= (2 > 0) * (2 < 1)
Out[3]= 0
```

`(x > a) * (x < b)` is an **indicator function** for the interval (a, b). Multiply it by a height and you have a rectangular barrier; add two and you have a double barrier. This is how a piecewise potential is written — see §5.10 and Example 17.

Comparisons sit at the **lowest** precedence, below `+` and `-`, so `x + 1 > 2` groups as `(x + 1) > 2`. They are left associative, so `a < b < c` means `(a < b) < c`: legal, and almost certainly not what you meant.

`=` is still assignment; `==` is equality. Ordering is defined for numbers only; `==` and `!=` also accept two vectors or two quaternions, because asking whether two positions coincide is reasonable while ordering them is not.

NaN compares false to everything, including itself, so `x != x` is the idiomatic NaN test:

```
In[4]:= 0/0 != 0/0
Out[4]= 1
```

That is not special-cased — it falls out of IEEE-754.

4.3 Complex numbers

A number with an `i` suffix is imaginary, so `2 + 3i` needs no complex literal syntax of its own — it is ordinary addition of a real and an imaginary:

```
In[1]:= 3i
Out[1]= 0 + 3i
In[2]:= 2 + 3i
Out[2]= 2 + 3i
In[3]:= (2 + 3i) * (2 - 3i)
Out[3]= 13 + 0i
In[4]:= 1 / (0 + 1i)
Out[4]= 0 - 1i
```

`+`, `-`, `*` and `/` all accept complex operands, and reals promote automatically. Note `In[3]`: the result stays typed as complex and displays `13 + 0i` rather than collapsing to `13`. That is deliberate — the type you get out should not depend on whether a particular cancellation happened to be exact.

The suffix only binds when the `i` is not followed by another identifier character, so `2intercept` still lexes exactly as it did before complex numbers existed.

This is what makes the complex Crank–Nicolson solvers reachable. `solve_tridiag_c` and `solve_cyclic_tridiag_` take the same arguments as the real solver and accept a mix — a real band with a complex diagonal and a real right-hand side is fine, since reals promote:

```
In[5]:= solve_tridiag_c([0, 1, 1], [1i, 1i, 1i], [1, 1, 0], [1, 0, 0])
Out[5]= [0 - 0.6666666666666666i, 0.3333333333333337 + 0i, 0 + 0.3333333333333333i]
```

Over the machine bridge (JupyterLab), a complex value is reported as the two-element array `[re, im]`, matching how `sph_harm` reports its value — JSON has no complex type.

5 Command semantics

5.1 `NEW` — create an object

```
NEW POINT    { field = expr, ... }
NEW SPHERE   { field = expr, ... }
NEW CUBOID   { field = expr, ... }        (alias: NEW CUBE)
NEW TORUS    { field = expr, ... }
NEW DISK     { field = expr, ... }        (alias: NEW DISC)
NEW CYLINDER { field = expr, ... }
```

```
NEW DUMBBELL { field = expr, ... }        (alias: NEW DUMBELL)

NEW <shape> AS <name> { field = expr, ... }   (register a user name)
```

Creates one *physical object* and prints its handle (`obj0`, `obj1`, ..., numbered by position). Defaults: mass 1, at the origin, at rest, no charge; sphere radius 1; cuboid half-extents `[1,1,1]`; torus `ring_radius` 1, `tube_radius` 0.25; disk radius 1; cylinder radius 0.5, `half_height` 1; dumbbell `m1` = 1, `m2` = 1, `m_rod` = 0.5, `r1` = 0.25, `r2` = 0.25, `rod_radius` = 0.1, `length` = 1 (total mass 2.5).

Initializer fields: `mass, charge, position, velocity, momentum, orientation, angular_velocity, angular_momentum, radius` (spheres, disks, cylinders), `half_extents` (cuboids), `ring_radius, tube_radius` (tori — or the `inner_radius` + `outer_radius` pair, see below), `height, half_height` (cylinders; `HEIGHT` is the full height = 2·`half_height`), `m1, m2, m_rod, r1, r2, rod_radius, length` (dumbbells, see below), `inertia_tensor, inverse_inertia_tensor, magnetic_moment_tensor, force, torque, id, inverse_mass`.

A torus can be sized two ways: directly (`ring_radius` c = the radius of the circle traced by the tube's center, `tube_radius` a = the tube's own radius) or by the **`inner_radius` + `outer_radius` pair** (the hole's radius and the outermost radius: inner = $c-a$, outer = $c+a$). Inside a `NEW` initializer list the torus geometry is **deferred**: the four radius fields are collected and resolved *once*, at the end of the list — `ring_radius`/`tube_radius` are applied first, then `inner_radius`/`outer_radius` override the derived pair — and the result is validated once, against the *final* values (a bad pair fails with `torus needs 0 <= inner < outer (got inner = 2, outer = 1)`). Giving both members of the pair is therefore **genuinely order-independent**: `{ inner_radius = 1, outer_radius = 2 }` yields ring 1.5, tube 0.5 whichever comes first, and a pair that shrinks the default torus — `{ outer_radius = 0.5, inner_radius = 0.2 }` — works in either order too (checking each write against the half-updated default would have refused one of the two orders). `inner_radius = 0` is legal — that is the **horn torus**, whose tube touches the axis — on `NEW` and on `SET` alike.

The dumbbell is ONE rigid body: two solid spheres joined by a solid rod. Its seven part fields are `m1`, `m2`, `m_rod` (the two sphere masses and the rod mass), `r1`, `r2` (the sphere radii), `rod_radius` (alias `rod_r`) and `length` (alias `len` — the distance between the sphere centers). Like the torus radius pair, the part fields are **deferred**: they are collected during the initializer list and resolved *once* at its end, so they are order-independent and validated once against the final values. The constructor places the local origin at the composite **center of mass** — the sphere centers sit at $z_1 = -(m_2 + m_{\text{rod}}/2)\,L/M$ and $z_2 = +(m_1 + m_{\text{rod}}/2)\,L/M$ on the body z-axis, which makes $m_1 z_1 + m_2 z_2 + m_{\text{rod}}(z_1 + z_2)/2 = 0$ an identity — so `position` is the COM, exactly as for every other shape. The inertia tensor is the exact composite (solid-sphere $\frac{2}{5}mr^2$ terms with parallel-axis shifts, plus the rod), and the surface is the exact union of the three parts. It is the simulator's first non-centrally-symmetric shape; collisions against every other shape decompose exactly over its parts (see `collision_detection.pdf`). The part fields remain readable *and writable* after creation (§5.2).

`AS` registers a user name for the object. The name is a string literal (`NEW SPHERE AS "ball"`) or a bare identifier (`NEW SPHERE AS ball`); a bare identifier is first resolved against a function parameter or `LET` variable holding a string — so a function can name the object it creates from its argument (§5.9, Example 14) — and otherwise the identifier's own spelling is the name. The reply shows both handles: `obj0 as ball`. Named paths then work everywhere a positional path does —

`ball.mass`, `dumbell0.position.x`, in `SET`/`GET` and in bare expressions. Names are case-insensitive identifiers; `system`/`sys` and the positional spellings are refused (`'system' is reserved`, `'obj7' is reserved for positional paths`), and so are duplicates (`the name 'd' already refers to obj0 -- DEL it or pick another name`). The registry follows every renumbering: `DEL` removes the deleted object's name and shifts the names of higher-numbered objects down with their objects, and the `BOX` commands do the same when walls come and go.

Four guarantees make initializers forgiving:

1. **Order does not matter for velocities.** `velocity` and `angular_velocity` are applied *after* mass and inertia are final, so `{ velocity = [1,0,0], mass = 2 }` still yields momentum `[2,0,0]`.
2. **Inertia is computed for you.** For every extended shape the inertia tensor is recomputed from the final mass and shape (solid sphere $\frac{2}{5}mr^2$; cuboid $\frac{m}{3}(h_y^2 + h_z^2)$ etc.; torus $I_z = m(c^2 + \frac{3}{4}a^2)$, $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$; disk $I_z = \frac{1}{2}ma^2$, $I_{xy} = \frac{1}{4}ma^2$; cylinder $I_z = \frac{1}{2}mr^2$, $I_{xy} = m(3r^2 + 4h^2)/12$ with h the half-height) — *unless* you supplied `inertia_tensor` yourself, in which case yours is kept.
3. **Coupled fields stay consistent** (see `SET` below).
4. **`NEW` is transactional.** If any initializer — or the final geometry validation — fails, the half-built object is removed before the error is reported: a failing `NEW` never leaves a ghost behind, and `system.count` and the `objN` numbering are exactly what they were before the command.

5.2 `SET` / `GET` — write and read any field

A *path* is `objN.field`, `system.field`, `contactK.field` (§5.7) or `name.field` for a name registered with `NEW ... AS` (§5.1), optionally followed by a component `.x .y .z` (vectors) or `.w .x .y .z` (quaternions). Component writes are safe read-modify-write operations through the full field's get/set pair. Every object also has six **component shorthands**: `.x .y .z` read and write the position components and `.vx .vy .vz` the velocity components — `ball.x` is `ball.position.x`, `SET ball.vx = 2` is a velocity write.

Object fields (R = readable, W = writable):

field	type	R/W	meaning
`id`	number	RW	user label (no effect on physics)
`mass`	number	RW	writing also updates `inverse_mass` ($m \le 0 \Rightarrow 1/m := 0$)
`inverse_mass`	number	RW	writing back-computes `mass`; 0 makes the body **static**
`charge`	number	RW	Coulombs
`position`, `pos`	vec3	RW	world position
`velocity`, `vel`	vec3	RW	*derived*: reads p/m, writes $p := mv$
`momentum`	vec3	RW	canonical stored linear state
`orientation`	quat	RW	renormalized on write
`angular_velocity`	vec3	RW	derived: $\omega = R\,I^{-1}R^{\mathsf{T}}L$; writes $L := R\,I\,R^{\mathsf{T}}\omega$
`angular_momentum`	vec3	RW	canonical stored angular state (spin)

`inertia_tensor`	mat3	RW	body frame; write updates the inverse (singular → zero inverse = cannot rotate)
`inverse_inertia_tensor`	mat3	RW	write back-computes the tensor
`magnetic_moment_tensor`	mat3	RW	torque $\tau = (RMR^{\mathsf{T}})B$
`radius`	number	RW	spheres, disks, cylinders; write recomputes inertia and **keeps the shape family** (a disk stays a disk, a cylinder keeps its height); a torus is **refused** — `obj6 is a torus - set ring_radius/tube_radius or inner_radius/outer_radius instead of radius` — rather than silently becoming a sphere
`half_extents`	vec3	RW	cuboids only; write recomputes inertia
`ring_radius`, `tube_radius`	number	RW	tori only; write recomputes inertia
`inner_radius`, `outer_radius`	number	RW	tori only, derived (inner $= c - a$, outer $= c + a$); writing one holds the other fixed; `inner_radius` accepts ≥ 0 (0 = the horn torus; the other radii must be > 0)
`height`, `half_height`	number	RW	cylinders only; `height` is the full height ($= 2 \cdot$`half_height`); write recomputes inertia
`m1`, `m2`, `m_rod`	number	RW	dumbbells only: the two sphere masses and the rod mass; writing rebuilds the body — total mass, the COM offsets and the inertia tensor all follow (position is untouched; the stored momenta are preserved — momentum-canonical, like every mass write — so the velocities rescale with the new mass and inertia)
`r1`, `r2`, `rod_radius`	number	RW	dumbbells only: the sphere radii and the rod radius (alias `rod_r`); same rebuild on write
`length`	number	RW	dumbbells only: distance between the sphere centers (alias `len`); same rebuild on write
`x`, `y`, `z`	number	RW	every object: shorthands for `position.x/.y/.z`
`vx`, `vy`, `vz`	number	RW	every object: shorthands for `velocity.x/.y/.z`
`boundary`, `shape`	string	R	description of the shape
`kinetic_energy`, `energy`	number	R	$\frac{1}{2}m\|v\|^2 + \frac{1}{2}\,\omega \cdot L$
`force`	vec3	RW	constant external force during `STEP`/`RUN`

`torque`	vec3	RW	constant external torque
`restitution`	number	RW	collision bounciness $e \in [0,1]$ (default 1 = elastic; a pair uses $\min(e_i, e_j)$)

System fields:

field	type	R/W	meaning
`g_constant`, `g`	number	RW	Newton's G (default 1)
`softening`	number	RW	Plummer length ε (default 10^{-6}); forces use $(r^2+\varepsilon^2)^{3/2}$
`uniform_gravity`, `gravity`	vec3	RW	constant acceleration field
`e_field`	vec3	RW	uniform E; force qE
`b_field`	vec3	RW	uniform B; force $q\,v \times B$, torque $(RMR^{\mathsf{T}})B$
`rtol`, `atol`	number	RW	CVODE tolerances (10^{-10}, 10^{-12})
`time`, `t`	number	RW	current simulation time
`method`	string	R	current integrator
`count`, `n`	number	R	number of objects
`collide`	number	R	1 when collision detection is on (switch with `COLLIDE ON/OFF`)
`contacts`	number	R	contacts recorded by the last `STEP`/`RUN`
`collisions`	number	R	running total of resolved impulses
`restitution_threshold`	number	RW	approach speeds below this bounce with $e = 0$ (default 10^{-3})
`contact_slop`	number	RW	tolerated overlap before positional projection (default 10^{-9})
`box`	number	R	inner side length of the rigid bounding box (§5.8); 0 = none. Create/remove with `BOX`

5.3 `STEP` and `RUN` — integrate (always via SUNDIALS)

`STEP dt` advances time by `dt`. `RUN t STEPS n` advances by `t`, stopping at `n` evenly spaced output points (default 10). Both accept full expressions (`run 2 * pi steps 8` is legal). The reply summarizes the run; `|dE/E|` is the relative change in total energy — your built-in sanity check. **All integration is performed by the pure-Rust SUNDIALS solvers**; there is no hand-rolled stepper.

5.4 `METHOD` — choose the integrator

```
METHOD ADAMS               (default; CVODE Adams-Moulton, adaptive)
METHOD BDF                 (CVODE BDF -- stiff problems, fast gyration)
METHOD SPRK <table> [dt]   (ARKODE symplectic, fixed step; default dt 0.01)
```

Table names may be abbreviated: `leapfrog_2_2` becomes `ARKODE_SPRK_LEAPFROG_2_2`. Useful tables: `EULER_1_1`, `LEAPFROG_2_2` (≡ classic velocity Verlet), `MCLACHLAN_2_2/3_3/4_4/5_6`, `RUTH_3_3`, `YOSHIDA_6_8`.

SPRK requires a *separable* system: point-like translation only. If anything velocity-dependent or rotational is active (a `b_field`, a magnetic tensor, an external torque, a spinning rigid body), `RUN` refuses with an error *naming the offending feature*.

5.5 Observables, bookkeeping, help

command	prints
`ENERGY`	kinetic + softened pairwise potential + uniform-field potentials
`COM`	system center of mass
`MOMENTUM`	total linear momentum
`ANGMOM`	total angular momentum about the origin (orbital + spin)
`LAPLACE` n	Laplace–Runge–Lenz vector of object n about the center of mass, $k = G\,M_{\text{total}}$
`LIST`	one line per object
`DEL` n	removes object n (**later objects renumber!**)
`RESET`	wipes everything (not to be confused with `SCENE RESET`, which re-initializes only the scene window's playback copy — see `scene_info.pdf`)
`HELP`	the quick-reference card

5.6 `SCENE` — the graphical scene window

`SCENE CREATE` starts a tiny web server *inside* posim (pure Rust standard library — no external dependencies) and opens a page in your web browser. That page **is** the scene window: it draws every simulator entity on a 3-D canvas, has a **toolbar** (Start, Pause, Stop, Reverse, a permanent Reset button, single-step, a dt box, zoom, view reset, grid/trails/labels toggles, help) and a **status bar** (connection light, playback mode, simulation time, dt, total energy, body count, hidden count, history depth, camera readout, frames per second). Inside the window:

gesture	effect
← → (arrow keys)	translate the view left / right
↑ ↓	translate the view up / down
left-click + drag	rotate (orbit) around the scene
mouse wheel	zoom in / out
`+` / `-`	zoom in / out from the keyboard
shift-drag or right-drag	translate (pan) with the mouse
Space	start / pause playback
`R`, `G`, `T`, `L`, `H`	reset view, toggle grid / trails / labels, help

The same controls are scriptable from the notebook:

command	effect
`SCENE CREATE [port]`	open the window (all entities shown). Default port: one chosen by the OS; give a number (0–65535) to pin it. Prints the URL. A second `CREATE` is harmless — it just reminds you of the URL.
`SCENE CLOSE` (= `DESTROY`)	shut the server down; every window disconnects

command	effect
`SCENE TRANSLATE dx dy [dz]`	move the camera's look-at point by (dx, dy, dz) world units (`dz` defaults to 0)
`SCENE ROTATE dyaw dpitch`	orbit the camera: yaw (azimuth) and pitch (elevation) in **degrees**; pitch is clamped to $\pm 89°$
`SCENE ZOOM IN / OUT / f`	zoom by $1.25\times$, by $1/1.25$, or by any factor $f > 0$ ($f > 1$ zooms in)
`SCENE HIDE [n|ALL]`	hide object n, or everything (bare `HIDE` = `HIDE ALL`)
`SCENE SHOW [n|ALL]`	undo `HIDE`
`SCENE REFRESH`	copy the notebook's current system into the window (see below) and clear playback history
`SCENE REDRAW`	re-send the complete scene description to every window (forces a full redraw)
`SCENE START`	begin time-stepped evolution, forward
`SCENE PAUSE`	freeze; `START` resumes, history is kept
`SCENE STOP`	halt **and clear the recorded history**
`SCENE REVERSE`	play **backward in time** through the recorded history
`SCENE RESET`	**re-initialize the playback**: every mutable value and the time return to their initial values — the state last synced at `CREATE`/`REFRESH`, restored bit-identically — history and the step counter clear, and the mode returns to *stopped*; `START` then re-runs the simulation from the beginning. The window's permanent toolbar **Reset** button does exactly the same (both call one primitive)
`SCENE SET_TIME_STEP dt`	set the playback time step (must be positive and finite)
`SCENE STATUS`	a four-line report: URL + connected windows; mode/t/dt/steps/history; entities + hidden list; camera
`SCENE EVENTS`	print (and clear) the asynchronous messages the window has sent: errors, connect/disconnect notices, toolbar actions, data requests

The playback copy. The window animates its *own synchronized copy* of your system, evolved by a background thread — your notebook state does **not** move while the animation runs, so `GET obj0.position` still answers instantly and exactly. The copy is taken at `CREATE` and again at every `REFRESH`. All forward stepping goes through the same SUNDIALS integrators as `STEP`/`RUN` — there is no separate physics engine in the window.

Playback is a four-state machine (*stopped* → *running* ⇄ *paused*, plus *reversing*): `STOP` clears history and a later `START` begins fresh; `PAUSE` keeps history so both `START` (forward) and `REVERSE` (backward) can continue from the freeze point. `RESET` is stronger than `STOP`: besides clearing history it puts every value back to its initial state, so the next `START` replays the simulation from the very beginning rather than continuing from wherever playback halted. The reply confirms it:

```
In[6]:= scene reset
Out[6]= scene playback reset to its initial state (t = 0, 1 entity); Start runs the
    simulation again from the beginning
```

How REVERSE works. While running forward, the playback thread records a snapshot of

the whole system before every step (a ring buffer, at most 20 000 frames). `REVERSE` replays those snapshots newest-first, which is an *exact* rewind — bit-for-bit the states you already visited. When the buffer runs out it pauses and sends an event (`reverse: reached the beginning of recorded history -- paused`). Reversing with no history at all is refused with an error.

Asynchronous events. The window can talk back at any time — it reports JavaScript errors, connections, disconnections, and every toolbar action. These messages queue up inside posim; `SCENE EVENTS` drains the queue (up to 1000 messages are kept). In JupyterLab they also appear *by themselves*, without you asking — see §7.

5.7 `COLLIDE` and `CONTACTS` — rigid-body collisions

```
COLLIDE            (report: on/off, collidable pairs, impulses so far)
COLLIDE ON         (default - collisions are detected in EVERY scene)
COLLIDE OFF        (pure point-mass/gravity studies)
CONTACTS           (list every contact of the last STEP/RUN)
```

When collisions are on (the default) and the system contains at least one collidable pair (spheres, cuboids, or a point against either — two points cannot collide), `STEP` and `RUN` detect impacts **during** the time step by SUNDIALS *event rootfinding*: the integrator itself lands on the instant where a pair's signed separation crosses zero, interpolated to solver precision. At that instant an impulse acts along the **contact normal** — the line of the action–reaction force pair — and integration continues. Nothing tunnels, and the reply reports what happened.

Every contact of the last `STEP`/`RUN` is recorded and exposed to the rest of the simulation through read-only `contactK` paths:

field	type	meaning
`contactK.i`, `contactK.j`	number	the colliding pair (indices, $i < j$)
`contactK.t`	number	the event time (the root the solver landed on)
`contactK.point`	vec3	world-space contact point
`contactK.normal`	vec3	**unit normal, from `obji` toward `objj`** — the action–reaction line; `objj` receives $+J\hat{n}$, `obji` receives $-J\hat{n}$
`contactK.depth`	number	penetration depth at the event (≈ 0)
`contactK.rel_vel_n`	number	approach speed along the normal (negative = approaching)
`contactK.impulse`	number	scalar impulse magnitude J

Response physics (full derivation in `collision_detection.pdf`): per-object `restitution` $e \in [0,1]$ (1 = elastic default, 0 = perfectly plastic), pair-combined as $\min(e_i, e_j)$; impulse $J = -(1+e)\,(\mathbf{v}_{\text{rel}} \cdot \hat{n}) \,/\, (\hat{n}^{\top} K \hat{n})$ with the effective-mass matrix K including the angular terms, applied through the canonical momentum/angular-momentum setters; static bodies (`inverse_mass` = 0) are immovable walls; approach speeds below `system.restitution_threshold` settle without bounce; leftover overlap beyond `system.contact_slop` is projected out. `SCENE` windows draw each contact normal as an arrow at the contact point (**Contacts** toolbar button or the `C` key).

5.8 `BOX` — the rigid bounding box

```
BOX <size>         (create: six static walls enclosing an inner
                    size x size x size cube centered on the origin)
BOX OFF            (remove the box - its six walls are deleted)
```

```
BOX                (bare: report status)
```

`BOX <size>` builds a closed rigid room out of **six ordinary cuboid objects** — wall slabs behind the planes $x = \pm\,\text{size}/2$, $y = \pm\,\text{size}/2$, $z = \pm\,\text{size}/2$ (half-thickness size/4, centers at $\pm 3 \cdot \text{size}/4$, cross-sections wide enough to cover the corners) — and prints their handles:

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
    obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
    the inside faces
```

Infinite mass is exact, not approximate. The equations of motion never use the mass itself — only the *inverse* mass: velocity is $v = p\,m^{-1}$, and the collision impulse divides by $\hat{n}^\top K \hat{n} = m_i^{-1} + m_j^{-1} + \text{(angular terms)}$ (§5.7). Each wall is created with `inverse_mass` $= 0$ and a zero inverse inertia tensor, so it contributes exactly 0 to every impulse denominator and receives no state writes: bodies bounce off the inside faces elastically while the walls stay **bit-identically at rest**. One measurable consequence: system **momentum is not conserved** inside a box — the infinitely massive walls absorb it without moving (Example 13). `LIST` tags every wall `[wall: static, inverse_mass=0]` (and shows `mass=0` — the canonical stored quantity for a static body is the inverse).

Bookkeeping:

- `GET system.box` reads the inner side length; 0 means no box. The path is read-only — the box is created and removed by the command.
- A second `BOX <size>` **replaces** the existing box: the old walls are removed first and the reply notes `(replacing the previous box)`.
- The walls are ordinary objects with ordinary indices: `DEL` on a wall deletes it like any other object **and dissolves the box** — `system.box` drops to 0 (five walls no longer enclose anything) — but the surviving slabs **stay tracked**: `LIST` keeps their `[wall: static, inverse_mass=0]` tag, and bare `BOX` reports `box: dissolved (a wall was deleted; 5 tracked slab(s) remain - BOX <size> replaces them, BOX OFF removes them)`. The next `BOX <size>` removes the survivors *before* building the new box (the reply notes `(replacing the previous box)`) and `BOX OFF` simply deletes them — either way no orphan slab leaks. Wall indices are tracked through every `DEL` renumbering, so deleting a non-wall object never confuses the box.
- `BOX OFF` replies `box removed (6 wall(s) deleted, indices renumbered)` — or `box: none` if there was nothing to remove; bare `BOX` reports the size and wall handles, or `box: none (BOX <size> creates one)`.
- A `SCENE` window draws the box as a **dashed interior wireframe**; the wall slabs themselves are not drawn as bodies. Creating a box while a window is open appends a reminder to the reply: `(scene window open: SCENE REFRESH shows the box)`. `RESET` re-syncs an open window to the now-empty system **and clears its box wireframe and wall flags** — no stale overlay survives the wipe.

5.9 User definitions: `DEF`, `LET`, `FUNCS`, `SHOW`

```
DEF name(param [= default], ...) { body }     (define a function)
name(arg, ...)                                (call it)
LET name = <expr>                             (session variable)
FUNCS                                         (list signatures; alias: FUNCTIONS)
SHOW name                                     (print a function's source)
```

`DEF` is a line form, not a grammar production. A line that starts with `DEF` is recognized *before* the ordinary grammar (§3): the notebook captures everything up to the closing `}` — interactively it keeps prompting `...:=` for continuation lines until the brace closes; scripts and JupyterLab cells just keep reading — and installs the function. The **body** is a sequence of ordinary commands, separated by newlines or `;`, in which the parameters act as variables. **Every body line is syntax-checked at definition time** (a typo fails the `DEF` immediately, naming the body line), and **defaults are ordinary expressions evaluated once, at definition** (`LET` variables are visible in them).

Calling uses the ordinary call syntax `name(arg, ...)` — the same atom as `sqrt(2)`; a name that is not a builtin is looked up among your functions. Missing trailing arguments take their defaults (an argument with no default must be supplied). Each call pushes a **call frame** binding the parameters (depth cap: 32 — the language has no conditionals, so recursion could never terminate anyway) and runs the body lines through the same compile-and-execute pipeline as typed input. The call **returns the last body line's value** — if that line is a `SET`, the call prints no `Out[n]`, exactly like `SET` itself. A failing body line aborts the call and the error names the function *and* the line; lines that already ran keep their effects (each command's own guarantees still hold per line — e.g. a failing `NEW` inside a body is still transactional), so a partly-run call leaves no half-built object, only completed commands.

Editing is `SHOW` + re-`DEF`. `SHOW name` prints the definition verbatim, exactly as you typed it; redefining the same name replaces the old version and the reply appends `(redefined)`. `FUNCS` lists every signature with its defaults.

`LET name = expr` binds a session variable (reply: `name set`). It is visible in bare expressions, in function bodies, in `DEF` defaults — and, holding a string, it can *name* things: `NEW ... AS` and named paths resolve a bare identifier through the parameter/`LET` bindings first (§5.1). Note `GET` takes only a *path* (§3), so a variable is read back as a bare expression: `g0`, not `GET g0`.

A complete session (genuine output):

```
In[1]:= let g0 = 9.81
Out[1]= g0 set
In[2]:= def drop(name, m = 1, h = 10) {
  new sphere as name { mass = m, position = [0, h, 0] }
  set system.gravity = [0, -g0, 0]
}
Out[2]= function drop(3 parameter(s)) defined -- 2 body line(s)
In[3]:= drop("ball")
In[4]:= get ball.position.y
Out[4]= 10
In[5]:= drop("pebble", 0.1, 2)
In[6]:= get pebble.mass
Out[6]= 0.1
In[7]:= funcs
```

```
Out[7]= drop(name, m = 1, h = 10) -- 2 body line(s); SHOW drop prints it
In[8]:= show drop
Out[8]= def drop(name, m = 1, h = 10) {
  new sphere as name { mass = m, position = [0, h, 0] }
  set system.gravity = [0, -g0, 0]
}
In[9]:= def drop(name, m = 1, h = 100) { new sphere as name { mass = m, position = [0,
    h, 0] }; set system.gravity = [0, -g0, 0] }
Out[9]= function drop(3 parameter(s)) defined -- 2 body line(s) (redefined)
In[10]:= drop("skydiver")
In[11]:= skydiver.y
Out[11]= 100
In[12]:= speed
Err[12]: unknown name ‘speed‘ (define it with LET, pass it as a function parameter, or
    use ‘speed.field‘ for a registered object)
```

What to notice. The function's `name` parameter holds a string, and `new sphere as name` inside the body names each created object from it — `ball`, `pebble`, `skydiver` — so one definition mints a family of named objects. `drop("ball")` used both defaults; cell 9 is the edit loop (copy cell 8's `SHOW` output, change `h`, re-`DEF` — note `;` separating body lines works as well as newlines); calls 3, 5 and 10 print no `Out` because the last body line is a `SET`. Cell 12 is the bare-identifier error: execution-time resolution means the message can list every way to bind a name.

Definition-time and call-time errors are specific: a reserved or builtin name is refused (`DEF: ‘norm‘ is a builtin function and cannot be redefined`), nested `DEF` is refused, an empty body is refused, a missing non-default argument fails as `name(): missing argument ‘p‘ (it has no default)`, too many arguments fail naming the signature, and the depth cap fails as `function call depth limit (32) exceeded`.

5.10 `QM` — one-dimensional quantum mechanics

The `QM` family solves the two problems a 1-D quantum solver is for: **bound states** in an arbitrary potential, and **time evolution** of a wavepacket. A session carries one quantum problem, built up command by command.

command	meaning
`QM`	report what is currently set up
`QM GRID <x_min> <x_max> <n>`	the domain and its n interior points
`QM POTENTIAL ZERO`	free particle
`QM POTENTIAL BARRIER <v0>, <x1>, <x2>`	`v0` on $[x_1, x_2]$, zero elsewhere
`QM POTENTIAL WELL <depth>, <x1>, <x2>`	$-$`depth` on $[x_1, x_2]$
`QM POTENTIAL <function>`	sample a `DEF`ined $V(x)$ onto the grid
`QM MASS <m>` / `QM HBAR <h>`	both default to 1
`QM METHOD CAYLEY`	Crank–Nicolson on the Dirichlet grid — the default
`QM METHOD NASH [LIE|STRANG]`	the Bessel-stencil split-operator scheme, **periodic**
`QM STATES <k>`	the k lowest bound-state energies
`QM STATE <n>`	load bound state n as ψ
`QM PACKET <x0> <sigma> <k0>`	a normalised Gaussian wavepacket
`QM STEP <dt>` / `QM RUN <t> [STEPS <n>]`	propagate with the current `QM METHOD`
`QM TRANSMISSION <e>`	$T(E)$ and $R(E)$ at one energy, by transfer matrix
`QM SCAN <e1> <e2> <n>`	scan $T(E)$, report resonances, push the list
`QM NORM` / `QM ENERGY` / `QM POSITION` / `QM MOMENTUM`	observables
`QM PROB <a> <b>`	probability of being found in $[a, b]$
`QM DENSITY`	$\|\psi\|^2$ as a list
`QM DRIVE <shape> <modulation>`	time-dependent $V(x,t)$ += $f(t)g(x)$
`QM DRIVE OFF`	back to a static potential
`QM ABSORB <width> <strength> [<power>]`	absorbing edges; `power` defaults to 2
`QM ABSORB OFF`	back to reflecting walls
`QM ANIMATE "<file>" <t> [FRAMES <n>]`	write a self-contained HTML animation
`QM RESET`	forget the quantum problem

Five things are worth knowing before you start, because each will otherwise cost you an afternoon.

`QM METHOD` changes the boundary condition, not just the algorithm. `CAYLEY` is Crank–Nicolson on the Dirichlet grid: the walls reflect. `NASH` is the Bessel-stencil split-operator scheme ported from the original C++, and it is **periodic** — a packet leaving the right edge re-enters at the left. The grid points and the potential samples are identical either way, so switching moves nothing; only the two ends change meaning. The status line always states which is in force.

Because bound states are computed with Dirichlet walls, `QM STATES` and `QM STATE` are **refused** while the method is `NASH` rather than quietly handing you eigenstates of a different problem. `QM ABSORB` and `QM DRIVE` are likewise refused under `NASH`: the propagator takes a real, static potential, and an absorber is a complex one. Each refusal names the way out.

`NASH` alone is first order in dt — that is what the original does, so it is the default. `NASH STRANG` is second order for essentially the same cost, and is the better choice unless you are reproducing SolveIt output.

The walls are infinite and they reflect — under `CAYLEY`, which is the default. `QM GRID` pins ψ to zero just outside the domain, which is exactly right for bound states and a trap for scattering: a packet that reaches a wall bounces back and corrupts your transmission number. `QM PACKET`

and `QM RUN` therefore *watch for it* and print a warning when probability accumulates within 5 % of either wall. Take the warning seriously — widen the domain.

The potential is sampled when the command runs. `QM POTENTIAL v` evaluates $v(x)$ at each grid point once and stores the numbers. Editing `v` afterwards changes nothing until you re-issue the command. The status line says so.

Separate negative arguments with commas. `QM POTENTIAL WELL 5 -2 2` reads `5 - 2` as subtraction and then finds only two arguments where three were wanted. Write `5, -2, 2` — or `5 (-2) 2`. Space separation is fine when everything is positive, which is the usual case.

Why `BARRIER` and `WELL` are built in. This language has no comparison operators, so a piecewise potential cannot be written as a `DEF` at all — and a square barrier is *the* canonical 1-D problem. They are a deliberate workaround for a language limitation, not a preference for built-ins. Any smooth potential should be a `DEF`.

Time-dependent potentials

`QM DRIVE` makes the potential depend on time, as $V(x,t) = V_0(x) + f(t)\,g(x)$, built from two `DEF`ined functions.

Why it is factorised rather than a general $V(x,t)$. A general form would need re-sampling at every grid point on every step — for a user-supplied function, thousands of VM calls per step, costing more than the solve they feed. Factorising costs one evaluation per step and covers the physically important cases exactly: a dipole drive $f(t)x$, a shaken trap, a pulse envelope, an adiabatic ramp. A drive whose spatial *profile* changes shape with time is not expressible this way, and that is a real limit rather than an oversight.

The modulation is sampled at the **midpoint** of each step, keeping the scheme second order; sampling at the start would quietly halve it.

Two consequences:

- **Energy is no longer conserved.** A driven system exchanges energy with whatever drives it — physics, not error. `QM RUN` says so, and the $\langle E\rangle$ it reports is that of the *static* potential.
- **Propagation is still unitary.** $H(t)$ is Hermitian at every instant, so each step remains an exact Cayley transform.

For a quadratic potential with a linear drive Ehrenfest's theorem is **exact**: $\langle x\rangle$ obeys the classical equation of motion with no approximation. Driving the oscillator ground state with $f(t) = F_0 \cos\omega t$ and $g(x) = x$ gives

$$x(t) = -\frac{F_0}{1-\omega^2}\big[\cos\omega t - \cos t\big],$$

and the simulation reproduces it — at $F_0 = 0.3$, $\omega = 0.7$, $t = 5$ the notebook gives $\langle x\rangle = 0.716918$ against an analytic 0.717840, with the norm at 1.000000000000146.

Absorbing edges

The reflecting walls are the main practical limit on a scattering run. `QM ABSORB` replaces them with a **complex absorbing potential**: the Hamiltonian becomes $H - iW(x)$ with $W \geq 0$ ramping up smoothly over `width` at each edge, so probability arriving there is *drained* rather than bounced.

Three consequences, all of them reported rather than left to be discovered:

- **Propagation is no longer unitary.** The norm decays. That is the absorber working, so the drift figure printed by `QM RUN` stops being a health check while it is on.
- **`QM STATES` is refused.** $H - iW$ is not Hermitian, and a symmetric eigensolver fed a non-Hermitian matrix returns confident nonsense.
- **The tuning is real.** Too weak and the packet reaches the wall and reflects off *that*; too strong and it reflects off the absorber's own leading edge, because a steep change in the potential is a mirror whether it is real or imaginary. The useful window spans more than an order of magnitude in strength. `cargo run -p quantum --release --example absorber_tuning` measures the whole surface; for $k_0 \approx 3$ the optimum is near width 18, strength 3, giving about 10^{-9} reflection against a plain wall's 1.0.

A worked comparison: the barrier problem gives $T = 0.327563$ on a 200-wide reflecting domain, and $T = 0.327690$ on a **90-wide** domain with `QM ABSORB 15 3` — agreement to 4 parts in 10 000 for less than half the grid.

Bound states are found by diagonalising a dense $n \times n$ matrix with the cyclic Jacobi method, which is $O(n^3)$: comfortable to a few hundred points, slow past about a thousand. Propagation is $O(n)$ per step and has no such limit, so a scattering run can afford a much finer grid than a bound-state calculation.

5.11 `QM2` — two dimensions, by ADI

A *separate* family from `QM`, not a mode on it: a 2-D problem differs in almost every argument list, and a hidden mode that silently reinterprets your commands is worse than a second word.

command	meaning
`QM2`	report the current 2-D setup
`QM2 GRID <x0> <x1> <nx>, <y0> <y1> <ny>`	domain and resolution per axis
`QM2 POTENTIAL ZERO | <function>`	a `DEF`ined $V(x, y)$
`QM2 PACKET <x0> <y0>, <sx> <sy>, <kx> <ky>`	2-D Gaussian packet
`QM2 STATES <k>`	the k lowest bound-state energies
`QM2 STATE <n>`	load bound state n as ψ
`QM2 STEP <dt>` / `QM2 RUN <t> [STEPS <n>]`	ADI propagation
`QM2 NORM | QM2 ENERGY | QM2 CENTROID`	observables (`CENTROID` also spells `POSITION`)
`QM2 PROB <xa> <xb>, <ya> <yb>`	probability in a rectangle
`QM2 DRIVE <shape> <modulation> | OFF`	time-dependent $V(x, y, t)$
`QM2 ABSORB <w> <s> [<p>] | OFF`	absorbing edges, all four walls
`QM2 ANIMATE "<file>" <t> [FRAMES <n>]`	heat-map animation
`QM2 RESET`	forget the 2-D problem

Why ADI, and why not the textbook ADI

In 1-D the Crank–Nicolson operator is tridiagonal, so one solve per step gives an exactly unitary propagator. **In 2-D it is not**: the five-point Laplacian couples each point to neighbours in both

directions and the matrix is block-tridiagonal with bandwidth n_x, costing $O(n_x^3 n_y)$ per step to solve directly.

ADI splits the step by direction, so each half-step is a *set of independent tridiagonal solves* — n_y along x, then n_x along y — and the cost falls to $O(n_x n_y)$.

The textbook scheme is **Peaceman–Rachford**, implicit in x against an explicit y and then swapped. It is second-order and unconditionally stable, but **not unitary**: the two half-steps use different operators, so the norm drifts at $O(dt^2)$ per step.

This implementation applies a **Cayley transform in each direction separately**, composed by Strang splitting:

$$\psi(t+dt) = U_x(dt/2)\, U_y(dt)\, U_x(dt/2)\, \psi(t).$$

Each $A_d = T_d + V/2$ is Hermitian, so each U_d is exactly unitary and so is their product. **The norm is conserved to machine precision for any** dt, while the splitting error stays $O(dt^2)$ in the *dynamics*. Norm conservation therefore remains a sharp check on the linear algebra, independent of the accuracy question.

One subtlety: with $A_x = T_x + V/2$ and $A_y = T_y + V/2$ the commutator $[A_x, A_y]$ is non-zero *even for a separable potential*, so a product eigenstate is not perfectly stationary. The drift is second order in dt and the tests assert that scaling rather than a magnitude.

Bound states, and why they need a different eigensolver

On an $n_x \times n_y$ grid the Hamiltonian is $(n_x n_y)^2$: a modest 200×200 grid gives a $40\,000 \times 40\,000$ matrix, about 12 GB dense before any arithmetic. The Jacobi solver used in 1-D does not apply.

`QM2 STATES` uses **Lanczos**, which never forms the matrix — it needs only Hv, the five-point stencil at $O(n_x n_y)$. It builds a Krylov basis, projects H onto it as a small tridiagonal, and those eigenvalues converge fastest to the *extremes* of the spectrum, which is where bound states are.

Two separate things must go right for degeneracies, and they are easy to confuse:

- **Ghosts.** Lanczos vectors lose orthogonality in floating point as soon as a value converges, and the method rediscovers eigenvalues it already has. Those spurious copies look exactly like degeneracy. Cured by **full reorthogonalisation**.
- **Genuine multiplicity.** Full reorthogonalisation does *not* help. A Krylov space built from one starting vector contains exactly **one** direction from each degenerate eigenspace, so plain Lanczos finds each distinct eigenvalue once however long it runs; no tolerance fixes it. Cured by **deflation**: each converged state is shifted to the top of the spectrum and the solver re-run *from a different starting vector* — reusing the same start would re-derive the direction just removed.

The 2-D isotropic oscillator on a 70×70 grid:

```
In[4]:= qm2 states 6
Out[4]= 6 lowest bound state(s) -- Lanczos, 620 iterations:
  E[0] = 0.9975639778   residual 3.86e-8
  E[1] = 1.9926799032   residual 1.92e-8
  E[2] = 1.9926799032   residual 2.89e-8
  E[3] = 2.9828813394   residual 3.03e-8
  E[4] = 2.9828813394   residual 5.15e-8
```

```
E[5] = 2.9877958286   residual 5.21e-8
```

Against the exact $1, 2, 2, 3, 3, 3$. Note what the grid does to the third level: $E[3]$ and $E[4]$ agree to **ten digits**, because the square grid keeps the $x \leftrightarrow y$ symmetry relating the $(2, 0)$ and $(0, 2)$ states — while $E[5]$, the $(1, 1)$ state, sits 0.005 away because the continuum *rotational* symmetry that would complete the degeneracy is not a symmetry of the grid. The splitting is discretisation, not solver error.

Residuals are printed as part of the answer: an iterative eigensolver has no exact stopping point, and a caller who cannot see how well a state converged cannot know whether to trust it.

Propagation has no comparable limit.

5.12 `QM3` — three dimensions

A third family, for the same reason `QM2` is separate from `QM`.

command	meaning
`QM3 GRID <x0> <x1> <nx>, ..., <z0> <z1> <nz>`	domain and resolution per axis
`QM3 POTENTIAL ZERO | <function>`	a `DEF`ined $V(x, y, z)$
`QM3 PACKET <x0> <y0> <z0>, ...`	3-D Gaussian packet
`QM3 STATES <k> | QM3 STATE <n>`	bound states
`QM3 STEP <dt> | QM3 RUN <t> [STEPS <n>]`	ADI propagation
`QM3 NORM | QM3 ENERGY | QM3 CENTROID`	observables (`CENTROID` also spells `POSITION`)
`QM3 PROB <xa> <xb>, ..., <za> <zb>`	probability in a box
`QM3 DRIVE <shape> <modulation> | OFF`	time-dependent $V(x, y, z, t)$
`QM3 ABSORB <w> <s> [<p>] | OFF`	absorbing faces, all six sides
`QM3 ANIMATE "<file>" <t> [FRAMES <n>]`	three marginal densities, animated
`QM3 ISO "<file>" <t> [FRAMES <n>] [LEVEL <f>]`	rotatable isosurface
`QM3 RESET`	forget the 3-D problem

The scheme is the 2-D one with a third direction, Strang-composed so every factor stays an exact Cayley transform:

$$\psi(t + dt) = U_x(dt/2)\, U_y(dt/2)\, U_z(dt)\, U_y(dt/2)\, U_x(dt/2)\, \psi(t),$$

with $A_d = T_d + V/3$. Five directional sweeps per step instead of three, each a set of independent tridiagonal solves, so a step is still $O(n_x n_y n_z)$ and **the norm is conserved to machine precision for any** dt.

What changes in three dimensions is memory

A 100^3 grid is a million points. One complex wavefunction is 16 MB, which is fine — but the 2-D propagator precomputes full-size band arrays, three per direction, and doing that here would cost about 140 MB before any work began. `QM3` builds its bands **per line, on the fly**, so the only extra storage is $O(\max(n_x, n_y, n_z))$.

The eigensolver has a ceiling, and says so

Propagation is comfortable past 64^3. `QM3 STATES` is not: Lanczos reorthogonalises fully and stores its whole Krylov basis, costing $O(m^2 n)$ in time and $O(mn)$ in memory. The practical ceiling is about $\mathbf{40^3}$, and beyond it the command **refuses up front** rather than exhausting memory. Propagation on the same grid still works.

Looking at a volume

A volume cannot be drawn on a flat canvas without an isosurface mesh or a ray-caster, either of which would mean shipping a WebGL pipeline inside a file that must work from `file://`. `QM3 ANIMATE` shows the three **marginal densities** instead:

$$P(x,y) = \int |\psi|^2 dz, \quad P(x,z) = \int |\psi|^2 dy, \quad P(y,z) = \int |\psi|^2 dx.$$

Each is a genuine observable — the probability of finding the particle at those two coordinates whatever the third — not a rendering convention, and each integrates to the total norm, which the page prints under every panel.

What they cannot show is correlation between axes: a state concentrated on a diagonal shell and one spread over a box can share all three marginals. That is a real limit of the representation, and the reason a true isosurface view is listed as unfinished rather than unnecessary.

Verified in a browser on a driven 34^3 run: all three marginals integrate to 1.000000 in every sampled frame, the $P(x,y)$ peak travels along x while its y coordinate does not move, and $P(y,z)$ does not move at all — exactly right for a drive along x.

Isosurfaces

Marginals cannot show correlation between axes; an isosurface can. `QM3 ISO` extracts the surface where $|\psi|^2$ equals a chosen fraction of its peak.

Marching tetrahedra, not marching cubes. Marching cubes needs a 256-entry table mapping corner-sign patterns to triangle lists — and that table *is* the algorithm, so one wrong entry gives a surface with a hole that looks fine from most angles. Marching tetrahedra needs no table: six tetrahedra per cube, $2^4 = 16$ sign patterns, three cases up to symmetry. The classic ambiguity where two cubes triangulate a shared face incompatibly cannot arise, because a tetrahedron's faces are triangles.

The extractor is a library function with quantitative tests: for a sphere and an ellipsoid it checks enclosed volume by the divergence theorem, surface area, convergence under refinement, and that the mesh is **watertight and consistently oriented** — every directed edge exactly once, its reverse exactly once.

Two defects that testing found rather than inspection: samples lying *exactly* on the isolevel produced coincident vertices and pinholes (fixed by nudging the level, not the samples); and orienting a split quad's two triangles independently let one flip and open the shared diagonal (they now share one winding decision).

Rendering is a **software rasteriser on a 2-D canvas**, not WebGL: WebGL can fail silently without a GPU and then the page shows nothing, whereas a painter's-algorithm rasteriser works everywhere and can be verified by reading pixels back.

The 3-D oscillator

```
In[4]:= qm3 states 4
Out[4]= 4 lowest bound state(s) -- Lanczos, 245 iterations:
  E[0] = 1.4738115042   residual 5.71e-8
  E[1] = 2.4382167269   residual 2.51e-8
  E[2] = 2.4382167269   residual 5.03e-8
  E[3] = 2.4382167269   residual 5.69e-8
```

Against the exact $E = n_x + n_y + n_z + 3/2$: 3/2, then 5/2 **three times**. The three excited values agree to ten digits — that degeneracy is what deflation exists to resolve — while both levels sit about 0.03 below the continuum, which is second-order discretisation error at $h = 0.52$.

5.13 `DATASET`, `SQL`, `MOVIE` — SQLite recording, queries, and derived movies

Every simulation can record itself into a **dataset**: one ordinary SQLite database file under `datasets/<name>.db`. The dataset is the canonical artifact of a run — the notebooks build theirs automatically, the `SQL` command interrogates it with the complete SQLite language, `DATASET PLAY` plots it in your web browser, and a **movie** is a 3-D animation *derived from* a dataset, never recorded independently of one.

The architecture, honestly. posim itself contains **no SQLite code** — the simulator workspace keeps its zero-external-dependency, zero-`unsafe` guarantee. All database work happens in a separate sidecar binary, `solveit_db` (the `dataset_tools/` package), which posim spawns and speaks to in line-delimited JSON. Build it once, from the repository root:

```
cd dataset_tools && cargo build --release
```

The sidecar follows the standard SQLite concurrency recipe, exactly:

1. **A thread-safe connection pool** (`r2d2` + `r2d2_sqlite`, pool size 10). No raw connection ever crosses a thread; the POOL is cloned into the sidecar's four worker threads.
2. **WAL journal mode**, enabled on every connection the moment the pool creates it (`PRAGMA journal_mode=WAL`). Readers never block the writer and the writer never blocks readers — which is what lets a Jupyter notebook's Python `sqlite3` reader watch a dataset **while posim is still appending to it**.
3. **A busy timeout** (`PRAGMA busy_timeout = 5000`): a connection that finds the database momentarily locked waits up to five seconds instead of failing with `database is locked`.

The sidecar also registers `sqrt(x)`, `pow(x, y)` and `hypot(x, y)` as deterministic SQL functions (the bundled SQLite 3.46.0 is compiled without its optional math functions, and physics queries want them).

Names and places. Dataset and movie names are file stems — letters, digits, `_`, `-` — written as an identifier or, when a name collides with a keyword, as a quoted string (`dataset open "box"`). Datasets live in `datasets/`, movies in `movies/`, both relative to the working directory, exactly like `videos/` and `scripts/`.

The schema (version 1). Four tables, no timestamps anywhere (byte-stable output is a verified property of this repository, and a wall-clock column would silently break it):

table	one row per	columns
meta	key/value pair	key, value (schema_version, title, box)
bodies	body	body (the STABLE id — DEL renumbers indices but never reuses ids), name, shape, mass, wall, geometry (JSON text of the shape parameters)
frames	recorded instant	frame, t, energy, px py pz, lx ly lz, comx comy comz, ncollisions (cumulative)
states	body per frame	frame, body, x y z, qw qx qy qz (**w-first**, like everywhere in this repository), vx vy vz, wx wy wz

The commands, one by one.

DATASET CREATE <name> makes datasets/<name>.db FRESH — an existing dataset of the same name is replaced, so re-running a script or notebook always reproduces the same database (keep an old run with COPY or SAVEAS first). The dataset opens with **recording armed**:

```
In[5]:= dataset create solveit20_kepler
Out[5]= dataset solveit20_kepler created (datasets/solveit20_kepler.db, journal_mode=
    wal, schema v1) -- recording armed
```

While recording is armed, **every output row of a RUN** (all STEPS n of them, captured at the exact output instants the solver visits) and **the end of every STEP** is appended as one frame, in a single sidecar transaction per command. DATASET RECORD OFF pauses that, DATASET RECORD ON resumes it, bare DATASET RECORD reports the switch. DATASET SNAPSHOT appends one frame right now, recording switch or no.

DATASET OPEN <name> attaches an existing dataset (recording armed); DATASET CLOSE checkpoints the WAL into the main file — leaving one tidy .db with no -wal/-shm sidecar files — and shuts the sidecar child down. RESET closes the open dataset the same way: a dataset describes ONE world, and RESET destroys that world. (The scene window survives a RESET; the dataset deliberately does not.)

DATASET LIST names everything under datasets/; DATASET INFO [<name>] reports row counts for the named dataset, or for the open one with no name:

```
In[7]:= dataset info
Out[7]= dataset solveit20_kepler: 2 bodies, 200 frames, 400 states
```

DATASET COPY <src> <dst> snapshot-copies a dataset (SQLite's VACUUM INTO: compact, checkpointed, safe to take even while the source is open). DATASET SAVEAS <name> does the same from the OPEN dataset — the checkpoint idiom. DATASET DELETE <name> removes a dataset and its WAL sidecars; deleting the open dataset is refused until you CLOSE it.

DATASET PLAY [<name>] renders the recorded time series into a self-contained page datasets/<name>_plots.html — total energy $E(t)$, $|P|$ and $|L|$, per-body $x/y/z$ components, and a top view of every trajectory — and opens it in your web browser. Like every page this repository generates, it is vanilla JS + canvas with zero external fetches. POSIM_NO_BROWSER=1 suppresses the browser launch (batch runs); the page is written either way.

DATASET DIFF <a> <b> AS <out> and DATASET SUM <a> <b> AS <out> — trajectory algebra. The teacher's experiment in one command: run a trial, change a mass or a charge, run the second trial over the SAME space-time domain, and subtract. DIFF (and SUM) produce a new dataset

whose every column is the component-wise difference (sum) of the two inputs — the states table (positions, quaternion components, velocities, angular velocities), and the frames table (energy, momentum, angular momentum, centre of mass, collision counts). The time column is carried through unchanged. **Alignment is checked, not assumed**: the two datasets must have the same number of frames, a bit-equal time grid, and the same body-id set at every frame — any mismatch is an error naming the first offending frame and both values. A worked consistency check: `(a - b) + b - a` comes back all-zero to floating-point roundoff (SolveIt example 36 measures `6.9e-18`). Note the orientation columns of a DIFF/SUM dataset hold component-wise quaternion arithmetic — a difference of quaternions is not a rotation, so such a dataset is a difference TABLE for plots and SQL, not a rigid state (build movies from the source recordings instead).

`DATASET MAP <src> TO <system> [params] AS <out>` — **curvilinear coordinates.** Every recorded position is pushed through one of the fourteen classical coordinate maps of §5.14: eleven three-dimensional (`spherical`, `cylindrical`, `ellipsoidal a b c`, `oblate a`, `prolate a`, `paraboloidal`, `elliptic_cyl a`, `parabolic_cyl`, `bipolar_cyl a`, `toroidal a`, `bispherical a`) and three two-dimensional (`polar2`, `elliptic2 a`, `bipolar2 a`), where `a` (or the ellipsoidal semi-axes `a b c`) is the system's scale parameter, written as plain numbers after the system word. In the output dataset the states table's `x, y, z` columns hold the curvilinear triple (`q1, q2, q3`; a 2-D system stores `(q1, q2, 0)` and REQUIRES a planar trajectory — every $|z| \leq$ `1e-9`, checked); orientations are carried unchanged (they are still rotation quaternions); the velocity columns are stored as **NULL**, because velocity components in curvilinear coordinates require the metric, which a point map does not carry; and the frames table (E, P, L, com — Cartesian bookkeeping) is copied verbatim. Everything is checked: an unknown system or a wrong parameter count is a PARSE error, and a trajectory that crosses a singular locus (a bipolar focus, the toroidal focal ring) errors naming the frame and body. A planar orbit mapped with `spherical` gives the cleanest possible sanity check: the polar-angle column is `1.5707963267948966` ($\pi/2$) at every frame, `MIN = MAX`, measured in SolveIt example 37.

`SQL "<statements>"` — **the whole SQLite language.** The quoted string is handed to SQLite **verbatim** (several statements separated by `;` are fine), so every SQLite command works: `SELECT` with joins, subqueries, CTEs (`WITH`) and window functions (`LAG ... OVER`), `INSERT`, `UPDATE`, `DELETE`, `CREATE TABLE / INDEX / VIEW / TRIGGER`, `ALTER TABLE`, `DROP`, `ATTACH DATABASE / DETACH DATABASE`, `PRAGMA`, `VACUUM`, `EXPLAIN`, transactions — anything SQLite 3.46 accepts. Row-returning statements print a header line and one line per row, columns separated by `|`; integer values print without a decimal point, other reals in Rust's shortest round-trip form, NULL as `NULL`. Statements that change rows print `ok (N row(s) changed)`.

```
In[8]:= sql "SELECT COUNT(*) AS frames FROM frames"
Out[8]= frames
200
```

SQL runs on the **open** dataset; with none open it says so and points at `DATASET CREATE/OPEN`. Cross-dataset work is one `ATTACH` away — the worked example in §14 and SolveIt example 31 both do it.

`MOVIE` — **animations derived from datasets.** `MOVIE CREATE <dataset> [AS <name>]` reads the `bodies`, `frames` and `states` tables and writes the self-contained player page `movies/<name>.html` (the name defaults to the dataset's). The player animates the recorded bodies in 3-D: play/pause/step, a scrub bar, 0.25×–4× speed, an orbiting camera (drag to orbit, wheel or `+/-` to zoom, arrow keys

to pan and step), per-body trails and labels, and a live readout of t, E, $|P|$, $|L|$ and the cumulative collision count. It follows the same pinned drawing rules as the scene window and the recorded videos: wall slabs are excluded from both the draw list and the camera auto-fit (a `BOX` appears as the dashed wireframe instead), and every round shape is rendered quaternion-rotated about its local z axis so spin is visible. `MOVIE PLAY <name>` opens a movie in the browser, `MOVIE LIST` names what exists, `MOVIE DELETE <name>` removes one. Because the dataset is the source of truth, movies can be deleted and rebuilt at any time without re-running any physics, and one dataset can carry several movies (SolveIt example 34 builds two from one recording).

A movie needs at least 2 frames of at least one body; a dataset that has neither gets a plain error saying so. A dataset recorded by a purely quantum-mechanical session has frames but no rigid-body states — the quantum families draw with `QM ANIMATE`/`QM2 ANIMATE`/`QM3 ISO` instead, which is their native picture.

Error style. Everything here reports `Result`-style errors naming the missing thing and the command that fixes it: a missing dataset names the path and suggests `DATASET LIST`; a missing sidecar binary prints the exact build command; SQL errors carry SQLite's own message plus the offending statement.

5.14 Coordinate transformations — the fourteen classical curvilinear systems

Pure-Rust, checked, bidirectional maps between Cartesian coordinates and every classical curvilinear system (conventions per Morse & Feshbach ch. 5 / Arfken ch. 2), reachable two ways: **value-level**, as ordinary expression functions on vectors, and **dataset-level**, via `DATASET MAP` (§5.13). All of them live in `special_functions/src/coords.rs` with round-trip, known-value and error-path tests.

The three-dimensional systems and their builtins. Each `cart_to_*` takes a vector `[x, y, z]` (plus scale parameters where the system has them) and returns the curvilinear triple as a vector; each `*_to_cart` is its inverse. Angle ranges: `atan2` angles in $(-\pi, \pi]$, `acos` angles in $[0, \pi]$.

system	builtins		coordinates and convention
Spherical-polar	`cart_to_spherical(v)` / `spherical_to_cart(q)`		(r, θ, ϕ): $x = r\sin\theta\cos\phi$, $y = r\sin\theta\sin\phi$, $z = r\cos\theta$; θ from the $+z$ axis
Cylindrical	`cart_to_cylindrical(v)` / `cylindrical_to_cart(q)`		(ρ, ϕ, z): $x = \rho\cos\phi$, $y = \rho\sin\phi$
Ellipsoidal	`cart_to_ellipsoidal(v, a, b, c)` / `ellipsoidal_to_cart(q, a, b, c)`		(λ, μ, ν), semi-axes $a > b > c > 0$: the three real roots of $x^2/(a^2{+}s) + y^2/(b^2{+}s) + z^2/(c^2{+}s) = 1$, interlaced $\lambda > -c^2 > \mu > -b^2 > \nu > -a^2$ (found by safeguarded bisection — the cubic is strictly decreasing between its poles)
Oblate spheroidal	`cart_to_oblate(v, a)` / `oblate_to_cart(q, a)`		(μ, ν, ϕ), focal ring $\rho = a$: $\rho = a\cosh\mu\cos\nu$, $z = a\sinh\mu\sin\nu$; inverse from $\rho + iz = a\cosh(\mu + i\nu)$ via the ring distances
Prolate spheroidal	`cart_to_prolate(v, a)` / `prolate_to_cart(q, a)`		(μ, ν, ϕ), foci $(0, 0, \pm a)$: $\rho = a\sinh\mu\sin\nu$, $z = a\cosh\mu\cos\nu$
Paraboloidal	`cart_to_paraboloidal(v)` / `paraboloidal_to_cart(q)`		(u, v, ϕ) (rotational parabolic): $\rho = uv$, $z = (u^2 - v^2)/2$; inverse $u = \sqrt{r + z}$, $v = \sqrt{r - z}$
Elliptic cylindrical	`cart_to_elliptic_cyl(v, a)` / `elliptic_cyl_to_cart(q, a)`		(u, v, z), foci $(\pm a, 0)$ in the plane: $x = a\cosh u\cos v$, $y = a\sinh u\sin v$
Parabolic cylindrical	`cart_to_parabolic_cyl(v)` / `parabolic_cyl_to_cart(q)`		(σ, τ, z): $x = \sigma\tau$, $y = (\tau^2 - \sigma^2)/2$, $\tau \geq 0$
Bipolar cylindrical	`cart_to_bipolar_cyl(v, a)` / `bipolar_cyl_to_cart(q, a)`		(σ, τ, z), foci $(\pm a, 0)$: $x = a\sinh\tau/D$, $y = a\sin\sigma/D$, $D = \cosh\tau - \cos\sigma$
Toroidal	`cart_to_toroidal(v, a)` / `toroidal_to_cart(q, a)`		(σ, τ, ϕ), focal ring $\rho = a$: $\rho = a\sinh\tau/D$, $z = a\sin\sigma/D$
Bispherical	`cart_to_bispherical(v, a)` / `bispherical_to_cart(q, a)`		(σ, τ, ϕ), foci $(0, 0, \pm a)$: $\rho = a\sin\sigma/D$, $z = a\sinh\tau/D$

The two-dimensional systems. Points are 2-element lists `[x, y]`, results likewise:

system	builtins		coordinates and convention
Plane polar	`cart2_to_polar(u)` / `polar_to_cart2(w)`		(r, ϕ): $x = r\cos\phi$, $y = r\sin\phi$
Elliptic	`cart2_to_elliptic(u, a)` / `elliptic_to_cart2(w, a)`		(u, v), foci $(\pm a, 0)$: the planar section of elliptic cylindrical
Bipolar (special conformal extension)	`cart2_to_bipolar(u, a)` / `bipolar_to_cart2(w, a)`		(σ, τ), foci $(\pm a, 0)$: the planar bipolar system; CONFORMAL — the real form of the analytic map $x + iy = a\coth((\tau - i\sigma)/2)$, so angles are preserved everywhere off the foci (verified numerically in the tests)

Error checking, spelled out. Every map returns an error — never a NaN — for: non-finite inputs; invalid scale parameters ($a \leq 0$, or an ellipsoidal triple that is not $a > b > c > 0$); out-of-range curvilinear coordinates where the range is load-bearing ($r < 0$, $\rho < 0$, $\mu < 0$, $\tau < 0$, ν outside its interval); points ON a singular locus (a bipolar focus, the toroidal focal ring, a bispherical focus — where τ diverges); $\sigma = \tau = 0$ in the bipolar family (the point at infinity); and the coordinate planes for ellipsoidal coordinates, whose chart covers open octants (the coordinates

encode squares, so the inverse returns the positive-octant point). Merely CONVENTIONAL values — the azimuth on the polar axis, the angles at the origin — use `atan2(0, 0) = 0`, so every map round-trips exactly on its domain.

Captured from this machine (the round trips and the 3-4-5 triangle):

```
In[32]:= let v = [1.3, 0.7, 2.1]
Out[32]= v set
In[33]:= norm(toroidal_to_cart(cart_to_toroidal(v, 1.25), 1.25) - v)
Out[33]= 0
In[34]:= norm(ellipsoidal_to_cart(cart_to_ellipsoidal(v, 3, 2, 1), 3, 2, 1) - v)
Out[34]= 0.00000000000000119574679205636 33
In[35]:= cart2_to_polar([3, 4])
Out[35]= [5, 0.9272952180016122]
```

And an error path, exactly as a user sees it:

```
In[36]:= cart_to_toroidal([1, 0, 0], 1)
Err[36]: cart_to_toroidal: (rho, z) = (1, 0) is on the focal ring -- a singular locus
    of toroidal coordinates (tau diverges there)
```

6 The notebook (cells, editing, magics)

6.1 Cells

Start `posim` with no arguments. You get numbered prompts, exactly like Jupyter or Mathematica:

```
In[1]:= new sphere { mass = 2, radius = 0.5 }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[0.2, 0, 0], [0, 0.2, 0], [0, 0, 0.2]]
```

Pressing **Enter executes the cell** — in a plain terminal, Enter *is* shift-enter. Commands with no interesting result (like `SET`) print no `Out[n]`. Errors print `Err[n]:` and the session continues.

6.2 State is cumulative

Like Jupyter, the notebook has one live simulator state that every cell mutates in order. Re-running an old `NEW` cell creates a *new* object; it does not replace the old one. To rebuild cleanly, `%reset` and replay.

6.3 Magics

magic	effect
`%history`	lists every cell with input and output; failed cells marked `!`
`%edit n <text>`	replaces cell n's input and executes it as the next cell
`%rerun n`	executes cell n's input again as a new cell
`%save <file>`	writes all successful non-magic inputs to a replayable script
`%load <file>`	replays a script file cell by cell
`%reset`	clears the simulator (history kept)
`%quit`, `%exit`	leave

A plain terminal has no cursor-addressable cells (posim uses only the Rust standard library — no raw terminal mode), so backward/forward movement is by these magics. **For true click-and-edit cells with shift-enter, use the JupyterLab front end** (§7).

6.4 Scripts

`posim -script file` executes a file of command lines (blank lines and `#` comments skipped), echoing `In[n]`/`Out[n]`, and exits nonzero if any cell failed — good for CI.

`posim -notebook file` executes the same file format but then **stays in the interactive notebook** instead of exiting: the loaded cells keep their `In[n]` numbers, the next command continues the numbering, and a scene window the file opened (`SCENE CREATE`) stays alive — the launcher for the *dynamic notebooks* in `dynamic_notebooks/`, files that build a simulation and open the GUI ready for Start. A failing cell aborts the load with a nonzero exit instead of entering the notebook.

7 JupyterLab in one paragraph

The `jupyter/` directory ships a small *wrapper kernel*: JupyterLab starts a Python shim, the shim starts `posim -machine` (a JSON request/reply protocol over stdin/stdout), and each notebook cell you shift-enter is forwarded line-by-line as `{"op":"exec","code":...}`. Setup and tests: `jupyter/README.md`. Everything in §§2–5 applies verbatim inside JupyterLab cells; multi-line cells run one line at a time, stopping at the first error.

8 The stack machine (what runs your line)

The parser emits *postfix instructions*; e.g. `set obj0.mass = 2 + 3 * 4` compiles to

```
Push 2
Push 3
Push 4
Mul               <- pops 3,4  pushes 12
Add               <- pops 2,12 pushes 14
Store obj0.mass   <- pops 14, calls set_mass(14)
```

The machine's whole memory is a stack of typed values. Instructions: `Push`, `Load`, `Store`, `LoadIdent` (a bare identifier — parameter or `LET` variable, resolved at execution, §5.9), `StoreGlobal` (`LET`), `Add Sub Mul Div Neg`, `PackList` (literals), `Call` (builtin or user function — a user call runs

each body line through this same compile-and-execute pipeline inside a call frame), `NewObject/InitField/FinishN` (which also registers the `AS` name), `Delete`, `ListObjects`, `ListFns/ShowFn` (`FUNCS/SHOW`), `Step`, `Run`, `SetMethod`, `Energy`, `CenterOfMass`, `TotalMomentum`, `TotalAngularMomentum`, `Laplace`, `Reset`, `Help`. `Load/Store` go *only* through the `physical_object` get/set API — the machine cannot corrupt simulator state, and every coupled invariant (mass↔inverse, inertia↔inverse, unit quaternions) is enforced on every write. When the program ends, the top of the stack becomes `Out[n]`.

9 Eighteen worked examples

All transcripts are genuine program output.

9.1 Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass

The Laplace–Runge–Lenz vector **A** points along a Kepler orbit's major axis and is conserved *only* for a perfect $1/r^2$ force — the most delicate conservation test available. We build a "sun" so heavy the reduced problem is exact and integrate two orbits with a 4th-order *symplectic* method.

```
In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= laplace 1
Out[5]= [0.5999999974000001, 0, 0]
In[6]:= method sprk mclachlan_4_4 0.001
Out[6]= method = ARKODE SPRK ARKODE_SPRK_MCLACHLAN_4_4, fixed dt = 0.001
In[7]:= run 12.6 steps 2
Out[7]= t = 12.6 (12600 solver steps, 2 snapshots, |dE/E| = 9.237e-14)
In[8]:= laplace 1
Out[8]= [0.5999999974140128, -0.00000000034051644837163053, 0]
In[9]:= get obj1.position
Out[9]= [0.3964802853115723, 0.06706198328283366, 0]
```

What to notice. `g_constant = 1e-9` makes $GM_{\text{total}} = 1$ so the orbit has period 2π. `LAPLACE` 1 divided by mk is the *eccentricity vector*: $e = 0.6$ is read directly off `Out[5]`. After $\sim$2 orbits the energy is conserved to 9×10^{-14} and **A** has rotated by only 3×10^{-10} — no spurious perihelion precession. Softening was zeroed because any $\varepsilon \neq 0$ slightly breaks the $1/r^2$ law and *would* precess the axis.

9.2 Example 2 — a thrown ball vs. the textbook formula

A projectile has the closed form $x(t) = v_x t$, $y(t) = y_0 + v_y t - \frac{1}{2} g t^2$. We fly a baseball for 6.12 s and subtract the formula *inside the notebook* using bare expressions.

```
In[1]:= new point { mass = 0.145, position = [0, 1, 0], velocity = [30, 30, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= run 6.12 steps 3
Out[3]= t = 6.12 (13 solver steps, 3 snapshots, |dE/E| = 4.740e-15)
In[4]:= get obj0.position.x
```

```
Out[4]= 183.5999999999999
In[5]:= 30 * 6.12
Out[5]= 183.6
In[6]:= obj0.position.x - 30 * 6.12
Out[6]= -0.00000000000008526512829121202
In[7]:= obj0.position.y - (1 + 30 * 6.12 - 9.81 / 2 * 6.12 * 6.12)
Out[7]= -0.00000000000042632564145606 01
```

What to notice. Cells 6–7 are *bare expressions* mixing paths and arithmetic (`GET` itself would refuse the arithmetic). The trajectory matches the parabola to $\sim 10^{-13}$ — and the adaptive Adams integrator needed only **13 internal steps**, because for a polynomial solution its error estimator lets it take huge strides.

9.3 Example 3 — cyclotron motion: expressions as arguments

A charge in a uniform magnetic field circles with period $T = 2\pi m/(|q|B)$. We *compute* T *inside the* `RUN` *command*.

```
In[1]:= new sphere { mass = 2, radius = 0.1, charge = -1.5, velocity = [3, 0, 0] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0, 4]
In[3]:= method bdf
Out[3]= method = CVODE BDF
In[4]:= 2 * pi * 2 / (1.5 * 4)
Out[4]= 2.0943951023931953
In[5]:= run 2 * pi * 2 / (1.5 * 4) steps 8
Out[5]= t = 2.0943951023931953 (318 solver steps, 8 snapshots, |dE/E| = 1.444e-8)
In[6]:= get obj0.position
Out[6]= [0.00000000043106769672882073, 0.000000007220835657713453, 0]
In[7]:= norm(obj0.velocity)
Out[7]= 2.9999999783374913
```

What to notice. After one analytic period the particle is back at the origin to 7×10^{-9}, and `norm()` shows the speed unchanged — the magnetic force does no work. The Lorentz force $q\,v \times B$ is velocity-dependent, so the symplectic path is *not allowed* here; `BDF` is the right tool for fast gyration.

9.4 Example 4 — the tennis-racket theorem (Dzhanibekov effect)

A rigid body spun about its *intermediate* principal axis is unstable: a tiny wobble grows into a dramatic flip, yet energy and angular momentum stay conserved. Half-extents `[0.5, 1, 2]` give moments $[5, 4.25, 1.25]$; spinning about y (the middle value) triggers it.

```
In[1]:= new cuboid { mass = 3, half_extents = [0.5, 1, 2], angular_velocity = [0.01, 3,
     0.01] }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[5, 0, 0], [0, 4.25, 0], [0, 0, 1.25]]
In[3]:= get obj0.angular_velocity
Out[3]= [0.010000000000000002, 3, 0.010000000000000002]
In[4]:= run 40 steps 4
```

```
Out[4]= t = 40 (2361 solver steps, 4 snapshots, |dE/E| = 5.605e-9)
In[5]:= get obj0.angular_velocity
Out[5]= [1.5428438559953521, 2.9947703802651175, -0.7871804456905063]
In[6]:= run 40 steps 4
Out[6]= t = 80 (2334 solver steps, 4 snapshots, |dE/E| = 5.332e-9)
In[7]:= get obj0.angular_velocity
Out[7]= [0.020666590902422656, 2.9999548562146483, 0.013346831306280834]
In[8]:= get obj0.angular_momentum
Out[8]= [0.05, 12.75, 0.0125]
```

What to notice. These two samples are snapshots of a *recurring* motion, not the start and end of one flip. The turn-over is a *body-frame* event: the angular momentum is fixed in space, so the world-frame ω_y printed above stays near $+3$ for the whole run and never reverses at all. Extract $\omega_{\text{body}} = \bar{q}\,\omega_{\text{world}}\,q$ and its y-component reverses at roughly $t = 4, 18, 28, 40, 50, 62, 74$ — about every 12–13 units. By $t = 80$ the body has turned over some *seven* times; $t = 40$ merely lands mid-reversal and $t = 80$ between reversals. Throughout, the world-frame angular momentum (`Out[8]`) is exactly the initial $I\omega = [0.05, 12.75, 0.0125]$ — the solver integrates the full quaternion + angular-momentum state with energy error $\sim 5 \times 10^{-9}$. (With these moments a symmetric wobble $(d, 3, d)$ sits exactly on the separatrix $G^2 = BT$, since $G^2 - BT = d^2[A(A-B)+C(C-B)] = d^2[5(0.75)+1.25(-3)] = 0$ for every d; the repeated reversal is robust, but the exact times shift with the tolerances. `dynamic_notebooks/tumbling_body.posim` shows the body-frame component directly.)

9.5 Example 5 — three bodies, then surgery with `DEL`

```
In[1]:= new point { mass = 1,   position = [1, 0, 0],  velocity = [0, 1, 0] }
Out[1]= obj0
In[2]:= new point { mass = 4,   position = [-1, 0, 0], velocity = [0, -0.25, 0] }
Out[2]= obj1
In[3]:= new point { mass = 256, position = [0, 8, 0],  velocity = [0, 0, 0] }
Out[3]= obj2
In[4]:= set system.g_constant = 0.001
In[5]:= momentum
Out[5]= [0, 0, 0]
In[6]:= com
Out[6]= [-0.011494252873563218, 7.846743295019157, 0]
In[7]:= run 3 steps 3
Out[7]= t = 3 (84 solver steps, 3 snapshots, |dE/E| = 3.141e-10)
In[8]:= list
Out[8]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
obj2: point, mass=256, charge=0, pos=[-0.00017852126656859762, 7.999648688652149, 0]
In[9]:= del 2
Out[9]= deleted obj2; 2 object(s) remain (indices renumbered)
In[10]:= list
Out[10]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
In[11]:= momentum
Out[11]= [0.0021430470372574067, 0.061528401914275554, 0]
```

What to notice. Cells 1–3 chose velocities so the initial total momentum is exactly zero ($1 \cdot 1 + 4 \cdot$

$(-0.25) = 0$), and it stays zero through the run. Deleting the heavy body removes its momentum: the remainder (`Out[11]`) is what the surviving pair carries — conservation applies to the system you keep. `DEL` renumbers later objects.

9.6 Example 6 — magnetic torque (and why `ENERGY` grows)

`magnetic_moment_tensor` M couples the world B field to torque $\tau = (RMR^{\mathsf{T}})B$. Unlike everything else, this coupling is *not derived from a potential*, so total energy is *expected* to change.

```
In[1]:= new sphere { mass = 1, radius = 0.5, magnetic_moment_tensor = [[0.2, 0, 0], [0,
     0.2, 0], [0, 0, 0.2]] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0.5, 0]
In[3]:= get obj0.angular_momentum
Out[3]= [0, 0, 0]
In[4]:= energy
Out[4]= 0
In[5]:= run 4 steps 4
Out[5]= t = 4 (214 solver steps, 4 snapshots, |dE/E| = 8.000e-1)
In[6]:= get obj0.angular_momentum
Out[6]= [0, 0.40000000000000085, 0]
In[7]:= get obj0.angular_velocity
Out[7]= [0, 4.000000000000009, 0]
In[8]:= energy
Out[8]= 0.8000000000000035
```

What to notice. Constant torque $\tau = MB = 0.2 \times 0.5 = 0.1$ about y gives $L(t) = 0.1t$: after 4 s, $L = 0.4$ exactly (the ODE is linear, so the solver nails it). Sphere inertia $= 0.4 \cdot 1 \cdot 0.25 = 0.1$, so $\omega = L/I = 4$ and the rotational energy $\frac{1}{2}\omega L = 0.8$ matches `Out[8]`. The reported `|dE/E| = 8e-1` is not an error — it honestly reports that the torque pumped energy in.

9.7 Example 7 — fixing a typo in an old cell with `%edit`

```
In[1]:= new sphere { mass = 20, radius = 0.5 }
Out[1]= obj0
In[2]:= set obj0.velocity = [1, 0, 0]
In[3]:= get obj0.momentum
Out[3]= [20, 0, 0]
In[4]:= %edit 1 new sphere { mass = 2, radius = 0.5 }
In[4]:= new sphere { mass = 2, radius = 0.5 }
Out[4]= obj1
In[5]:= %history
 In[1]:= new sphere { mass = 2, radius = 0.5 }
  Out[1]= obj0
 In[2]:= set obj0.velocity = [1, 0, 0]
 In[3]:= get obj0.momentum
  Out[3]= [20, 0, 0]
 In[4]:= new sphere { mass = 2, radius = 0.5 }
  Out[4]= obj1
```

What to notice. `%edit 1 ...` rewrites cell 1's stored input **and re-executes it as the next cell**

— like editing a Jupyter cell and shift-entering again. Because state is cumulative, the re-executed `NEW` made a *second* object; the fat `obj0` is still there. For a clean rebuild use `%reset` + `%rerun` or `%load`; when a field tweak suffices, `set obj0.mass = 2`. `%history` shows the *edited* input but the *original* outputs — an audit trail of what actually ran.

9.8 Example 8 — checking the simulator against itself

```
In[1]:= new point { mass = 2, position = [1, 0, 0], velocity = [0, 3, 0] }
Out[1]= obj0
In[2]:= cross(obj0.position, obj0.momentum)
Out[2]= [0, 0, 6]
In[3]:= angmom
Out[3]= [0, 0, 6]
In[4]:= dot(obj0.position, obj0.velocity)
Out[4]= 0
In[5]:= norm(cross(obj0.position, obj0.momentum)) / (norm(obj0.position) * norm(obj0.
    momentum))
Out[5]= 1
```

What to notice. Cell 2 computes $L = r \times p$ by hand and cell 3 confirms `ANGMOM` agrees. Cell 4 proves $r \perp v$; cell 5 computes $|r \times p|/(|r||p|) = \sin\theta = 1$ — the angle between r and p is 90° — inside one nested expression.

9.9 Example 9 — hand-built tensors and a quaternion orientation

```
In[1]:= new cuboid { mass = 1, inertia_tensor = [[2, 0, 0], [0, 3, 0], [0, 0, 4]],
    orientation = [0.7071067811865476, 0, 0, 0.7071067811865476] }
Out[1]= obj0
In[2]:= get obj0.inverse_inertia_tensor
Out[2]= [[0.5, 0, 0], [0, 0.3333333333333333, 0], [0, 0, 0.25]]
In[3]:= set obj0.angular_velocity = [0, 2, 0]
In[4]:= get obj0.angular_momentum
Out[4]= [0.00000000000000044408920985006280, 4.000000000000002, 0]
In[5]:= get obj0.orientation
Out[5]= quat[w=0.7071067811865476, x=0, y=0, z=0.7071067811865476]
```

What to notice. Because `inertia_tensor` appears in the initializer, the analytic inertia is **not** recomputed — your diag$(2,3,4)$ is kept, with the inverse maintained automatically. The quaternion $[w,x,y,z] = [\sqrt{1/2}, 0, 0, \sqrt{1/2}]$ is a 90° rotation about z, so setting the *world* angular velocity $[0,2,0]$ exercises the full $L = (RIR^{\mathsf{T}})\omega$ transformation. By hand: rotating diag$(2,3,4)$ by 90° about z swaps the x and y moments, so $RIR^{\mathsf{T}} = \mathrm{diag}(3,2,4)$, the world-$y$ moment is 2, and $L = [0,4,0]$ — exactly `Out[4]` (up to 4×10^{-16} of floating-point dust).

9.10 Example 10 — reproducibility: `%save`, `%reset`, `%load`

```
In[1]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= step 0.5
Out[3]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[4]:= %save session.posim
```

```
saved 3 cell(s) to session.posim
In[4]:= %reset
system reset
In[4]:= list
Out[4]= (no objects)
In[5]:= %load session.posim
In[5]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[5]= obj0
In[6]:= set system.gravity = [0, -9.81, 0]
In[7]:= step 0.5
Out[7]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[8]:= get obj0.velocity
Out[8]= [1, -4.905000000000001, 0]
```

What to notice. `%save` writes only the successful, non-magic inputs — a clean, replayable script. After `%reset`, `%load` replays it and the state is bit-identical ($v_y = -9.81 \cdot 0.5 = -4.905$). The same file runs headlessly via `posim -script`.

Notice too that **magics do not consume cell numbers**: `%save` and `%reset` print their message and leave the counter alone, which is why the prompt shows `In[4]` three times in a row before `LIST` finally becomes cell 4. Only executed commands become numbered cells — so the `In[n]` numbering always matches what `%history` will replay.

The machine mode speaks the same language over JSON:

```
$ printf '%s\n' '{"op":"exec","code":"new point { mass = 1, velocity = [0, 1, 0] }"}' \
                '{"op":"get","path":"obj0.momentum"}' \
                '{"op":"set","path":"obj0.mass","value":5}' \
                '{"op":"get","path":"obj0.momentum"}' | posim --machine
{"display":"obj0","ok":true,"result":"obj0"}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}
{"display":"","ok":true,"result":null}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}
```

Changing the mass did **not** change the momentum — momentum is the canonical stored state; the *velocity* is what changed. That final subtlety is the union design showing through the wire protocol.

9.11 Example 11 — watching a Kepler orbit live, then running it backward

The same two-body setup as Example 1 — but this time we *watch* it. `SCENE CREATE` opens the scene window in the browser; `SCENE START` sets it in motion; `SCENE REVERSE` runs the movie backward.

```
In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= scene create 7878
Out[5]= scene window created: http://127.0.0.1:7878/
```

```
(opened in your browser; if no window appeared, open that address yourself)
showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[6]:= scene set_time_step 0.005
Out[6]= scene time step dt = 0.005
In[7]:= scene start
Out[7]= scene playback: running
In[8]:= scene pause
Out[8]= scene playback: paused
In[9]:= scene status
Out[9]= scene: http://127.0.0.1:7878/  (1 window(s) connected)
mode = paused, t = 0.4550000000000003, dt = 0.005, steps = 91, history = 91 frame(s)
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[10]:= scene reverse
Out[10]= scene playback: reversing
In[11]:= scene status
Out[11]= scene: http://127.0.0.1:7878/  (1 window(s) connected)
mode = reversing, t = 0.15500000000000005, dt = 0.005, steps = 91, history = 31 frame(s
    )
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[12]:= scene events
Out[12]= window connected (1 total)
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)
```

What to notice. Between `In[7]` and `In[8]` a few real seconds passed while the orbit ran in the window — playback happens on a background thread at ∼30 frames per second, so **the notebook prompt never blocks**: cell 9's `STATUS` shows the *playback copy* has advanced 91 steps to $t \approx 0.455$ while your notebook state still sits untouched at $t = 0$ (that is the "playback copy" design: the window animates its own synchronized copy of your system, so `GET system.time` here would still print `0`). After `REVERSE` (cell 10), the status in cell 11 shows time flowing *backward* ($t \approx 0.155$) and the history buffer draining ($91 \to 31$ frames): each reversed frame is an exact recorded snapshot, so the rewind retraces the orbit bit-for-bit. Cell 12 drains the asynchronous event queue — the window announced itself when the browser connected. In JupyterLab you would not even need cell 12: events surface on their own as `[scene] ...` lines (§7).

9.12 Example 12 — driving the camera from the notebook

Every gesture the mouse can make in the window has a command twin, so a *script* can compose the exact view you want — useful for repeatable demonstrations and screenshots.

```
In[1]:= new sphere { mass = 3, radius = 0.6, position = [2, 0, 0] }
Out[1]= obj0
In[2]:= new cuboid { mass = 1, half_extents = [0.4, 0.3, 0.2], position = [-2, 0, 1] }
Out[2]= obj1
In[3]:= scene create 7878
Out[3]= scene window created: http://127.0.0.1:7878/
(opened in your browser; if no window appeared, open that address yourself)
showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[4]:= scene translate 2 0
Out[4]= camera target = [2, 0, 0]
```

```
In[5]:= scene rotate 30 -10
Out[5]= camera yaw = -30°, pitch = 45°
In[6]:= scene zoom in
Out[6]= camera distance = 9.6
In[7]:= scene zoom 2
Out[7]= camera distance = 4.8
In[8]:= scene zoom out
Out[8]= camera distance = 5.999999999999999
In[9]:= scene hide 0
Out[9]= 1 object(s) hidden
In[10]:= scene show all
Out[10]= 0 object(s) hidden
In[11]:= scene redraw
Out[11]= scene redraw queued for every window
In[12]:= scene status
Out[12]= scene: http://127.0.0.1:7878/  (0 window(s) connected)
mode = stopped, t = 0, dt = 0.01, steps = 0, history = 0 frame(s)
entities = 2 (hidden: none)
camera: yaw = -30°, pitch = 45°, dist = 5.999999999999999, target = [2, 0, 0]
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)
```

What to notice. The camera is an **orbit camera**: it circles a *look-at point* at a *distance*, aimed by *yaw* (compass direction) and *pitch* (elevation). `TRANSLATE 2 0` moves the look-at point onto the sphere (this is what the arrow keys do in the window); `ROTATE 30 -10` adds 30° of yaw to the default −60° and −10° of pitch to the default 55° (this is what left-dragging does); the three zooms multiply the distance by 1/1.25, then 1/2, then 1.25 — watch the arithmetic: $12 \to 9.6 \to 4.8 \to 6$ (this is the mouse wheel). `HIDE 0` blanks the sphere out of every connected window without deleting anything — `SHOW ALL` brings it back. Because this transcript ran headless, cell 12 reports `0 window(s) connected` — the commands work regardless, and any window that connects later receives the composed view. Note also the two-argument form `scene translate 2 0`: the optional `dz` defaulted to 0, and `-10` in cell 5 was one negative argument, not a subtraction (scene arguments are *terms* — the second subtlety of §3).

9.13 Example 13 — the box of shapes: every body type in a rigid, infinitely massive box

The finale scene: `BOX` 4 builds the rigid room (§5.8) and one of every shape goes inside ($R = 1$, $M = 1$) — a **torus** (mass M, inner radius 1, outer radius 2), a **point** (M, the only mover: $v = (100, 200, 100)$), a **sphere** ($2M$, $r = \frac{1}{2}$), an ideal zero-thickness **disk** ($2M/3$, $r = 1$), a **cube** ($5M/3$, side 1) and a **cylinder** ($2M$, $r = \frac{1}{2}$, height 3/2). An axis-aligned torus of outer radius 2 would *exactly inscribe* the 4-box (its outer equator touching four walls), so the torus is tilted with its axis along $(1, 1, 1)/\sqrt{3}$: its extent per axis is $1.5\sqrt{2/3} + 0.5 \approx 1.7247 < 2$ — clearance 0.2753. The other positions are random, drawn by a documented LCG ($x \leftarrow 1664525\,x + 1013904223 \bmod 2^{32}$, seed 20260724, sequential rejection at ≥ 0.05 separation) — see `scripts/collisions/11_box_of_shapes.posim`, the script this transcript replays.

```
In[1]:= set system.g_constant = 0
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
    obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
```

```
    the inside faces
In[3]:= new torus { mass = 1, inner_radius = 1, outer_radius = 2, orientation =
    [0.888073833977115, -0.325057583671868, 0.325057583671868, 0] }
Out[3]= obj6
In[4]:= new point { mass = 1, position = [1.406590, -0.995859, 0.569601], velocity =
    [100, 200, 100] }
Out[4]= obj7
In[5]:= new sphere { mass = 2, radius = 1/2, position = [1.424704, 1.367496, -0.493612]
     }
Out[5]= obj8
In[6]:= new disk { mass = 2/3, radius = 1, position = [-0.677386, -1.041493,
    -1.091679], orientation = [0.900447102352677, 0.307567078752479,
    -0.307567078752479, 0] }
Out[6]= obj9
In[7]:= new cuboid { mass = 5/3, half_extents = [0.5, 0.5, 0.5], position = [1.074397,
    0.816223, 1.102099] }
Out[7]= obj10
In[8]:= new cylinder { mass = 2, radius = 1/2, height = 3/2, position = [-1.027024,
    -1.403890, -0.485619], orientation = [0.968912421710645, 0, 0.247403959254523, 0] }
Out[8]= obj11
In[9]:= list
Out[9]= obj0: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[3, 0, 0] [wall: static,
    inverse_mass=0]
obj1: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[-3, 0, 0] [wall: static, inverse_mass
    =0]
obj2: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, 3, 0] [wall: static, inverse_mass
    =0]
obj3: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, -3, 0] [wall: static, inverse_mass
    =0]
obj4: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, 3] [wall: static, inverse_mass
    =0]
obj5: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, -3] [wall: static, inverse_mass
    =0]
obj6: torus ring=1.5 tube=0.5, mass=1, charge=0, pos=[0, 0, 0]
obj7: point, mass=1, charge=0, pos=[1.40659, -0.995859, 0.569601]
obj8: sphere r=0.5, mass=2, charge=0, pos=[1.424704, 1.367496, -0.493612]
obj9: disk r=1, mass=0.6666666666666666, charge=0, pos=[-0.677386, -1.041493,
    -1.091679]
obj10: cuboid he=[0.5, 0.5, 0.5], mass=1.6666666666666667, charge=0, pos=[1.074397,
    0.816223, 1.102099]
obj11: cylinder r=0.5 h=1.5, mass=2, charge=0, pos=[-1.027024, -1.40389, -0.485619]
In[10]:= collide
Out[10]= collisions ON (51 collidable pair(s); 0 impulse(s) so far)
In[11]:= get obj0.inverse_mass
Out[11]= 0
In[12]:= get obj6.inertia_tensor
Out[12]= [[1.28125, 0, 0], [0, 1.28125, 0], [0, 0, 2.4375]]
In[13]:= energy
Out[13]= 30000
In[14]:= momentum
Out[14]= [100, 200, 100]
In[15]:= run 0.1 steps 100
Out[15]= t = 0.1 (2121 solver steps, 100 snapshots, |dE/E| = 2.040e-10, 119 collision(s
    ) -- CONTACTS lists them)
```

```
In[16]:= energy
Out[16]= 29999.999993880127
In[17]:= momentum
Out[17]= [146.48911126803657, 102.1478121131382, 46.01601291636828]
In[18]:= get system.collisions
Out[18]= 119
In[19]:= get system.box
Out[19]= 4
```

What to notice. Cell 3 sizes the torus by the order-independent `inner_radius` + `outer_radius` pair; cell 8 uses `height = 3/2` — the *full* height, so `half_height` reads 0.75. `COLLIDE` counts **51 collidable pairs**: 12 objects make 66 pairs, minus the 15 wall–wall pairs (two static bodies can never collide). `Out[12]` is the torus's analytic inertia $\mathrm{diag}(1.28125, 1.28125, 2.4375)$ — exactly $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$, $I_z = m(c^2 + \frac{3}{4}a^2)$ for $c = 1.5$, $a = 0.5$. The energy starts at $E_0 = \frac{1}{2} \cdot 1 \cdot |v|^2 = 30000$ **exactly** and, after **119 collisions** in 0.1 s, is conserved to $|dE/E| = 2.040 \times 10^{-10}$ — but momentum went from $(100, 200, 100)$ to $(146.49, 102.15, 46.02)$: **not conserved**, because the infinitely massive walls absorb momentum without moving — the physical signature of infinite mass. Along the way the point particle can *thread the torus hole* and passes through the ideal zero-thickness disk (a measure-zero contact) — the ball-vs-anything collision tier is exact SDF geometry, not an approximation — while every finite-size body bounces off both. The walls end the run bit-identically at rest.

9.14 Example 14 — two dumbbells from a user-defined function

The two-release finale: a user function (§5.9) that builds **named dumbbells** (§5.1), called twice to launch a pair of tumbling dumbbells at each other, off-center and spinning. No walls this time — so *all three* conserved quantities must survive the collisions. The script is `scripts/collisions/12_two_dumbbells.p` this transcript replays it.

```
In[1]:= set system.g_constant = 0
In[2]:= def create_dumbell(name, m1 = 1, m2 = 1, m_rod = 0.5, r1 = 0.25, r2 = 0.25,
    rod_radius = 0.1, length = 1, position = [0, 0, 0], velocity = [0, 0, 0],
    angular_velocity = [0, 0, 0]) {
  new dumbbell as name { m1 = m1, m2 = m2, m_rod = m_rod, r1 = r1, r2 = r2, rod_radius
    = rod_radius, length = length, position = position, velocity = velocity,
    angular_velocity = angular_velocity }
}
Out[2]= function create_dumbell(11 parameter(s)) defined -- 1 body line(s)
In[3]:= create_dumbell("dumbell0", 1, 2, 0.5, 0.25, 0.25, 0.1, 1, [-2, 0.15, 0], [1.5,
    0, 0], [0, 0, 0.6])
Out[3]= obj0 as dumbell0
In[4]:= create_dumbell("dumbell1", 2, 1, 0.4, 0.3, 0.2, 0.08, 1.2, [2, -0.15, 0],
    [-1.5, 0, 0], [0.4, 0, 0])
Out[4]= obj1 as dumbell1
In[5]:= list
Out[5]= obj0: dumbbell r1=0.25 r2=0.25 rod_r=0.1 len=1, mass=3.5, charge=0, pos=[-2,
    0.15, 0]
obj1: dumbbell r1=0.3 r2=0.2 rod_r=0.08 len=1.2, mass=3.4, charge=0, pos=[2, -0.15, 0]
In[6]:= get dumbell0.m1
Out[6]= 1
In[7]:= get dumbell1.m_rod
```

```
Out[7]= 0.3999999999999999
In[8]:= get dumbell0.vx
Out[8]= 1.5
In[9]:= energy
Out[9]= 7.865310611764706
In[10]:= momentum
Out[10]= [0.15000000000000036, 0, 0]
In[11]:= angmom
Out[11]= [0.4443030588235295, 0, -1.5059999999999998]
In[12]:= run 3 steps 60
Out[12]= t = 3 (3861 solver steps, 60 snapshots, |dE/E| = 9.463e-11, 2 collision(s) --
    CONTACTS lists them)
In[13]:= energy
Out[13]= 7.865310611020375
In[14]:= momentum
Out[14]= [0.15000000000000036, 0, 0]
In[15]:= angmom
Out[15]= [0.44430305882373156, -0.000000000000009547918011776346, -1.505999999999855]
In[16]:= get system.collisions
Out[16]= 2
```

What to notice. Cell 2 is one `DEF` whose single body line does everything: `new dumbbell as name { ... }` forwards all eleven parameters, and because `name` arrives as a string (`"dumbell0"`, `"dumbell1"`), each call registers the name it was given — `Out[3]`/`Out[4]` show both handles, `LIST` shows the part geometry, and cells 6–8 read the parts and the `.vx` shorthand through the names. (`Out[7]` reads 0.4 back as `0.3999999999999999`: the parts are stored as mass *fractions* of the total, so one reconstruction rounding step shows through — 15 correct digits.) Then the physics: two tumbling, asymmetric rigid bodies meet off-center, collide **twice** (cell 16), and every conserved quantity survives the real CVODE event handling. Energy: 7.865310611764706 → 7.865310611020375 — the run banner prints `|dE/E| = 9.463e-11`. Momentum: `[0.15000000000000036, 0, 0]` **bit-identical**. Angular momentum about the origin — the delicate one, orbital $r \times p$ plus spin, exchanged between the two at every off-center impact: conserved to $\sim 10^{-13}$ per component (`Out[11]` vs. `Out[15]`), because the part-wise exact narrow phase applies the action–reaction impulse pair at one shared contact point. Contrast Example 13: there the infinitely massive walls absorbed momentum; here, with no walls, E, P *and* L all hold at solver precision.

9.15 Example 15 — a particle in a box, solved inside the language

The special functions are not decoration: with `eigenvalues` you can set up and solve a small quantum problem in the notebook itself.

Take a particle in an infinite square well of width $L = 1$ with $\hbar = m = 1$. Discretise $-\frac{1}{2}\psi'' = E\psi$ on 4 interior points, spacing $h = 0.2$. The second-derivative stencil puts $1/h^2$ on the diagonal and $-1/2h^2$ beside it, so the Hamiltonian is a 4×4 tridiagonal matrix you can type out. The exact answer is $E_n = n^2\pi^2/2$, so $E_1 = \pi^2/2$.

```
In[1]:= let h = 0.2
Out[1]= h set
In[2]:= let k = 1 / (h * h)
Out[2]= k set
In[3]:= eigenvalues([[k, -0.5*k, 0, 0], [-0.5*k, k, -0.5*k, 0], [0, -0.5*k, k, -0.5*k],
```

```
    [0, 0, -0.5*k, k]])
Out[3]= [4.774575140626312, 17.274575140626325, 32.72542485937371, 45.225424859373675]
In[4]:= pi * pi / 2
Out[4]= 4.934802200544679
```

The ground state comes out at 4.7746 against an exact 4.9348 — **3.2 % low**. That is not a bug, and it is worth understanding rather than tightening a tolerance until it goes away. A 3-point stencil on 4 points is a *coarse* grid, and its eigenvalues have the closed form $k(1-\cos(n\pi/5))$ with $k = 1/h^2 = 25$ — which reproduces all four numbers above exactly ($25(1-\cos 36^\circ) = 4.7746$). The discretisation error is real, known, and second order in h: refine the grid and it falls off as h^2.

Note the row shapes. Each row here has four entries, so it lexes as a *quaternion* rather than a list — the bracket literal is overloaded (see §4.1). It works anyway, because every bracket shape is accepted where a numeric list is wanted. You should not have to think about quaternions to type a matrix.

The rest of the family is there to be checked against its own identities, which is the honest way to use a special-function library you did not write:

```
In[5]:= let t = 0.7
Out[5]= t set
In[6]:= chebyshev_t(5, cos(t)) - cos(5 * t)
Out[6]= -0.00000000000000011102230246251565
In[7]:= sph_j(0, 1.3) - sin(1.3) / 1.3
Out[7]= 0
In[8]:= legendre_p(3, 0.4) - 0.5 * (5 * 0.4 * 0.4 * 0.4 - 3 * 0.4)
Out[8]= 0.00000000000000011102230246251565
In[9]:= bessel_j_array(4, 2)
Out[9]= [0.22389077914123567, 0.5767248077568734, 0.35283402861563773,
    0.12894324947440208, 0.03399571980756844]
```

`In[6]` is $T_n(\cos\theta) = \cos n\theta$; `In[7]` is $j_0(x) = \sin x/x$; `In[8]` is $P_3(x) = (5x^3 - 3x)/2$. All three land at zero or one part in 10^{16} — a single rounding step. `In[9]` returns the whole table $J_0 \dots J_4$ in one call, which is what a Bessel-expanded propagator actually needs.

Finally, the thing the library refuses to do:

```
In[10]:= hermite_h(2.5, 1)
Err[10]: hermite_h(): argument 1 must be a whole number (an integer order), got 2.5
```

There is no Hermite polynomial of order 2.5. The alternative — silently computing H_2 — would hand you a plausible number with no indication anything went wrong, and you would carry it through the rest of the calculation.

9.16 Example 16 — quantum mechanics: bound states, then tunnelling

The `QM` family solves an actual quantum problem in the notebook. Start with the harmonic oscillator, whose answer everyone knows: with $\hbar = m = \omega = 1$ the spectrum is $E_n = n + \frac{1}{2}$. The potential is an ordinary user function.

```
In[1]:= def v(x) { 0.5 * x * x }
Out[1]= function v(1 parameter(s)) defined -- 1 body line(s)
```

```
In[2]:= qm grid -8 8 250
Out[2]= grid [-8, 8] with 250 interior points, h = 0.063745 (potential and psi cleared)
In[3]:= qm potential v
Out[3]= potential ‘v‘ sampled at 250 points, V in [0.000507928445580228,
    31.49207155441977] (psi cleared)
In[4]:= qm states 4
Out[4]= 4 lowest bound state(s):
  E[0] = 0.499872985615
  E[1] = 1.499364798834
  E[2] = 2.498348101796
  E[3] = 3.496822505539
```

0.4999, 1.4994, 2.4983, 3.4968 against an exact 0.5, 1.5, 2.5, 3.5. The error grows with n — that is not noise, it is the three-point stencil: higher states oscillate faster and a coarse grid resolves them less well, with the error tracking $2n^2 + 2n + 1$.

Now the sharpest test available. A stationary state is *stationary*, so loading one and propagating it must change nothing:

```
In[5]:= qm state 1
Out[5]= psi = bound state 1, E = 1.499364798834, t reset to 0
In[6]:= qm energy
Out[6]= 1.499364798833686
In[7]:= qm run 4 steps 400
Out[7]= t = 4 (400 step(s) of dt = 0.01), <E> = 1.499364798834, norm drift = 4.219e-15
In[8]:= qm energy
Out[8]= 1.499364798833692
```

Four hundred Crank–Nicolson steps moved the energy by 6×10^{-15} — one bit — and the norm by 4×10^{-15}. The Cayley operator is unitary for *any* time step, so that drift measures the linear solver rather than `dt`; and the energy holding still couples the eigensolver to the propagator, so it would break if either were wrong or if they disagreed about the Hamiltonian.

Now tunnelling, the problem with no classical analogue. A packet of central energy $E_0 = k_0^2/2 = 2$ is fired at a barrier of height 2.5 — *above* its energy, so classically nothing gets through:

```
In[1]:= qm grid -100 100 2000
Out[1]= grid [-100, 100] with 2000 interior points, h = 0.099950 (potential and psi
    cleared)
In[2]:= qm potential barrier 2.5 0 1
Out[2]= potential ‘barrier 2.5 on [0, 1]‘ sampled at 2000 points, V in [0, 2.5] (psi
    cleared)
In[3]:= qm packet -25 2 2
Out[3]= psi = Gaussian packet at x0 = -25, sigma = 2, k0 = 2, t = 0
In[4]:= qm run 30 steps 3000
Out[4]= t = 30 (3000 step(s) of dt = 0.01), <E> = 2.023971780446, norm drift = 3.018e
    -13
In[5]:= qm prob 1 100
Out[5]= 0.327563019391693
In[6]:= qm prob -100 0
Out[6]= 0.672436408645103
```

33 % of the particle got through a barrier it did not have the energy to cross. The two channels add to 1.000000 — nothing was lost, and nothing reached a wall (there was no warning). The domain is 200 wide precisely so the reflected packet has nowhere to bounce from within $t = 30$.

Note the grid sizes. Bound states used 250 points because that calculation diagonalises a dense matrix and costs $O(n^3)$; scattering used 2000 because propagation is $O(n)$ per step and can afford the resolution. Choosing both to be the same would waste one or starve the other.

9.17 Example 17 — tunnelling, with the barrier as a user function

Comparison operators exist so that a piecewise potential can be an ordinary function. This is the canonical 1-D quantum problem written that way, and it is the notebook `dynamic_notebooks/tunneling.posim`.

```
In[1]:= def barrier(x) { 2.5 * (x > 0) * (x < 1) }
Out[1]= function barrier(1 parameter(s)) defined -- 1 body line(s)
In[2]:= barrier(-1)
Out[2]= 0
In[3]:= barrier(0.5)
Out[3]= 2.5
In[4]:= barrier(2)
Out[4]= 0
```

`(x > 0) * (x < 1)` is 1 inside the interval and 0 outside — an indicator function built from arithmetic on truth values.

A packet of central energy $E_0 = k_0^2/2 = 2$ is fired at a barrier of height 2.5, *above* its energy:

```
In[5]:= qm grid -100 100 2000
Out[5]= grid [-100, 100] with 2000 interior points, h = 0.099950 (potential and psi
    cleared)
In[6]:= qm potential barrier
Out[6]= potential ‘barrier‘ sampled at 2000 points, V in [0, 2.5] (psi cleared)
In[7]:= qm packet -25 2 2
Out[7]= psi = Gaussian packet at x0 = -25, sigma = 2, k0 = 2, t = 0
In[10]:= qm run 30 steps 3000
Out[10]= t = 30 (3000 step(s) of dt = 0.01), <E> = 2.023971780446, norm drift = 3.018e
    -13
In[11]:= qm prob 1 100
Out[11]= 0.327563019391693
In[12]:= qm prob -100 0
Out[12]= 0.672436408645103
```

33 % crossed a barrier it had no classical right to cross, and the two channels account for all but 10^{-6} of the probability.

Note that `qm potential barrier` picked the *user's* function, not the built-in shape of the same name. A bare name is always yours; the built-in needs arguments.

Watching it happen

```
In[14]:= qm animate "scatter.html" 32 frames 140
Out[14]= wrote scatter.html -- 140 frames over t = 32 (dt = 0.011429, 667 points per
    frame), worst norm drift 5.822e-13. Open it in a browser.
```

A self-contained page — nothing fetched from the network — showing $|\psi|^2$ against x with the potential overlaid, play/pause, a scrub bar, and a live norm and transmitted readout computed in the browser. Its final transmitted value is 0.327562 against the notebook's 0.327563: two independent implementations of the same integral.

The same answer on half the domain

```
In[15]:= qm grid -45 45 1350
In[17]:= qm absorb 15 3
Out[17]= absorbing edges: width 15, strength 3, power 2. Propagation is NO LONGER
    unitary.
In[19]:= qm run 20 steps 2000
Out[19]= t = 20 (2000 step(s) of dt = 0.01), <E> = 2.027672292154, norm drift = 1.056e
    -4
In[20]:= qm prob 1 30
Out[20]= 0.327689667523621
```

0.327690 against 0.327563 — four parts in ten thousand, on a domain **less than half the size**.

A double barrier, and an honest negative result

```
In[24]:= def double(x) { 2.5 * ((x > 0) * (x < 1) + (x > 3) * (x < 4)) }
In[31]:= qm prob 4 100
Out[31]= 0.271498902829932
```

Two barriers with a gap form a resonant cavity. At this energy it transmits **0.2715, less than the single barrier's 0.3276**, with about 0.8 % of the probability left in the gap. That trapped fraction *is* the cavity; but resonant enhancement occurs only at its quasi-bound energies, and those peaks are narrower than this packet's momentum spread ($\sigma = 2$ gives $\Delta k = 0.25$), which averages straight over them. A scan over k_0 from 1.4 to 2.9 rises monotonically with no peak. See `TUNNELING_RESULTS.md`.

9.18 Example 18 — the double slit, in two dimensions

The canonical 2-D quantum problem, and the notebook `dynamic_notebooks/double_slit.posim`. The wall is an ordinary user function of two arguments, built entirely from comparisons:

```
In[1]:= def openings(y) { (y > 1.5) * (y < 2.5) + (y > -2.5) * (y < -1.5) }
In[2]:= def slit(x, y) { 60 * (x > 0) * (x < 0.4) * (1 - openings(y)) }
In[3]:= slit(0.2, 0)
Out[3]= 60
In[4]:= slit(0.2, 2)
Out[4]= 0
```

```
In[8]:= qm2 grid -12 24 450, -16 16 400
In[10]:= qm2 absorb 4, 8
In[11]:= qm2 packet -4.5, 0, 1.2 4, 8 0
In[12]:= qm2 energy
```

```
Out[12]= 31.008336923103123
In[15]:= qm2 run 2.6 steps 800
Out[15]= t = 2.6 (800 ADI step(s) of dt = 0.00325), <E> = 30.990650438623, norm drift =
     7.796e-1
In[18]:= qm2 norm
Out[18]= 0.220410865359174
```

Those three lines together say something worth pausing on: **78 % of the probability was absorbed at the walls, and the energy moved by 0.06 %**. The absorber removed the reflected wave — the wall is 60 high against a packet of energy 32, so most of it bounces — without touching the energy of what remains. $\langle E \rangle$ is $\langle \psi | H | \psi \rangle / \langle \psi | \psi \rangle$, divided by the norm on purpose; undivided it would have fallen with the norm and looked like an energy leak.

The fringes, counted in bands on a screen at $11 < x < 19$:

band	P	expected
0 ± 0.5	**0.01577**	central maximum
0.5–1.5	0.00452	first minimum, $y = 1.46$
2.5–3.5	**0.01742**	first maximum, $y = 2.96$
3.5–4.5	0.00471	second minimum, $y = 4.56$
5.5–6.5	**0.01399**	second maximum, $y = 6.32$

With slit separation $d = 4$ and $\lambda = 2\pi/k = 0.785$, maxima sit at $\sin\theta = m\lambda/d$ with the screen $L \approx 14.8$ from the slits, giving 2.96 and 6.32. **Every predicted maximum and minimum lands in the right band.**

Two corrections got there. A first attempt used $k = 4$, $d = 2$, putting the first maximum at 52° — off the screen, so the scan showed a featureless lobe. A second launched the packet *inside* the absorber, and 86 % of it was eaten before reaching the slits.

```
In[28]:= qm2 animate "double_slit.html" 3 frames 100
```

A self-contained heat-map page. Its own in-page analysis of the final frame finds fringe maxima at $y = 0.04$ and $y = 3.23$, matching the band scan and the theory independently of the Rust that produced it.

10 Error message tour

you type	you get
`get obj7.mass` (no obj7)	`no object obj7`
`get obj0.bogus`	`unknown object field ‘bogus’ -- see HELP for the field list`
`[1,0,0] * [0,1,0]`	`cannot multiply vec3 by vec3 (for vec3*vec3 use dot()/cross())`
`set = 3`	`parse error at column 5: expected a path root (‘objN‘ or ‘system‘), found ‘=‘`
`run 1` under SPRK with $B \neq 0$	`SPRK method requires a separable Hamiltonian: magnetic field B must be zero ... use METHOD ADAMS or BDF`
`bogusname`	`unknown name ‘bogusname‘ (define it with LET, pass it as a function parameter, or use ‘bogusname.field‘ for a registered object)`
`get d.m1` (only `ball` is registered)	`no object named ‘d‘ (registered names: ["ball"]; NEW ... AS <name> creates one; positional paths are objN.field, contactK.field, system.field)`
`new sphere as d` (name taken)	`the name ‘d‘ already refers to obj0 -- DEL it or pick another name`
`new sphere as system`	`‘system‘ is reserved`
`show nosuch`	`no user function ‘nosuch‘ -- FUNCS lists the defined ones`

Every error names the column or the exact field/feature at fault, and never aborts the session.

11 The numerical engine underneath (SUNDIALS 7.8.0)

You never name a solver library when you type `STEP` or `RUN` — but it is worth one page knowing what answers you.

SUNDIALS is Lawrence Livermore National Laboratory's suite of differential-equation solvers, in production use in physics codes since the 1990s. `sundials_rs/` in this repository is a *pure-Rust translation of SUNDIALS 7.8.0*: no `unsafe`, no FFI, no C compiler, no crates.io dependency. It is vendored byte-for-byte from `once-ere/SUNDIALS_7_8_Rust_port_for_Linux@780b916`; the upgrade from the 7.7.0 engine the predecessor repository used is recorded, with its evidence, in `PORT_7.8.0_PROVENANCE.md`. This repository is the macOS / Apple Silicon port; the evidence that the same engine reproduces the Linux physics byte for byte on this platform is in `VERIFICATION_MACOS.md`.

11.1 Which solver runs which command

you type	what runs	from
`METHOD ADAMS` (default)	CVODE, variable-order variable-step Adams–Moulton, Newton iteration, dense difference-quotient Jacobian	`cvode_rs`
`METHOD BDF`	CVODE BDF — the stiff-problem family	`cvode_rs`
`METHOD SPRK` *table* [*dt*]	ARKODE SPRKStep, symplectic partitioned Runge–Kutta at a fixed step	`arkode_rs`
`COLLIDE ON` with a collidable pair	the same solver with *rootfinding* armed on the pairwise signed separations; a downward zero crossing *is* the moment of impact	`CVodeRootInit`

There is deliberately no fourth option. Every trajectory this simulator prints came out of an error-controlled or symplectic solver; the legacy Euler and Verlet steppers were not carried over.

11.2 What "rootfinding" buys you

A naive collision check samples the geometry once per output frame and asks "are they overlapping now?". If the ball moved farther than the wall is thick, the answer is no on both sides and the ball passes through. Rootfinding asks a different question: it hands the solver a continuous function — here, the signed gap between two surfaces — and the solver *interpolates back to the instant that function crossed zero*. The bounce happens at the exact time of impact, not at the next frame boundary.

```
In[1]:= new sphere { mass = 1, radius = 0.5, position = [0, 4.5, 0] }
In[2]:= new cuboid { mass = 1, half_extents = [10, 0.05, 10],
                     position = [0, -0.05, 0], inverse_mass = 0 }
In[3]:= set system.uniform_gravity = [0, -10, 0]
In[4]:= collide on
In[5]:= run 1 steps 100
Out[5]= t = 1 (209 solver steps, 100 snapshots, |dE/E| = 2.890e-11,
        1 collision(s) - CONTACTS lists them)
In[6]:= contacts
Out[6]= contact0: obj0 <-> obj1 at t = 0.8944271909999157
  point  = [0, 0, 0]
  normal = [0, -1, 0]  (from obj0 toward obj1)
  approach speed = 8.94427190999916, impulse = 17.88854381999832
```

Work out the answer by hand. The plate's top face is at $y = 0$ (centre -0.05, half-thickness 0.05); the ball's centre starts at 4.5 and its radius is 0.5, so the surfaces meet after the centre falls 4.0. Under $g = 10$ that is $t = \sqrt{2 \cdot 4/10} = \sqrt{0.8}$:

```
analytic  sqrt(0.8) = 0.8944271909999159
measured            = 0.8944271909999157
relative error      = 1.24e-16
```

One part in 10^{16} — a single unit in the last place of a 64-bit float. Nothing about the output cadence produced that: the run asked for 100 snapshots, and none of them falls anywhere near 0.894…. The solver interpolated to the crossing.

11.3 All six families, and what reaches each

crate	solves	reached by
`cvode_rs`	ODE initial-value problems (Adams / BDF)	`METHOD ADAMS`, `METHOD BDF`
`arkode_rs`	Runge–Kutta: explicit, implicit, IMEX, multirate, symplectic	`METHOD SPRK`
`ida_rs`	differential-algebraic systems $F(t, y, \dot{y}) = 0$	`CONSTRAIN` + `METHOD IDA`
`kinsol_rs`	nonlinear algebraic systems, Anderson acceleration	`EQUILIBRIUM`
`cvodes_rs`	CVODE plus forward and adjoint sensitivity analysis	`SENSITIVITY`
`idas_rs`	IDA plus sensitivity analysis	`SENSITIVITY` while constrained

Every one of them is diffed against the upstream C reference output — see `sundials_rs/VERIFICATION.md`. The last four arrived with the 7.8.0 engine and were wired into the language afterwards; the shape of that wiring is in ARCHITECTURE.md §3.9.

12 Constrained dynamics, equilibrium and sensitivity

Everything so far answers *"what happens next?"*. Three more commands ask different questions, each answered by a different solver.

you type	question	solver
`STEP` / `RUN`	what happens next?	CVODE / ARKODE
`CONSTRAIN` + `METHOD IDA` + `RUN`	...with this geometry held exactly	IDA
`EQUILIBRIUM`	where does it come to rest?	KINSOL
`SENSITIVITY`	how much does the answer depend on an input?	CVODES / IDAS

12.1 The four joints

command	rows	holds	freedoms left
`CONSTRAIN` *a b* [*len*]	1	a fixed distance	5
`BALL` *a b*	3	a shared point	3
`UNIVERSAL` *a b u w*	4	a shared point, two shafts square	2
`HINGE` *a b axis*	5	a shared point and an axis	1 (the swing)

The last three grip *orientation*, which is why `METHOD IDA` carries the full 13-numbers-per-object state rather than positions alone. **The pivot is the midpoint of the two bodies as they stand when you make the joint**, carried into each body's frame — the same "freeze what you have" rule a bare `CONSTRAIN` follows, so the joint is satisfied the instant it is made.

A door is an anchor, a slab and a hinge:

```
In[4]:= new sphere as jamb { mass = 1, radius = 0.02,
```

```
                              position = [0, 0, 0], inverse_mass = 0 }
In[5]:= new cuboid as door { mass = 1, half_extents = [0.2, 0.4, 0.2],
          position = [0.0199986666933331, -0.9998000066665778, 0] }
In[6]:= hinge jamb door [0, 0, 1]
In[9]:= run 1 steps 10
Out[9]= t = 1 (70 solver steps, 10 snapshots, |dE/E| = 1.613e-9)
In[10]:= constraints
Out[10]= constraint0: hinge obj0 <-> obj1, 5 row(s)
worst |g| = 2.73750133672479e-10, worst |g_dot| = 2.3769240437118873e-9
In[11]:= get door.angular_momentum
Out[11]= [0, 0, 0.003742219622967182]
```

The door swings about *z and nothing else* — the two rows a hinge has over a ball joint are exactly the ones that forbid the other two axes — and the joint is held to 2.7×10^{-10} after 70 steps. A hinged rigid body is a *compound* pendulum, with period $T = 2\pi\sqrt{I_{\text{pivot}}/(mgd)}$ and $I_{\text{pivot}} = I_{\text{com}} + md^2$; the simulator reproduces it to about 3×10^{-8} of a full swing.

A joint constrains velocity, not just position. A ball joint says the two bodies share a point, so at the velocity level it says $v_i + \omega_i \times r_i = v_j + \omega_j \times r_j$ — a body turning about a pivot offset from its centre *must have its centre moving.* Give it a spin and leave its velocity at zero and the state is not on the constraint manifold at all. The run **projects** the starting velocities onto it — the smallest mass-weighted change that satisfies the joint, which is exactly the impulse a real coupling delivers when clutched onto a spinning shaft — and reports how big that change was. An already-consistent state is left exactly alone. (A rod has no angular Jacobian, so spin never enters its $\dot{g}$; that is why rods carried spinning bodies from the start and hid this.)

One real limit. Orientation joints carry a tolerance floor of `rtol` $= 10^{-6}$, because the differential-algebraic system a hinge produces is *index 2* and has an accuracy ceiling no tolerance can push past — measured across twelve compound pendulums, 10^{-6} converges in every one and 10^{-8} in none. `RUN` reports when the floor was applied.

`BALL`, `HINGE` and `UNIVERSAL` are *contextual* keywords — commands only at the start of a line — so `new sphere as ball` and `get ball.mass` keep working.

12.2 `CONSTRAIN` — a rod, not a spring

```
CONSTRAIN <a> <b> [length]   rigid rod between two objects
CONSTRAIN OFF                drop every rod
CONSTRAINTS                  list them, and how well they are held
```

`<a>` and `<b>` are objects, written either positionally (`obj0`) or by a name registered with `NEW ... AS`. **With no length the rod freezes the separation the two bodies already have**, which is always consistent and is what you almost always want.

You could model a rod as a very stiff spring. That is not the same thing: a stiff spring is an approximation that vibrates, needs a tiny step, and still lets the length wander. A constraint is an *algebraic equation* the motion must satisfy exactly. Adding one changes the problem from an ODE into a **differential-algebraic equation**, and a constrained system refuses every method but IDA, by name:

```
In[7]:= run 1 steps 5
```

```
Err[7]: this system has 1 rigid constraint(s), which only the DAE
        integrator can hold: use METHOD IDA (or remove them with
        CONSTRAIN OFF). The current method is Adams
```

A body with `inverse_mass = 0` is an anchor. It never moves and it absorbs the rod's reaction. Anchor + bob + rod is a pendulum.

What is held, and how well. `CONSTRAINTS` reports the worst $|g|$ (how far the rod is from its length) and $|\dot{g}|$ (how fast that is changing). Both stay at roundoff for the whole run, because the formulation carries them *both* as equations. A cheaper scheme that constrains only the acceleration lets g drift quadratically, and by the time you notice, the answer is quietly wrong.

Scope. Constraints act on positions. A spinning rigid body, or an external torque, is refused with a message naming it.

12.3 `EQUILIBRIUM` — where does it come to rest?

Finds a configuration where every free body has zero net force, every anchor is where it started, and every rod is the right length; moves the bodies there and stops them. `system.time` is untouched — this is not an integration.

```
In[7]:= equilibrium
Out[7]= equilibrium found in 17 Newton iteration(s), 67 residual
        evaluation(s); largest net force on any free body =
        7.459152323898993e-13, worst |g| = 1.9317880628477724e-14
```

The starting guess is wherever the bodies already are, so it finds *the nearby* rest state. **It says nothing about stability** — a pencil balanced on its point is an equilibrium. Perturb the answer and `RUN` if you need to know. And a system where every body is free has no isolated equilibrium at all (translate it and nothing changes); the refusal says exactly that.

12.4 `SENSITIVITY` — how much does the answer depend on the input?

```
SENSITIVITY <t> "<param>" ["<param>" ...]
```

Runs for `<t>` *and* reports $\partial(\text{state})/\partial p$ for each parameter. Names are quoted strings because `mass 0` is two tokens: `"g_constant"`, `"mass 0"`, `"charge 1"`, `"gravity.y"`, `"e_field.x"`, `"b_field.z"`.

Running twice with slightly different inputs and subtracting is the difference of two nearly equal numbers and loses most of its digits. This integrates the derivative *alongside* the state instead. CVODES normally; IDAS when the system is constrained, because only then are the equations a DAE.

Free fall gives $y(T) = \frac{1}{2}gT^2$, so $\partial y/\partial g = T^2/2$, which at $T = 3$ is exactly 4.5:

```
In[15]:= sensitivity 3 "gravity.y" "mass 0"
Out[15]= t = 3 (CVODES, 129 solver steps)
d/d(gravity.y):
  obj0 position [0, 4.500000056696235, 0]
d/d(mass 0):
  obj0 position [0, 0, 0]
```

The second answer is worth a moment. In uniform gravity every mass accelerates equally, so the trajectory does not depend on the mass **at all** — and the derivative comes back as exactly zero, not as a small number. That is the difference between a real sensitivity calculation and a finite difference.

12.5 The one rule that changed for anyone reading the Rust

The 7.8.0 translation models C's opaque pointers directly. An `N_Vector` is a handle, and its payload is reached through `N_VGetArrayPointer`, which returns a *borrow guard*:

```
let y = N_VNew_Serial(neq, &sunctx).ok_or("N_VNew_Serial returned NULL")?;
{
    let mut d = N_VGetArrayPointer(&y).expect("serial vector");
    d[0] = 1.0;
}                        // <- guard dropped here, ON PURPOSE
CVode(&cvode_mem, tout, &y, &mut t, CV_NORMAL);
```

Holding that guard across the `CVode` call would be a live borrow of a vector the solver is about to write. `physical_object/src/integrate.rs` therefore wraps every access in `with_data` / `with_data_mut`, which take the guard, do the work and drop it. Nothing about the command language changes.

13 Browser videos: watching a run after it has finished

`SCENE CREATE` (§5.6) gives you a *live* window: it needs a running posim behind it. The other thing you often want is a file — something you can mail to a colleague, open next year on a laptop with no Rust toolchain, and still scrub frame by frame.

`tools/record_video.py` makes one:

```
cargo build --release -p posim          # once
recorder/src/record_video.py videos/scenes/kepler_ellipse.posim \
     -o videos/kepler_ellipse.html \
     --frames 360 --dt 0.02 \
     --title "Kepler orbit, e = 0.6" \
     --caption "CVODE Adams, 360 frames of dt = 0.02."
```

13.1 How it works, and what it does not do

1. It starts `posim -machine` — the same JSON protocol JupyterLab uses (§7).
2. It runs your setup script one line at a time.
3. Then it loops: ask for `{"op":"state"}`, keep the answer, send `step <dt>`, repeat.
4. It writes one HTML file with those frames embedded and a plain canvas player around them.

Every advance in step 3 is a real SUNDIALS step. The tool has no integrator of its own; it is a camera, not a physics engine. And the page it writes fetches nothing: no CDN, no font server, no analytics. Open it with `file://` on a machine with no network and it works.

13.2 The player

control	does
Play / Pause, or Space	run the recording
◄◄ / ►►, or ← →	one frame back / forward
scrub bar	jump anywhere
speed	0.25× to 4×
drag	orbit the camera
wheel, or + / −	zoom
↑ ↓	pan
Reset view	back to the framing it opened with
trails / labels / contacts	motion trails, body names, gold contact-normal arrows

The readout in the corner is live per frame: the frame number, t, the total energy, $|P|$, $|L|$, the running collision count, and the method and dt the recording was made with. **This is the point of the format**: you can stop on the frame where something looks wrong and read the conserved quantities off it.

13.3 The thirteen shipped recordings

file	what to watch for	measured
`videos/kepler_ellipse.html`	the bead swinging between perihelion and aphelion on an $e = 0.6$ ellipse	$\lvert dE\rvert/E$ = 9.8e-8, $\lvert dL\rvert/\lvert L\rvert$ = 1.3e-7
`videos/tumbling_racket.html`	the Dzhanibekov flip: a torque-free cuboid spun about its *intermediate* axis turns over, and over	$\lvert d\lvert L\rvert\rvert/\lvert L\rvert = 0$ **exactly**; $\lvert dE\rvert/E$ = 6.4e-9
`videos/box_of_shapes.html`	a cylinder, a disk and a cuboid rattling in a rigid `BOX` 4; gold arrows are the analytic contact normals, sized by impulse	36 collision events, $\lvert dE\rvert/E$ = 3.4e-16
`videos/double_pendulum_hinge.html`	two `HINGE` joints assembled into the chaotic linkage; the gold rings are the joints	the joints hold to $\lvert g\rvert$ = 5.6e-8; energy wanders 3 parts in 10,000
`videos/universal_joint.html`	a `UNIVERSAL` joint carrying a driven shaft's rotation to a second shaft; the bend flattens out straight and folds back, and the speed across the joint swings with it	the bend stops at $\cos\beta$ = 0.6000004 against a geometric bound of exactly 0.6; the three joints hold to $\lvert g\rvert$ = 4.0e-7
`videos/ball_joint_chain.html`	four links on `BALL` joints, whirling as they collapse; the chain leaves the plane it started on, which a hinged chain cannot	the four joints hold to $\lvert g\rvert$ = 3.3e-9; $\lvert z\rvert$ runs from exactly 0 to 1.7147
`videos/rod_pendulum_chain.html`	four `CONSTRAIN` rods, the cheapest linkage there is at one row each, going chaotic	run continuously at the default tolerance the rods hold to $\lvert g\rvert$ = 5.4e-15; this recording, 250 cold restarts, holds $\lvert g\rvert$ = 7.8e-8
`videos/spinning_top.html`	a top held at its tip by a `BALL` joint, precessing under gravity	precesses at 1.020440 rad/s against a closed form of 1.020408, three parts in 100,000, without nutating
`videos/gyroscope_gimbal.html`	a body hung in two gimbal rings on three perpendicular `HINGE` axes; the push goes in about one axis and comes out about another	total $L \cdot \hat{y}$ conserved to 6.4e-15; no centre moves further from the pivot than 7e-35
`videos/cardan_gear.html`	a wheel inside a ring of twice its radius, rolling on a `GEAR` row: the rim point runs along a **straight line**, the degenerate hypocycloid	the line is held to 1.1e-8, against 1.8e-3 for the same mechanism with the ratio merely imposed
`videos/rack_and_pinion.html`	a weight on a `RACK` winding up a flywheel, guided by a `PRISMATIC`	the rack falls at exactly $g/2$, and at the same rate for two different pitch radii
`videos/piston_crankshaft.html`	the slider-crank: `HINGE` + two `BALL`s + `PRISMATIC`, free-running	follows $x = a\cos\theta + \sqrt{L^2 - a^2\sin^2\theta}$ to 8.4e-8; stroke exactly $L - a$ to $L + a$
`videos/cardan_compass.html`	the same two rings, but with a **pendulous** bowl, so gravity is the restoring torque and the card seeks level	two physical-pendulum periods, 1.878587 and 2.307339 s, measured

13.4 A trap worth knowing about

The box recording sets `system.g_constant = 0` on its first line, and that line is not decoration. posim's default G is 1, so three bodies rattling inside a small box also *attract each other*, and the softened pairwise force at `softening = 1e-6` is very nearly singular when two surfaces touch. With gravity left on, the same scenario drifts **3.2 %** in energy through 25 events; with $G = 0$ it holds to 5.1e-16 through 36. Neither number is a solver defect — they are different physical systems.

The lesson generalizes: **a conservation claim carries its system settings with it.** When a run's $|dE/E|$ surprises you, check `ENERGY`, `system.g_constant`, `system.softening` and `system.uniform_gravity` before suspecting the integrator.

13.5 What a BALL joint buys, measured exactly

`BALL` and `HINGE` are easy to describe and easy to confuse: both hold a point, and a hinge additionally holds an axis. The ball-chain recording turns that sentence into a number.

Take four links laid end to end and start them as a **rigid rotation** of the whole assembly about the vertical, $v = \omega \times r$ with $\omega = [0, 1.5, 0]$, so $v = [0, 0, -1.5x]$. A rigid motion moves nothing relative to anything, so it violates no joint of any kind — as a *position* constraint. The velocity constraint is where the two joints part company:

the same four links, joined by	**worst $\lvert g\rvert$**	**worst $\lvert\dot{g}\rvert$**
four `BALL` joints	0 exactly	**0 exactly**
four `HINGE` joints about z	0 exactly	**1.5**

1.5 is not approximately anything. It is Ω, the whirl rate, because the whirl is precisely the component a hinge about z forbids, and the velocity residual of a forbidden motion is its own magnitude.

What follows from it. A start with $|\dot{g}| \neq 0$ is not on the constraint manifold, so before integrating anything the solver must project the velocities onto it, and that projection *changes the motion you asked for*. The hinge chain cannot be given this whirl; it can only be given whatever remains after the whirl is removed. The ball chain takes it untouched, and `initial_velocity_projected` is 0.

Watch the recording's z axis for the consequence. Every link starts exactly on the plane $z = 0$, and the chain reaches $|z| = 1.7147$. A hinged chain would still be at exactly zero, for ever, because that is what fixing an axis means.

13.6 One number decides what a suspension does

Three of the recordings hang a rotor in a mount, and the only thing that really differs between them is where the centre of mass sits relative to the pivot. Read them as one experiment with one dial.

	`spinning_top`	`gyro_gimbal`	`cardan_compass`
mount	one `BALL` at the tip	three `HINGE`s	two `HINGE`s
rows	3 on 6 freedoms	15 on 18	10 on 12
centre of mass	$r = 0.6$ **from** the pivot	**on** the pivot	$d = 0.12$ **below** it
gravity	lever arm r: a torque	lever arm 0: none	lever arm d: a *restoring* torque
result	steady precession $Mgr/(I_3\omega_3)$	no gravitational precession	swings back to level, $2\pi\sqrt{I/Mgd}$

All three carry a `uniform_gravity`. In the middle column it does nothing whatever, and that is the point rather than an oversight: a lever arm of zero produces a torque of zero, so a gimballed gyroscope is not balancing against gravity, it is free of it. Move the centre of mass off the pivot and gravity comes back — *above* the pivot it would tip the thing over, *below* it, it rights it. That last case is the whole of a ship's compass: it is not held level by its bearings, it is held level by its own weight.

The compass periods. With the bowl pendulous each axis is an ordinary physical pendulum. In pitch only the bowl swings, giving $I = M(3R^2 + 4h^2)/12 + Md^2 = 0.21046667$ and $T = 1.878587$ s; in roll ring and bowl swing together, $I = 0.28574980$ against a restoring $(M_b - M_r)gd$, giving $T = 2.307339$ s. Measured: 1.883426 and 2.313653. The 2.8×10^{-3} excess is not solver error — halve the kick and it falls by 4.01 and 4.02, the signature of a term in θ^2. Drive one axis alone and the attribution is exact: the pitch period comes out 4.99×10^{-4} long against a predicted $\theta^2/16 = 5.02 \times 10^{-4}$.

Both rings lie flat, and that is not a styling choice. A gimbal ring pivots about a **diameter** — a line in its own plane — so its plane has to contain both hinge axes. Orient it instead so its symmetry axis *is* the hinge axis and "swinging" becomes a spin about its own axis of revolution: a torus turned that way neither shows anything nor does anything, and the roll inertia silently becomes the axial $M_r(c^2 + 3a^2/4)$ instead of the diametral $M_r(c^2/2 + 5a^2/8)$. That is a factor of two in the ring's contribution, and it moves the roll period from 2.307339 to 2.582727 s. The recording caught it: the ring sat there visibly inert while the bowl rocked.

A cost of the midpoint pivot. Two hinges sharing one point force $p_{\text{frame}} = p_{\text{bowl}} = -p_{\text{ring}}$, so hanging the bowl d below the centre puts the *ring's* centre d above it. That is not cosmetic: the ring is then top-heavy and **subtracts** from the restoring torque, which is why the roll period carries $(M_b - M_r)$ and not $(M_b + M_r)$, and why the ring is made light.

And a hinge cannot torque about its own axis. That is precisely the freedom it grants — but read the other way round it is a conservation law, and the gimbal has two. The outermost hinge turns about the vertical, so nothing can torque the system about the vertical, and the total $L \cdot \hat{y} = 0.52688$ is conserved to 1.4×10^{-14}. The innermost hinge's axis *is* the rotor's spin axis, so nothing can torque the rotor about it either, and its axial spin $I_3\omega_3 = 3.75$ is conserved to 1 part in 100,000.

The first is held to machine precision rather than to solver tolerance, and the reason is structural: angular momentum is integrated *state* in the 13N packing, not a quantity derived afterwards from positions. The second involves the orientation as well, so it inherits the tolerance instead.

A gimbal holds a point, exactly. All four bodies are concentric, so each hinge's midpoint pivot lands on the same origin and the three axes meet there — which is what makes this a gimbal

rather than a linkage. Over the whole recording no centre moves further from that origin than 1.2×10^{-34}. The rings only *look* nested.

What to watch. The assembly starts turning about the vertical at 1 rad/s. Something without a gyroscope in it would simply keep turning. This does not: the outer ring never gets more than 7.43° from where it began, and the rotation reappears as a **tilt** — the rotor's axis swinging up to 16.33° away, at right angles to the push. Rotation in about one axis, rotation out about another, is the whole of gyroscopic behaviour, and here it is a consequence of the two laws above rather than a separate rule.

13.7 The right motion versus the right mechanism

The Cardan-gear recording exists to make one distinction concrete, and it is the distinction the `GEAR` joint was added for.

A wheel of radius r rolling inside a ring of radius $2r$ sends every point of its rim along a straight line — a diameter of the ring. The hypocycloid of ratio 2 does not approximate a line, it **is** one.

You can get that motion without a constraint. Set the carrier turning one way and the wheel the other at the same rate — which for this radius ratio is exactly the rolling condition — and both rates are conserved, so the ratio persists and the line comes out. Nothing in the picture tells you it was never enforced.

Then disturb it. The same scene, measured four ways:

	rim point off the line
ratio as an initial condition	1.795×10^{-3}
ratio enforced by `GEAR`	1.059×10^{-8}
initial condition, plus a torque on the wheel	$\mathbf{7.468 \times 10^{-1}}$
`GEAR`, plus the same torque	1.152×10^{-8}

Undisturbed, the constraint is worth five orders of magnitude. Disturbed, it is worth everything: the imposed version does not degrade, it **collapses**, and the straight line is gone.

The general point. A relation you can only set up in the initial conditions gives the right motion and the wrong mechanism. The two are indistinguishable while nothing happens, and a single disturbance separates them.

What the exercise taught. The joint set is now enough for the mechanisms this chapter set out to build — a wheel rolling *along the ground* is `PRISMATIC` for the line plus `RACK` for the rolling, a piston is `PRISMATIC`, a cam follower is `PRISMATIC` against whatever drives it. What outlasts the list is that the two hard joints were the two whose natural coordinate is an angle nobody counted, and each needed a different trick chosen by what else was to hand: `GEAR` has two angles and nothing else, so it hides the wrapping in a sine and pays with a rational ratio; `RACK` has an angle *and an unbounded distance*, so it unwraps the angle from the distance and pays nothing; `PRISMATIC` has no angle in its statement and needed neither. The difficulty was never the mechanism, but whether the constraint could be written as a function of the state — and when it could not directly, whether some other coordinate of the same mechanism could say what the state had forgotten.

13.8 A closed form that is exact, and how to actually hit it

The spinning-top recording is the one place in this chapter where the answer is a formula you can look up, so it is worth being precise about what is being checked.

A symmetric top spinning at ω_3 about its own axis, with its centre of mass a distance r from the pivot, precesses about the vertical at $\Omega = Mgr/(I_3\omega_3)$. Textbooks label this the **fast-top approximation**. The exact condition for steady precession at tilt θ from the vertical is

$$Mgr = \Omega I_3\omega_3 - I_1\Omega^2\cos\theta,$$

so the shortcut is exact whenever $\cos\theta = 0$ — a top whose axis is **horizontal**. That is why this scene mounts the gyroscope on its side, with the axis along z and gravity along $-y$: not for the look of it, but so the thing being asserted is an identity rather than a limit.

$M = 1$, $r = 0.6$, $g = 3$, radius 0.42	$I_3 = MR^2/2 = 0.0882$
$\omega_3 = 20$	$\Omega = 1.0204081632653061$ rad/s
measured over the recording	1.020440 rad/s, **3 parts in** 100,000
tilt out of horizontal	4.3×10^{-5} — it does not nutate

Hitting it takes both velocities. A steady precession is not "spin it and let go": give the top only its spin and it is not on the steady orbit at all. It dips and bobs its way round instead, and at this spin rate the *average* precession misses the formula by **7.8 %**. Steady means the whole body is already turning about the vertical through the pivot as well as spinning about its own axis, so `angular_velocity` carries $[0, \Omega, \omega_3]$ and `velocity` carries $[\Omega r, 0, 0]$. Both are rigid motions that leave the pivot point still, so $\dot{g} = 0$ exactly and nothing is projected.

The general lesson: **an initial condition has to satisfy the constraints at the velocity level, not just the position level**, and a closed form that assumes a particular steady state has to be started in that state or it is not the thing you are measuring.

13.9 A restart is not a continuation

The rod-chain recording exists to document a trap that costs an afternoon if you meet it without warning.

`CONSTRAIN` is the best-conditioned joint in the language — one well-scaled scalar equation, $g = |d| - L$, with none of the index-2 orientation coupling that forces a tolerance floor on `BALL`, `HINGE` and `UNIVERSAL`. So a rod-only system is correctly *not* floored, and runs at whatever you asked for. Four rods, five seconds, default rtol = 10^{-10}, atol = 10^{-12}, give a worst $|g|$ of 5.4×10^{-15}: roundoff. Record the same chain and it fails on the second frame, with `IDASolve` returning -4.

Nothing about the physics changed. What changed is that **a recording is not one integration.** It is one `STEP` per frame, and every `STEP` is a *cold restart* — a fresh solver, a fresh multiplier seed, no BDF history to lean on. The tolerance a continuous run sustains is not the tolerance a restart sustains, and for this chain the gap is four orders of magnitude:

four rods, five seconds	continuous `RUN`	250 `STEP`s
default $10^{-10}/10^{-12}$	$\lvert g\rvert = 5.4 \times 10^{-15}$	fails on restart 2
floor $10^{-6}/10^{-8}$	fine	$\lvert g\rvert = 7.8 \times 10^{-8}$

`BALL`, `HINGE` and `UNIVERSAL` never meet this, and now the reason is visible: they are floored to exactly $10^{-6}/10^{-8}$ *automatically*, so every orientation-joint recording has been asking for restart-

safe tolerances without knowing it. The floor those joints get for *accuracy* is also what makes them recordable frame by frame. A rod-only scene has to ask by hand.

How to recognize it. A `RUN` that succeeds and a loop of `STEP`s that fails, at the same tolerance on the same system, is this and not a physics bug. Loosen the tolerance, or do not chop the integration up.

A related sharp edge. This chain is chaotic, and its margin at the default tolerance is thin enough to be moved by *rounding the initial positions*. Written to six decimals, a five-rod version survives the continuous run; written exactly, it fails at $t = 0.43$. Four rods is robust either way, which is why the scene ships with four and writes its coordinates out in full. If a constrained run's success depends on how you rounded the input, you are on a knife edge, not on a result.

13.10 Count your rows before you brace a mechanism

The universal-joint recording is worth reading as a design exercise, because the obvious way to build it does not work.

A `UNIVERSAL` holds two things: one shared point, and one right angle between the two trunnion axes. It does *not* hold the two shafts straight. The bend between them is free — that is the whole reason the joint exists — so if the output shaft is left dangling it does not transmit anything, it just swings down past the joint like a pendulum and folds back on the input shaft at 176°.

So the output shaft has to be braced. The textbook drawing puts it in a second bearing, and a bearing is a `HINGE`. Try it and IDA quits at $t = 0$ with `retval = -4`. Count the rows and the reason is arithmetic, not numerics. Two free shafts are 12 freedoms, and

$$\underbrace{5}_{\texttt{HINGE}} + \underbrace{4}_{\texttt{UNIVERSAL}} + \underbrace{5}_{\texttt{HINGE}} = 14 \text{ rows on } 12 \text{ freedoms.}$$

Three of those rows are *redundant*: once both shafts are held in bearings whose axes meet, the shared point the universal joint asks for is already implied by the geometry. Redundant rows are not merely surplus — they make $JM^{-1}J^{\mathsf{T}}$ singular, and the constrained Newton solve has no answer to give. posim does no redundant-row elimination, so it reports the failure rather than guessing.

The fix is to brace with the *smallest thing that does the job*. A `CONSTRAIN` is one row:

$$5 + 4 + 1 = 10 \text{ rows on } 12 \text{ freedoms.}$$

Two freedoms left, full rank, and the mechanism runs. Better still, the rod fixes the bend in closed form. The output shaft's centre stays 0.3 from the cross and 0.4243 from the post, so it rides the circle where those two spheres meet, sweeping a cone of half-angle $\arctan(0.3/0.6) = 26.565°$ about a line itself 26.565° off the x axis. The bend therefore runs from 0 to 53.130° and no further:

$$\cos 53.130° = 0.6 \quad \text{exactly.}$$

Scrub the recording to the sharpest frame and the shafts are at 0.6000004. That number is never given to the integrator; it comes out because the hinge, the universal joint and the rod all held at once.

The rule. Before adding a joint, add up the rows it will bring and compare with the freedoms actually left. If the total meets or exceeds the freedoms, you have either locked the mechanism or made it singular, and a fatal return flag is the honest outcome either way.

14 SQLite datasets and movies: a complete worked session

Everything in §5.13, exercised end to end and captured verbatim from this machine (Apple Silicon, macOS). This is SolveIt worked example 20 (`scripts/solveit/20_dataset_kepler_orbit.posim`): a two-body Kepler ellipse whose perihelion and aphelion are measured BY THE DATABASE.

The setup: sun mass 3 and planet mass 1 under $G = 1$, so the relative orbit has gravitational parameter $\mu = G(M + m) = 4$. Starting at perihelion separation $r_p = a(1 - e) = 1$ with $a = 2.5$ and $e = 0.6$, the relative vis-viva speed is $\sqrt{\mu(2/r_p - 1/a)} = \sqrt{6.4}$, split about the stationary centre of mass (planet gets 3/4 of it, sun 1/4 backwards). Aphelion must come out at $a(1+e) = 4$, and one period is $T = 2\pi\sqrt{a^3/\mu} = 12.418264743622237$.

```
In[1]:= set system.g_constant = 1
In[2]:= new sphere as sun    { mass = 3, radius = 0.15, position = [-0.25, 0, 0],
    velocity = [0, -0.6324555320336759, 0] }
Out[2]= obj0 as sun
In[3]:= new sphere as planet { mass = 1, radius = 0.05, position = [0.75, 0, 0],
    velocity = [0, 1.8973665961010276, 0] }
Out[3]= obj1 as planet
In[4]:= method adams
Out[4]= method = CVODE Adams
In[5]:= dataset create solveit20_kepler
Out[5]= dataset solveit20_kepler created (datasets/solveit20_kepler.db, journal_mode=
    wal, schema v1) -- recording armed
In[6]:= run 12.418264743622237 steps 200
Out[6]= t = 12.418264743622238 (659 solver steps, 200 snapshots, |dE/E| = 2.096e-9)
In[7]:= dataset info
Out[7]= dataset solveit20_kepler: 2 bodies, 200 frames, 400 states
```

Two hundred `RUN` outputs became two hundred frames — each holding t, E, P, L, the centre of mass, the collision count, and both bodies' full states — appended through the sidecar's WAL-mode pool as one transaction. Now the physics question, asked in SQL: a **self-join** pairs the two bodies frame by frame and takes the extremes of their separation:

```
In[8]:= sql "SELECT MIN(sqrt((p.x-s.x)*(p.x-s.x) + (p.y-s.y)*(p.y-s.y))) AS perihelion,
     MAX(sqrt((p.x-s.x)*(p.x-s.x) + (p.y-s.y)*(p.y-s.y))) AS aphelion FROM states p
    JOIN states s ON s.frame = p.frame WHERE p.body = (SELECT body FROM bodies WHERE
    name = 'planet') AND s.body = (SELECT body FROM bodies WHERE name = 'sun')"
Out[8]= perihelion | aphelion
1.000000001093295 | 4.000000026379791
In[9]:= sql "SELECT MAX(ABS(comx)) AS com_x, MAX(ABS(comy)) AS com_y, MAX(energy) - MIN
    (energy) AS energy_spread FROM frames"
Out[9]= com_x | com_y | energy_spread
5.235284428195541e-12 | 5.539403137577725e-12 | 4.5303885087832896e-9
```

Perihelion 1, aphelion 4, to the solver's tolerance — the analytic ellipse, read back out of a database. The centre of mass sat still to 5e-12 and the energy ledger is flat to 4.5e-9 across the period.

The movie and the plots page are derived artifacts:

```
In[10]:= movie create solveit20_kepler
Out[10]= movie solveit20_kepler created from dataset solveit20_kepler (movies/
```

```
    solveit20_kepler.html, 200 frames, 2 bodies)
In[11]:= movie play solveit20_kepler
Out[11]= movie solveit20_kepler (movies/solveit20_kepler.html) ready (browser
    suppressed)
In[12]:= dataset play solveit20_kepler
Out[12]= dataset plots written to datasets/solveit20_kepler_plots.html (browser
    suppressed)
In[13]:= dataset close
Out[13]= dataset solveit20_kepler closed
```

(This transcript was captured under `POSIM_NO_BROWSER=1`, the batch switch; run it yourself without that variable and both `PLAY` commands open your web browser — the movie is the 3-D player described in §5.13, the plots page shows $E(t)$ flat, $|P|$ and $|L|$ constant, the two position traces, and the closed ellipse in the top view.)

Re-run the whole thing any time:

```
cargo build --release -p posim
cd dataset_tools && cargo build --release && cd ..
./target/release/posim --script scripts/solveit/20_dataset_kepler_orbit.posim
```

And the cross-dataset idiom, from SolveIt example 31 (`scripts/solveit/31_dataset_attach_compare.posim`), captured the same way: record the same free fall once under Adams and once under BDF, then let SQLite compare the two databases directly —

```
In[16]:= sql "ATTACH DATABASE 'datasets/solveit31_adams.db' AS adams"
Out[16]= ok (0 row(s) changed)
In[17]:= sql "SELECT MAX(ABS(s.y - a.y)) AS worst_method_gap FROM states s JOIN adams.
    states a ON a.frame = s.frame AND a.body = s.body"
Out[17]= worst_method_gap
5.615900633415549e-9
In[18]:= sql "DETACH DATABASE adams"
Out[18]= ok (0 row(s) changed)
```

Two independent integrators agreed on every recorded coordinate to 5.6e-9, and the referee was a SQL JOIN.